WHITE PAPER

Introduction to Cloud Compute on Arm: Design, Implementation, and Performance

Chris Shore

Arm Limited. Company 02557590 registered in England.
110 Fulbourn Road, Cambridge, England CB1 9NJ.
(PRE-1121-V1.0)

See also: Glossary| References

Contents

Part 1—Introduction to Neoverse Cores

1. Introduction

1.1. Purpose

The purpose of this resource is to introduce the Arm® Neoverse™ range of processors for developers who wish to write software for devices powered by Neoverse cores. Additionally, the material will be of some use to those wishing to design and manufacture those devices, but it doesn't contain detailed discussion of semiconductor design beyond the high-level understanding of architecture that is beneficial to software developers.

Readers should gain an appreciation of the purpose and reasoning behind major features of the Armv9 architecture and of the range of Neoverse processors. This is intended to enable readers to make informed choices when selecting a target platform for a particular use case, including tool selection and software development methodology. This can also serve as a resource for those looking to learn or teach about server computing.

1.2. Scope

This text covers an introduction to the Neoverse range of processor cores, a taxonomy of the devices currently available, major producers and users of those devices, and tools available for developing software for them. Although the book contains an evolutionary history of the Arm architecture and product range, this is not primarily a history book. Only historic features that have a bearing on current architectures will be discussed in any detail.

1.3. Audience

The primary audience for this resource is software developers or those looking to learn or teach about server computing. Much of the material will be of interest to semiconductor designers, system integrators and electronic engineers. However, they don't represent the target audience and will need to refer to further and more detailed texts to find the knowledge they require.

1.4. Summary

Beginning with an introduction to Arm, its history and current market positioning, we then outline the development of the Arm architecture from its earliest incarnations to the latest Armv9 architecture. The rest of the book concentrates on the range of processors that are produced under the Neoverse brand. The approach taken is primarily driven by the needs of software developers.

1.5. Currency

While all information presented is correct at time of publication—as far as the authors can ensure—readers must bear in mind that the semiconductor industry, including Arm, develops at an astonishing and accelerating pace. New architectures and new devices are released regularly, enabling new use cases and unlocking new target markets. The reader is advised to refer to external sources for the latest developments.

1.6. References and Related Reading

- Arm Architecture Reference Manual for A-profile Architecture (ARM DDI 0487K)
- Arm® Neoverse™ V3 Core Software Optimization Guide (PJDOC–1505342170–661452)
- Arm® Neoverse™ V2 Core Technical Reference Manual (Arm 102375_0002_03_en)
- Arm® Neoverse™ N3 Core Software Optimization Guide (Arm 109637)
- Arm® Neoverse™ V2 Core Software Optimization Guide (PJDOC–466751330–593177)
- Arm® Cortex™–A Series Programmer's Guide (ARM DEN0013D, 4th Edition)
- Arm® Neoverse™ N2 Reference Design Technical Overview (Arm 102337_0000_04_en)
- Arm® CoreLink™ GIC–700 Generic Interrupt Controller Technical Reference Manual, Document ID 101516_0400_12_en
- Arm® CoreLink™ MMU–700 System Memory Management Unit Technical Reference Manual, Document ID 101542_0001_04_en
- Arm® CoreLink™ NI–700 Network-on-Chip Interconnect Technical Reference Manual, Document ID: 101566_0203_10_en

- Arm® CoreLink™ CMN–700 Technical Reference Manual, Document ID: 102308_0302_07_en
- Arm® CoreLink™ NIC–450 Network Interface Connect Technical Overview, Document ID: 100459_0000_01_en
- Arm® DynamIQ™ Shared Unit Technical Reference Manual, Document ID: 102547_0201_07_en

1.7.Glossary

A full Arm glossary can be found on Arm's website, Document ID 105565_0200_02_en.

2. Introduction to Arm

Arm® processors are found everywhere. A couple of decades ago, the Arm ecosystem passed the milestone of producing more processors than there are human beings on planet Earth. At the time of writing, the numbers are truly staggering: in excess of 7 billion devices are shipped every quarter; a cumulative total of over 300 billion devices have been shipped since the foundation of the company in 1990; Arm has a 50% share of all electronic devices with processors; and in excess of 90% of smartphones use Arm microprocessors.

Arm, of course, does not achieve this on its own. Rather, it functions as the driving force behind a huge and growing ecosystem of partner organizations. At the heart of this partnership, Arm designs the processors themselves, constantly pushing for higher performance, wider scalability and flexibility, and greater power efficiency. It is this last attribute—power efficiency—on which Arm first made its reputation.

The meteoric growth of mobile phones through the late 1990s and early 2000s was powered by the adoption of Arm processors by almost every major manufacturer of the era. Arm's compelling combination of flexibility, processing power, and power efficiency made their processors the natural choice across the industry.

With the emergence of smartphones in around 2008, Arm again caught a rising wave of innovation and growth in the electronics industry. With very few exceptions, all the early smartphones, MP3 players, and tablets were based on Arm designs. Other businesses in the industry quickly found they could rely on Arm to produce better, faster, cheaper, and more power-efficient designs, year on year. They could then focus on their own product development. Arm became the processor outsourcing house for almost everyone in embedded electronics.

There were many other advantages to the growing breadth of the Arm ecosystem. A growing pool of engineers knew, loved, and used Arm architecture—an industry-wide ecosystem of talent, that brought economies of scale in education, tooling, and component sharing. This reduced costs and increased opportunities for all.

As well as the relentless drive for ever-greater power efficiency and scalability, Arm has worked to broaden the ecosystem in every direction. The initial markets in the deeply

embedded space, centered around mobile phones, are still hugely significant to Arm's activities, but these have been joined by other market segment such as automotive, machine learning and artificial intelligence.

As you shall see later when looking at the evolution of the Arm architecture and product range, new markets very often require new priorities for computing. The needs of a smartphone product designer are very different from the needs of an automotive engine controller, and from the needs of a server-class processor in a data center. Each use case has its own balance between cost, volume, processing capability, power efficiency, ease of manufacture, security, reliability, throughput, and other factors. These are some of the considerations which drive the choice of an appropriate platform for any new product.

For further information about Arm, please check the Arm website.

3. Development of the Arm Architecture

This chapter covers the evolution of the Arm® architecture from its earliest days through to the introduction of the Armv9 architecture. Armv9—the architecture implemented in Neoverse™ products—is the subject of the subsequent chapter, so if the history is of little interest to you, you may wish to skip straight there.

The history of the development of the Arm architecture and the accompanying range of products is, in many ways, a history of the development of electronic (specifically computing) engineering across the last forty years.

At this point though, you must note the difference between "architecture" and "processor".

The "architecture" is the contract between software and hardware. It defines how the hardware behaves when processing sequences of instructions, how it handles exceptions and error conditions, how it interfaces with memory, how caches behave, and so on. A new version of the architecture might introduce new instructions, classes of instruction, new memory behaviors, new registers, and so on.

Processors are implementations of a particular architecture. Each individual processor is designed to conform to a single version of the architecture, while each architecture might be implemented in many separate processors. So, while two processors might conform to the same architecture, and thus be "binary-compatible", they may differ considerably in the way in which that functionality is implemented, the trade-offs chosen between power consumption and compute power, the available physical memory size, cache configuration, and bus interconnect architecture, among many, many other possible variations in the physical construction of the device itself.

You should keep aware of this fundamental distinction between architecture and implementation when dealing with Arm-based compute devices.

3.1. Early Days

The first Arm processor was developed by Acorn Computers—a British manufacturer of personal computers, founded in Cambridge in 1978 by graduates from Cambridge University. In the 1980s, Acorn enjoyed considerable success in the U.K. education market after being adopted as the chosen machine for a nationwide computer literacy project sponsored by the British Broadcasting Corporation (BBC). The eponymous BBC Micro sold over 1.5 million units in its 13-year lifetime. The machine was based on the 6502 processor from MOS Technology, and by 1984 it became clear that a new processor was needed for subsequent products.

The Acorn design team evaluated a number of contemporary processors before concluding that none were suitable. They took the decision—remarkable at the time—to design their own. Sophie Wilson designed the instruction set, and Steve Furber designed the processor hardware. The first Acorn RISC Machine ran code on 26th April 1985 at Acorn's office in Cambridge, United Kingdom. Its first use was as a second processor for the BBC Micro. The first personal computer based on it—the Acorn Archimedes—was launched in 1987.

The mid–1980s was not a successful period for Acorn, though, and the company was largely broken up towards the end of the decade. The intellectual property (IP) making up the Acorn RISC Machine was spun out into a new company called Advanced RISC Machines. This was a joint venture between Acorn (who provided the technology and a number of staff), Apple (the first customer and supplier of seed finance), and VLSI (who fabricated the first devices). The company set itself the ambitious mission of becoming the "global processor standard". For its flotation on the London Stock Exchange in 1999, it renamed itself as "Arm".[1]

Apple invested in Arm in order to gain access to the processor for its Apple Newton PDA. While this product was not successful in the marketplace, it was an important milestone for Arm as a separate company with its own destiny independent of Acorn.

Arm realized that it couldn't compete with existing, established semiconductor manufacturers, and instead quickly formulated its own strategy of licensing the IP for its

1 This was changed to "Arm" as part of a rebranding exercise in 2017.

designs to partner companies, which produced chips based around the processors. Early licensees included Sanyo, Sharp, Texas Instruments, and Plessey.

The early Arm processors—Arm2 through Arm6—were moderately successful. But when Texas Instruments adopted the Arm7TDMI® processor for production of a System-on-Chip to be used in Nokia's mobile phones, it catapulted Arm to the forefront of the industry, a position which it has occupied ever since.

3.2. Rapid Evolution

Acorn Computers bequeathed the Arm3 processor to Advanced RISC Machines, when it was founded in 1990. This processor was far simpler and smaller than the contemporary 80286 and 80386 processors in most desktops, with a much lower transistor count. However, it also offered higher performance and was able to do this at lower power consumption.[2]

Early work culminated in the Arm610, which was used by Apple in 1992 for their Newton PDA, and in 1994 by Acorn in their RiscPC desktop machine. In 1993, Arm made a profit for the first time.

A steady trickle of licensees followed, including Samsung, Texas Instruments, Plessey, and Sony. The Arm7TDMI, however, released in 1994, was Arm's first major success. It embodied several key innovations, which established Arm's products in the marketplace. In the name, 'D' and 'I' indicate on-chip debug capabilities that made it possible to debug a core embedded within a silicon chip, without needing access to the internal bus connections. This allowed developers to debug without an external emulator. The 'M' indicates a fast multiplier circuit (earlier versions did not have dedicated hardware for multiplication, so this represented a significant additional capability). The 'T' indicates that the device supports the Thumb® instruction set.

The Thumb instruction set is the most significant early development in the Arm architecture, and it is worth examining it a little more closely. While capable of very high

2 Arm2 at 8MHz achieved 4DMIPS, nearly twice the 2.15DMIPS of the 16MHz i386DX (Source: Real World Technology). The chip consisted of 28000 transistors, vs the 275000 in the 80386. It was also 7x faster than the 7MHz 68000 used in the Amiga 1000 and the Macintosh, and 50% faster than the 16.6MHz 68020 in the Macintosh 2 and Sun 3.

performance and power efficiency, earlier Arm processors suffered in deeply embedded use cases due to their need for a 32-bit data bus, which was expensive and complicated to integrate in small, low-cost devices where 16-bit buses and 16-bit memory devices are a significant advantage. Since all Arm instructions are 32 bits long, instruction fetches operate at half speed in a 16-bit memory system, which seriously decreases performance. In addition, code density is relatively poor due to the large size of instructions.

Arm's solution was to devise a subset of the instruction set, re-coded in 16-bit instructions. To simplify the implementation, there was no separate Thumb decoder; instead, Thumb instructions were translated to equivalent Arm instructions, which were then executed by the original Arm decoder and execution unit. The Thumb decompression stage occupied a previously unused half cycle in the pipeline, so it had no direct impact on instruction throughput. Benchmarks showed that the Thumb instruction set gave a 35% improvement in code density and performance in 16-bit memory—close to Arm instructions in 32-bit memory.

The Arm7TDMI was licensed by Texas Instruments to be incorporated into the first single-chip mobile phone solution. This was provided to Nokia, and first appeared in the Nokia 6110. Within a very short period, Arm captured over 98% of the mobile phone market. In 2009, the Arm7TDMI was still one of the most widely-used Arm cores—a testament to its versatility, cost–effectiveness, and high efficiency. More importantly, it provided Arm with a platform for explosive growth across many other markets.

The Arm9™ processor family, introduced in 1998, marked a significant implementation decision to move to a Harvard bus architecture, with completely separate data and instruction memory systems. The resulting increase in memory bandwidth allows for a higher performance pipeline and greater throughput. Through its lifetime, the Arm9 family introduced extended instructions for Digital Signal Processing (DSP) applications, Java acceleration, and floating-point capability. It was the first family of Arm cores that were delivered to licensees as fully synthesizable Hardware Description Language (HDL). This allowed the licensee much greater freedom in configuring and implementing the core, for a wider variety of use cases. Earlier cores were delivered as GDSII files, targeted to a specific fabrication process, meaning that every core needed to be manually re-synthesized by Arm for each target process.

Arm9 processors were the first to be designed specifically to execute code and fetch data from caches, and their execution pipelines reflect this choice. Some variants (notably the ARM966), however, were optimized for hard real-time use cases (such as machinery control and hard disk drive software) in which predictability of execution time sometimes

takes preference over raw speed. These cores use Tightly Coupled Memory (TCM), SRAM, which is connected directly to the core and is accessible with no wait states.

Some Arm9 processors (e.g. ARM926) employ both caches and TCM for flexibility.

During this period, Arm made a significant change to its technology licensing model. Traditionally, and this is still largely the case, Arm's licensees pay for the right to create silicon devices based on a specific Arm core, or family of Arm cores. They take the HDL source from Arm, configure it as required, add their own proprietary components, and then fabricate the end product using their preferred manufacturing process.

An Arm "architecture license", however, takes this one step further. Here, the licensee purchases the right to build devices based on a particular architecture, but without using Arm's source code as the intermediate step. Such licensees implement the core entirely independently, often using their own proprietary design processes and tools. This allows them to offer greater differentiation, compared to completing products. However, this requires large design teams and greater design costs.

The first such licensee was Digital, who, in 1995, bought a license to the Armv4 architecture and developed the StrongArm range of processors. Using Digital's own in-house tooling and design rules, these processors were, for some time, the fastest and most power efficient Arm-compatible processors on the market. Exploiting the freedom offered by the architecture license, Digital's design team were able to make many custom enhancements to the microarchitecture of the processor to increase its speed. For instance, the StrongArm–110 included a dedicated circuit to compute the destination for branch instructions. At the expense of some extra gates, this reduced the branch latency by one cycle (contemporary Arm implementations re-used the addition circuits in the ALU to do this computation, taking an extra cycle to do so). The technology eventually passed to Intel in 1997, where it became XScale, and eventually to Marvell in 2006.

Arm10 followed, with wider 64-bit buses and more sophisticated cache architecture.

Arm11™ again included several significant innovations, which drove the next phase of growth: a third instruction set, Thumb–2, was designed to provide the advantages of 16-bit Thumb and 32-bit Arm in a single encoding; multicore capability was added with cache snooping to support up to four cores in a single cluster; and the TrustZone® security architecture addressed the growing need for greater security at the hardware level.

3.3. Diversification

In the early 2000s, it became obvious that a single architecture could no longer cope with the enormous breadth of use cases for Arm processors. For example, the processor requirements of a deeply embedded microcontroller are very different from those of a smart television. In response, Arm introduced Armv7™—a set of "architecture profiles". While all based on a common source architecture, these profiles enabled significantly different products across a wide range of price and performance points. These profiles are all marketed under the Arm® Cortex® brand.

As well as the different capabilities offered by the architecture profiling, implementations can vary widely. Licensees make these choices during the synthesis and integration process based on factors such as functional safety, multicore requirements, security, DSP and Multimedia capability, cryptography, and compartmentalization.

3.3.1. Cortex–A for Application Class Processors

Cortex–A comprises implementations of the Armv7–A, Armv8–A and Armv9–A architecture profiles. These remain the pinnacle of performance and capability in the Arm product range. Key features include high-performance processor cores with sophisticated branch prediction, complex and efficient cache architectures, highly-optimized capability for DSP, machine learning (ML) and artificial intelligence (AI) applications. Some of these cores are marketed under the Cortex–X brand, indicating that they prioritize performance over other considerations, such as silicon area and power consumption.

Cortex–A cores are designed to execute code and fetch data from cached memory systems. The structure of their execution pipelines reflects this choice. In most cases, caching behavior is prescribed in the architecture, while cache size is a choice the licensee makes during implementation.

Armv8–A introduced a 64-bit operating state for the processor, supporting a completely new, 64-bit instruction set known as A64. It introduced a new exception model, and new operating modes aimed at supporting multiple simultaneous applications, operating systems and hypervisors.

All Cortex–A cores have the option to support at least two instruction sets: Arm (an evolution of the original Arm instruction set); Thumb–2; and, since Armv8, A64.

Although they are optional features in the architecture, the Neon™ extension (for Multimedia and DSP) and VFP extension (for vector floating point) are key features of Cortex–A cores. Armv8 introduced a further set of extensions to accelerate cryptography.

Armv8.2–A introduced Simultaneous Multi-Threading (SMT), which was first implemented in the Neoverse™ E1. It also introduced the Scalable Vector Extension (SVE), which was specifically developed for variable-length vectorization of high-performance workloads.

Armv8.3–A provided a number of enhancements, including pointer authentication and complex number capability.

Armv8.5–A became the baseline for Armv9–A, introducing Memory Tagging Extension (MTE) and Branch Target Indicators (BTI) to combat illegal memory accesses and arbitrary code execution.

3.3.2. Cortex–R for Hard Real-Time Processors

The 'R' profile architectures are specifically targeted at applications needing hard real-time response. Modifications to the pipeline, exception model, and memory system are aimed at predictable and fast responses to exceptions, and deterministic instruction throughput. They may also implement hardware division.

These cores are found in applications such as machine controllers, automotive management, and high-speed modems.

From Armv8–R, these processors have the option of supporting the A64 64-bit instruction set.

3.3.3. Cortex–M for Microcontrollers

The M profile is significantly different from the other two profiles and is aimed specifically at deeply embedded microcontrollers. These cores implement only the Thumb–2 instruction set, are programmable entirely in high-level languages such as C and C++, and deploy a significantly different exception model based on existing microcontroller architectures.

Since Armv8–M, Cortex–M has optional support for a security architecture known as TrustZone for Armv8–M security architecture. Although the brand name is similar, the implementation is significantly different to the TrustZone security architecture found on A and R profile cores. It also supports the Helium vector processing instruction set extension.

3.4. Current Drivers of Architectural Change

The latest Arm architectures are driven, as ever, by the changing and evolving needs of the electronics industry. While the mobile phone sector is still clearly important to Arm, in recent years, there has been significant application in other markets, including networking, data center, automotive, and safety-critical devices.

While performance and power efficiency are still highly important, there is an increasing need for better and more robust security, specific requirements for safety-critical devices, and very high-performance data center and networking use cases. The majority of current innovation in the architecture is focused on the A–profile cores, aiming to provide the highest possible performance, particularly for data center, machine learning, and artificial intelligence workloads.

3.5. Other Products and Activities

Like any processor architecture, developing a product requires more than just manufacturing or buying the processor device itself. Development tooling, source management, operating systems, middleware, and firmware must either be developed in-house, or sourced from Arm or third parties.

From the earliest days, Arm has produced standard tools solutions that are either sold commercially or made available via contributions to Free and Open-Source Software

(FOSS) projects. At the beginning, Arm was the only source of its architecture devices, so it needed also to produce and sell its own range of tools (compiler, assembler, linker, librarian, debugger, etc.) as no other supplier was available.

Texas Instruments—one of the first licensees—wrote its own tooling solutions. But an increasing number of third-party suppliers partnered with Arm to produce competitive alternatives. Arm's own tools are still available and are valuable to licensees as the first to support new architectures.

The tooling situation for cloud-based development is necessarily different and will be explained in detail in subsequent chapters.

4. Armv9 Overview

Armv9 represents the current state-of-the-art Arm architecture for high–performance computing applications. It has been announced in increments, starting with Armv9.0–A in 2021. At the time of print, the latest version is Armv9.6–A, which was announced in 2024. Armv9 aims to address the needs of High-Performance Compute (HPC), cloud computing, hardware acceleration for machine learning and other artificial intelligence tasks, high data throughput computing, networking, and Confidential Computing.

Each version contains a combination of mandatory and optional features. Some features, while optional, may only be implemented in combination with specific subsets of other features. Features that are optional in one version of the architecture may become mandatory in later versions.

All versions are incremental—each version includes the mandatory features of all prior versions.[3] Meanwhile, Armv8–A has continued to develop in parallel with Armv9–A. They are closely linked, and the features implemented at each iteration are typically very similar.

In the descriptions that follow, only limited detail is provided for any features that are only of interest to silicon designers or are not exposed directly to software developers other than through defined tooling interfaces or firmware.

In all cases, further detail is in the Arm Architecture Reference Manual, though be aware that this document is written for silicon developers and architects and is of limited use for software developers due to the level of detail which it necessarily includes.

Before any detailed look at Armv9–A, it is important to understand some fundamental concepts and terminology regarding the structure and taxonomy of the architecture. §4.1

3 There are a small number of instances where features have been removed as part of an architectural update. These usually follow a significant period during which the feature concerned is marked as deprecated to ensure that it is no longer in use.

below provides a brief introduction to exceptions, operating modes and states, instruction sets, and security states and should be read before continuing.

4.1. Exceptions, Modes, States and Instruction Sets

Some level of awareness of the internals of the Arm architecture is necessary for any meaningful discussion about Arm processors. This section provides a very brief overview of some key terms and concepts. As ever, more detailed information is available on the Arm website for those with a deeper curiosity.

The discussion that follows does not attempt to address the 26-bit modes, which are of only historical interest, nor the Armv7–M architecture supported by the Cortex–M microcontrollers, which is significantly different.

4.1.1. Exceptions and Operating Modes

In the Arm architecture, exceptions and operating modes are closely linked. In architecture Armv7—and all earlier versions of the architecture—each exception type is handled using a specific operating mode. These modes have separate exception vectors and a private subset of the main register set. This simplifies the nesting of exceptions of different types and reduces the amount of context an exception handler needs to save on entry and restore on exit.

Some exceptions are linked to external interrupts—for example, IRQ is the standard external hardware interrupt, and FIQ is an external hardware interrupt of higher priority. Some exceptions are linked to error conditions, such as memory access aborts. Other exceptions are linked to software interrupts caused by execution of an exception instruction, such as SWI.

Armv7 supports a number of operating modes. Except for User mode, these are all “privileged”, meaning that they have access to system resources that have been protected from unprivileged access. To provide a degree of protection, the privileged modes are generally reserved for the operating system. The Armv7 modes are:

– FIQ, entered on detection of an FIQ hardware interrupt

– IRQ, entered on detection of an IRQ hardware interrupt

- Abort, entered on detection of a data memory access abort
- System, a privileged mode that is not entered via an exception
- SVC, entered on reset and on execution of a Software Interrupt (SWI) instruction
- User, the only unprivileged mode, used to execute user code

When in a privileged mode, software can freely switch modes by directly modifying the Current Program Status Register (CPSR). When in User mode, a mode change can only be caused by an exception.

At first glance, System mode may seem anomalous since it is a privileged mode, but it is not entered via an exception. System mode was a relatively late addition to the Arm architecture, introduced to simplify some issues which can occur when nesting exceptions.

Armv9–A adds another layer to this, with four "exception levels"—or EL, listed here in increasing order of privilege:

- EL0, the lowest level of privilege, usually used for execution of user applications
- EL1, typically used for execution of a rich operating system, such as Linux
- EL2, which can be used for hypervisors
- EL3, the highest level of privilege, typically reserved for firmware and security gateway code

A typical usage model for these exception levels is shown in Figure 1.

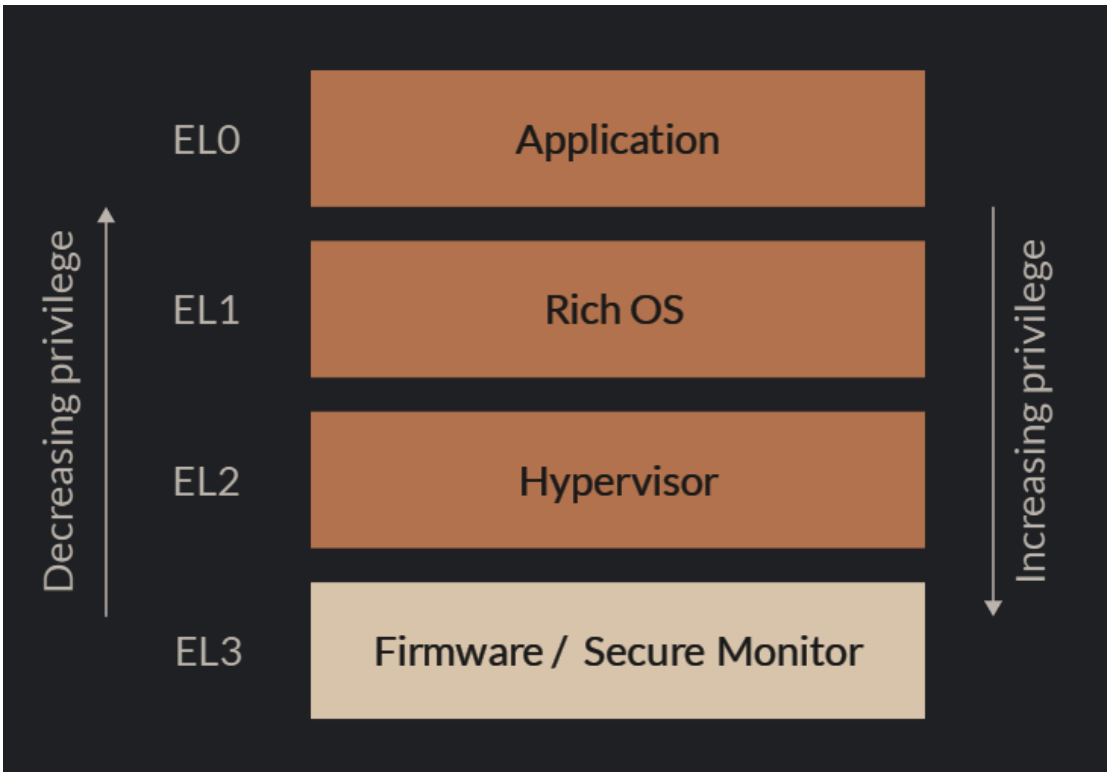

Figure 1 – Exception levels

Movement between ELs occurs when taking, or returning from, an exception. The general rule is that on taking an exception, the EL can only stay the same or go up; on returning from an exception, it can only stay the same or go down.[4]

Note that the architecture defines these four ELs but does not mandate the use of each level. The model in Figure 1 is, however, common and typical. Also, while support of EL0 and EL1 is mandatory in any implementation, EL2 and EL3 are optional. In practice, these levels are usually implemented, since EL2 supports the virtualization features required by most hypervisors, and EL3 is the only EL which can change security state.

4 The EL can also change on entry to or exit from Debug state but the details of this need not concern us here.

4.1.2. Execution States and Instruction Sets

Armv8–A and Armv9–A processors support two execution states: AArch32, a 32-bit execution state; and AArch64, a 64-bit execution state. In general, the execution state defines the width of the general–purpose register set, and also the available instruction sets.

In Armv8–A, either or both execution states may be supported at each exception level. Armv9 –A requires that AArch64 is supported at all ELs, while AArch32 may be optionally supported only at EL0.

Execution state can only change when the exception level changes, and, in a similar fashion to exception levels, the architecture requires that when moving to a higher EL, the execution state may either stay the same or change to AArch64. On moving to a lower EL, it can either stay the same or change to AArch64. The rules on changing exception level and execution state have a number of implications:

- If AArch32 is not implemented at a particular EL, then it cannot be implemented at any higher levels.
- AArch32 software at any particular EL can only host AArch32 software at any lower levels.
- An AArch64 operating system executing at EL1 can host both AArch32 and AArch64 software executing at EL0.
- An AArch64 hypervisor executing at EL2 can host both AArch32 and AArch64 software executing at EL1.

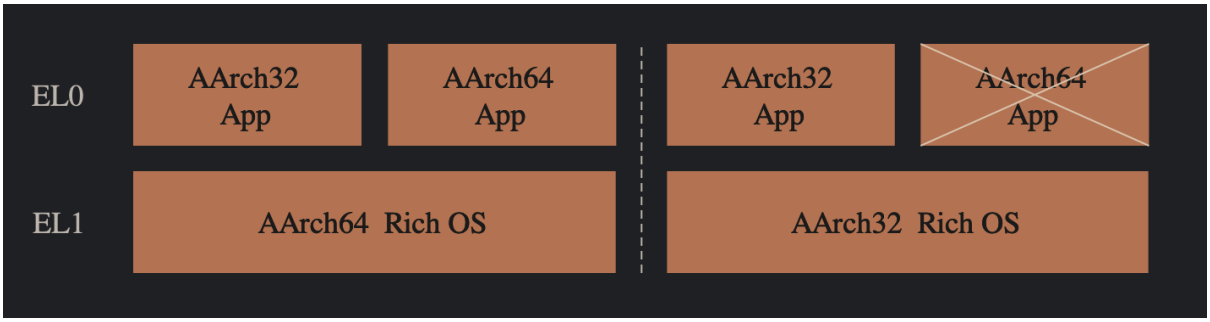

Figure 2– Exception levels and Execution States

In AArch32:

- Operation is backwards-compatible with Armv7–A and previous architectures.

– The A32 and T32 instruction sets are available.[5]
– The 32–bit Armv7–A register set is supported.

In AArch64:

– Only the A64 instruction set is available.
– The general-purpose register set is expanded, both in width and in number, compared to Armv7–A and earlier architectures.

A32 is the direct descendant of the original instruction set used by the earliest Arm processors. In A32, all instructions are 32 bits wide and are aligned on 32-bit address boundaries in memory.

T32 evolved from the Thumb instruction set introduced in the Arm7TDMI® (architecture Armv4T). For more detail on the origins of the Thumb instruction set, see §3.2 above. In the original Thumb instruction set, all instructions are 16 bits wide and are aligned on even address boundaries. These instructions form a 16-bit encoded subset of the Arm instruction set, and each instruction has a direct Arm equivalent.[6]

The original Thumb instruction set was significantly expanded, initially in the Arm1156–T2 processor, by addition of some 32-bit instructions, yielding a mixed-length instruction set originally known as Thumb–2. This instruction set has evolved into what is now called T32.

A64 is an entirely new instruction set introduced in Armv8–A, and carried forward into Armv9–A.

Within AArch32 state, the A32 and T32 instruction sets each have an associated "state"—Arm state for A32, and Thumb state for T32. These are indicated by the value of the T bit within the CPSR. A change of state can only be caused by a branch or return instruction, or on entry to, or return from, an exception.

---

5 See below for more information on instruction sets.

6 This leads to a simple hardware implementation in which each Thumb instruction is translated to an Arm equivalent when fetched from instruction memory, and these translated instructions are then executed by a single (Arm) decode and execution unit.

4.1.3. Security States

Most Cortex–A processors support two security states: Secure and Non-Secure.[7] Certain processor features are architecturally classified as Secure and can only be accessed by software executing in Secure state. Secure state information is also propagated outside the processor into external components such as the memory interface, caches, memory management units, and other peripherals. This allows system designers to partition the system into sets of Secure and Non-Secure components.

When executing in Secure state, software can access both Secure and Non-Secure components and features; when executing in Non-Secure state, Secure items cannot be accessed and attempts to do so will usually result in an exception.

This architecture allows the implementation of secure subsystems within the system. This might be, for example, a secure payment or DRM subsystem executing in Secure state and using some secure features, alongside an operating system such as Linux or Android, executing in Non-Secure state. As long as the mechanism for changing security state is strongly protected, this prevents Non-Secure software from accessing or observing Secure software and hardware. In an Armv8–A or Armv9–A system, the only route into Secure state is through a Secure Monitor, which executes in EL3. An example system configuration is shown in Figure 3.

7 Historically, Non–Secure state has been referred to as "Normal world". You may find this term in older documentation.

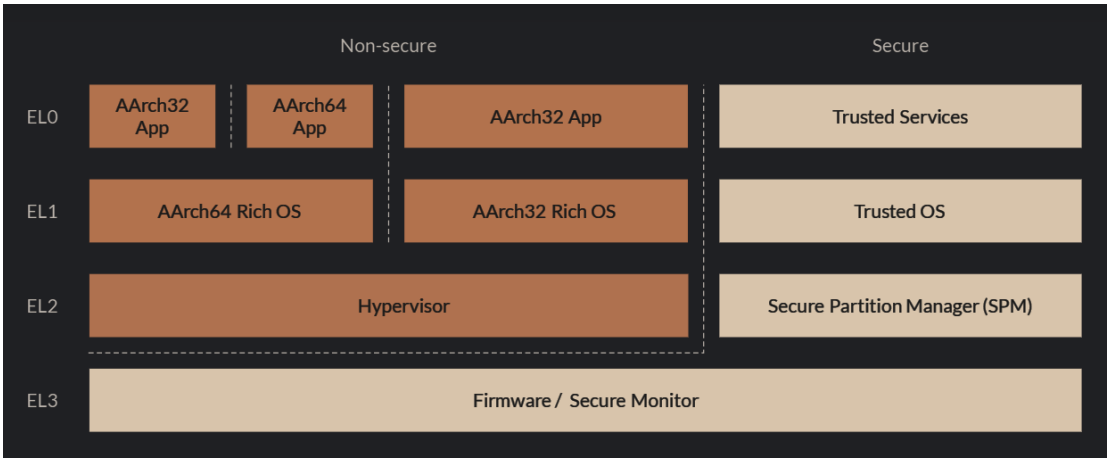

Figure 3 – Exception Levels, Security States and Execution States

Armv9–A extends the security model with the Realm Management Extension (RME). RME adds two new Security states: Root and Realm. In Root state, which is only available in EL3, the processor can access the entire physical address space. In Realm state, the processor can only access Non-Secure and "Realm" addresses. Real memory accesses are policed by a separate piece of software called the Realm Manager. Together with specific RME hardware features, this provides robust compartmentalization and greatly enhances security. Figure 4 shows an example configuration when RME is implemented.

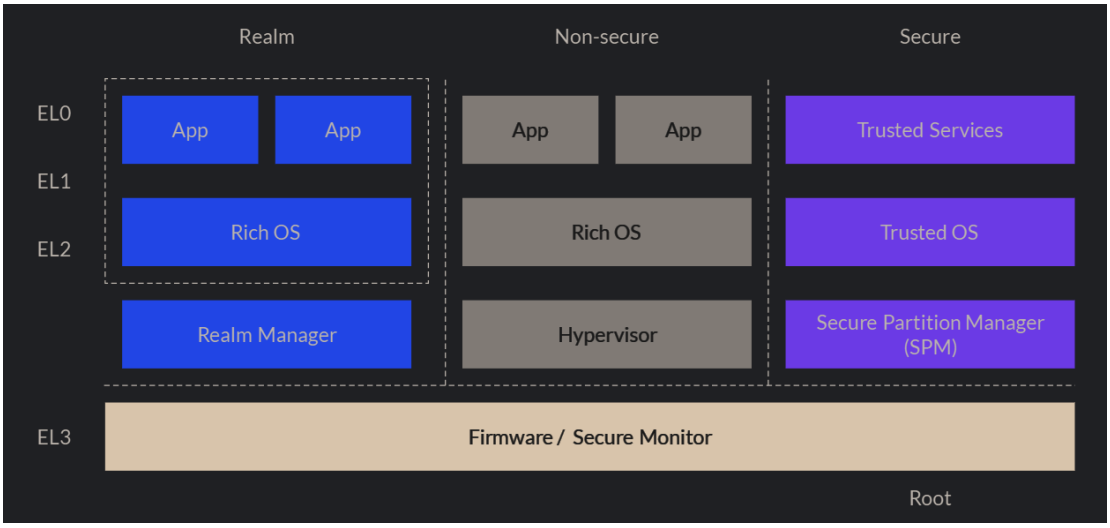

Figure 4 – RME Security States

4.2. Armv9.0–A

The Armv9.0–A architecture was announced in 2021, baselined on Armv8.5–A.

The main drivers for the Armv9 architecture are increased security and improved vector processing. The major architecture features involved are:

- Confidential Computer Architecture (CCA)
- Transactional Memory Extension (TME)
- Scalable Vector Extensions V2 (SVE2)

CCA is a major innovation in device security, driven by the fast-evolving needs of the industry. It was developed in collaboration with university research partners. CCA provides a compartmentalized architecture in which processes can execute in "realms", which are isolated from each other. As well as hardware support, a standard firmware architecture is also specified. CCA is a combination of a number of architecture extensions, including Realm Management Extension (RME), Dynamic TrustZone® Technology, Real Management Monitor (RMM), and standard CCA firmware.

TME provides for Hardware Transactional Memory (HTM), which allows sequences of instructions to be executed by the processor atomically. This greatly simplifies and enhances the execution of multi–threaded software on systems that consist of many independent processing elements. TME builds upon a number of architecture extensions, including Hardware Transactional Memory (HTM) and Transactional Lock Elision (TLE).

There are also a number of optional extensions, some of which are entirely new to the Arm architecture, while others build upon features from prior architecture versions. These include:

- Armv9 Cryptographic Extensions
- Embedded Trace Extension (ETE)
- Trace Buffer Extension (TBE)
- Scalable Vector Extension V2 (SVE2)

The optional SVE2 extension is an evolution of The SVE vector processing architecture introduced in Armv8. Extensions to the earlier iteration of SVE allow for better fine-

grained Data Level Parallelism (DLP) making it easier for compilers to vectorize key algorithms. Support is also introduced to accelerate key algorithms employed by the widely used Advanced Encryption Standard (AES).

4.3. Armv9.1–A

Armv9.1–A was announced in September 2019 and builds on both Armv9–0–A and Armv8.6–A. All mandatory features from those two earlier architectures are also mandatory in Armv9.1–A.

New features introduced include:

- additional floating-point matrix multiply instructions (GEMM);
- extensions to the Embedded Trace Extension introduced in Armv9.0–A; and
- Memory Partitioning and Monitoring (MPAM), which allows the system to monitor and restrict the memory bandwidth used by individual processes, in order to manage and mitigate the effect on performance of other processes.

4.4. Armv9.2–A

Armv9.2–A, baselined on Armv8.7–A, was announced in September 2020, and includes some minor instruction set enhancements, and an extension to Realm Management. Main changes include:

- Root & Realm security states and physical address spaces;
- Granule Protection Check mechanism;
- enhanced PCIe hot plug;
- atomic 64–byte load/store to accelerators;
- further extensions to ETE to support Realm Management (RME);
- Scalable Matrix Extension (SME) (Optional—allows tiled storage of vectors in registers);
- Realm Management Extension (RME) (Optional); and
- Branch–Record recording (Optional—captures execution path history).

4.5. Armv9.3–A

Armv9.3–A, baselined on Armv8.8–A, was announced in September 2021. Additions include:

- Branch Recording in EL3;
- "Non–maskable" (NMI) and "less-maskable" interrupts (LMI) for NMI for AArch64;
- memcpy and memset instructions;
- hinted conditional branches (the ability to include hints with branch instructions, which helps the branch prediction logic by indicating that a particular branch is consistent and is unlikely to change direction);
- Scalable Matrix Extensions v2 (Optional), which introduces the ability to re-address two-dimensional tiles as groups of one–dimensional vectors; and
- Memory Encryption Contexts (MEC) (Optional)—an extension to RME that provides additional contexts for memory encryption linked to existing realms.

4.6. Armv9.4–A

Armv9.4–A, baselined on Armv8.9–A, was announced in September 2022. Additions include:

- improved security against attacks that use branch history to infer and manipulate execution;
- VMSA enhancements including 128–bit translation tables, system registers, and atomic 128–bit instructions (Optional);
- SME2 and SVE2.1 (Optional), which add a number of extra instructions to SME and SVE2 to improve matrix and vector operations; and
- Guarded Control Stack (Optional)—a separate memory area for the storage of exception returns and branch returns, protected from modification.

4.7. Armv9.5–A

Armv9.5–A (October 2023) introduces:

8–bit Floating Point (FP8) support added to SME2, SVE2 and Neon™.

4.8. Armv9.6–A

Armv9.6–A (October 2024) introduces:

- MPAM Domains for improved support of shared memory in multi-chiplet and chip systems; and
- Granular Data Isolation for Confidential Compute.

5. The Neoverse Cores

So far, you have read about the development of two distinct threads in the Arm® product range. Firstly, the evolution of the architecture—the contract between hardware and software that defines the behavior of devices compatible with Arm architecture. Secondly, the implementation of a growing range of processor cores based on each variant of the architecture. These two factors, independent yet related, are important when understanding the rationale behind each Arm device.

The architecture of a particular device does not dictate what it will be used for. For example, not all Cortex–A devices are used in mobile phones, although a great number are. Similarly, not all Armv9–A processors are used in infrastructure applications, though many of them are very suitable for this. It is the combination of architecture and implementation that determines whether it is suitable for a particular use case.

The Neoverse devices conform to a mixture of architectures. Early cores were built to Armv8–A, and later cores to Armv9–A, but every implementation choice was made deliberately to make them suitable for use in infrastructure applications.

Each Neoverse™ device is different. These differences arise partly from the architecture, but also from choices made by whoever implements the device, and the particular manufacturing process used in their production. Understanding this is important when evaluating or selecting a device.

It is also important to understand this when reading, in the next chapter, the list of currently available Neoverse devices. They are wonderfully diverse, and this diversity is one of the strengths of the Arm product family.

5.1. Architecture, Implementation and Manufacturing

Much of the base functionality of a particular device is defined in the architecture. While some of it may be optional, it is essentially immutable. Other behavior is determined by whoever implements or manufactures the device, and they have significant freedom here.

Some sources make the distinction between architectural, microarchitectural, and implementational behavior.

5.1.1. Architecturally-Defined Behavior

In the preceding sections, you have looked at some of the behaviors that are mandated by the architecture itself. The consistency across the product range, enforced by the requirements of the architecture, has been a key factor in the success of the Arm architecture in the marketplace. In essence, as you have already seen, the architecture defines the behavior of the hardware when the hardware is presented with an instruction or a stream of instructions. The consistent behavior of all Arm devices conforming to a particular architecture version is crucial. It ensures consistent software behavior across all implementations, which is key in establishing a wide and vibrant ecosystem.

Although the architecture defines the required behavior precisely, it permits a degree of freedom in many aspects of the final device. For instance, the architecture defines whether caches are permissible, what their structure may be, and how they respond to management operations, but the architecture doesn't specify their size. That decision, along with many others, is left to whoever implements the device.

In addition, many architectural features are specified as "optional". These relate to features that may or may not be implemented in the production device, at the discretion of the implementer. However, if these features are included, the architecture defines precisely how they must behave. On the other hand, if an "optional" feature is not implemented, the architecture defines the corresponding behavior in their absence.

The architecture also defines mechanisms by which software developers can interrogate the system to determine which optional features are implemented and with which implementation parameters.

There are several architecture licensees operating in this market space, and the behavior of their products depends on both the architecture and on the specific implementation deployed by the licensee. In many cases, that implementation might be proprietary.

5.1.2. Implementation Decisions—Microarchitecture

The process of "implementation" involves taking the HDL source code for the hardware from Arm and transforming that into a description of a processing element that can be

fabricated into a working, physical device. This is a highly complex multi-stage process. Significant choices are made along the way that affect the behavior and capability of the production device.

Those choices might include:

- whether certain optional architectural features are implemented;
- the size and scope of processor and associated logic, such as caches, buses, and interconnect architecture;
- the number and organization of processing elements; and
- the length and structure of the processor pipeline.

Clearly, these features have a considerable effect on the performance and behavior of the production device.

In particular, the instruction throughput is not determined by the architecture but by factors such as the pipeline structure. Complex pipelines could implement instruction–level parallelism in the hardware, allow prediction and speculative execution, or accelerate the operation of some logical and mathematical operations. These decisions have a great effect on the performance of the production device, but the way that the device responds to an instruction stream must conform to the architecture.

5.1.3. Manufacturing Variation

The process of taking configured HDL, synthesizing it to a netlist, physically producing the resultant silicon, and packaging this into a finished device is one of the marvels of modern manufacturing technology. The enormous complexity of the devices involved and the unimaginably small features that they manipulate on silicon substrates is truly breathtaking.

The precise manufacturing process used has a significant effect on the characteristics of the device. Choices are made to achieve different goals such as processing power, clock speed, power consumption, silicon area, and cost. The primary decision is the “feature size”—the basic geometric size of the transistors, wires, and gates which constitute the device—but many other choices contribute to significant variation in the resulting product.

5.2. Neoverse Overview

The Neoverse cores from Arm are specifically designed and configured for use in cloud computing environments. But each cloud computing environment has different requirements, so the Neoverse cores are subdivided into three main ranges: 'V', 'N', and 'E'. These three variants recognize and address the trade-offs between price, performance, and cost, to serve the differing requirements of cloud, edge, and infrastructure deployment.

Neoverse V is designed for the highest performance and throughput. These are designed for general–purpose computing and datacenters performing machine learning and artificial intelligence tasks. The V range is designed to scale easily to multi-core systems, while maintaining a focus on total cost of ownership (TCO) through an emphasis on power efficiency and cost-effective manufacturing.

Neoverse N is positioned for applications connecting the cloud to the edge. These deliver high performance with low power requirements and high bandwidth flexible interconnect.

Neoverse E is best suited for deployment at the network edge, where high data throughput is key, employing simultaneous multi-threading (SMT) for high efficiency.

All are built to Armv8–A and Armv9–A architectures, with the newer cores incorporating the latest Confidential Compute Architecture to allow provision of secure datapaths from the edge of the network through to the datacenter.

The table in 5.9 summarizes some of the more important features provided by each Neoverse core.

5.3. Neoverse V–Series

Neoverse V is the performance-first flagship for Neoverse, offering the highest performance with the fewest compromises in the implementation. As such, cores in the Neoverse V–Series are targeted at high-performance computing in the cloud and acceleration for machine learning and artificial intelligence tasks.

5.3.1. Neoverse V1

The Arm Neoverse V1 core, based on Armv8.5–A, was the first Arm core to include Scalable Vector Extension support, providing 2× 256b SVE pipelines and twice the floating-point performance capability of the earlier Neoverse N1.

5.3.2. Neoverse V2

The Arm Neoverse V2 was the first Neoverse core based on Armv9 architecture. It has significantly larger caches than Neoverse V1, and an enhanced pipeline design, providing approximately double the performance on cloud and machine learning workloads.

CPUs based on Neoverse V2 include AWS Graviton4, Google Axion, and NVIDIA Grace.

5.3.3. Neoverse V3 and Neoverse V3–AE

Neoverse V3 is currently the highest–performance core provided by Arm. Based on architecture Armv9.2–A, it implements the latest CCA security features. An increased L2 cache enables increased performance. It is targeted at High-Performance Computing (HPC), machine learning, artificial intelligence, and general-purpose compute in a cloud application.

Neoverse V3 is used in products such as the first-generation Arm AGI CPU, which is based on Arm Neoverse CSS V3, which will be introduced in section 5.6.1.

The decision process that led to the Neoverse V3 also considered Total Cost of Ownership (TCO) as an important design factor.

Arm Neoverse V3–AE is a variant of Neoverse V3 that implements security and safety enhancements to enable deployment in environments—such as automotive—where Functional Safety certification is required.

5.4. Neoverse N

5.4.1. Neoverse N1

The Neoverse N1 is engineered for high scalability, making it particularly suited for the infrastructure between the cloud and edge. The bus and connectivity architecture allows multi-core clusters to be implemented together in configurations of up to 16 cores per chip. Multi-chip packaging solutions allow up to 128 cores per socket, with CCIX[8] connectivity, in hyperscale applications.

Neoverse N1 and Neoverse E1 share a common software architecture which allows for seamless heterogeneous systems encompassing both the edge and the cloud.

5.4.2. Neoverse N2

Neoverse N2 increases scalar performance over Neoverse N1 by 40%. Built on Armv9.0–A it includes enhancements to improve scalability, performance and security.

5.4.3. Neoverse N3

Arm Neoverse N3 is the highest-performance core in the Neoverse N range, yielding approximately three times the performance of Neoverse N2 for machine learning workloads. The advanced design also improves power efficiency (measured in MIPS per watt) by 20%, improving efficiency on multi-core deployments.

5.5. Neoverse E1

The Neoverse E1 processor is built to Armv8.2–A with emphasis on power efficiency and data throughput. This makes it particularly well suited for data transport at the edge. It is the first, and currently the only, Arm processor to implement simultaneous multi-threading (SMT), which allows instruction level parallelism (ILP) at the hardware level. The ability to execute two software threads concurrently yields better aggregate throughput.

---

8 Cache Coherent Interconnect for Accelerators (CCIX) is an interconnect standard which allows accelerators to be connected in a manner which maintains and leverages cache coherency between separate processing elements.

Compared to the earlier Cortex–A53, The Neoverse E1 delivers over double the compute performance, nearly three times higher throughput performance, and more than double the power efficiency.

5.6. Neoverse Compute Subsystems

The processor is not the only ingredient when implementing a device suitable for deployment in the data center and allied infrastructure.

In multiprocessor systems handling very large datasets, several factors become increasingly important: the ability to address large regions of memory; the speed at which large amounts of data can be moved around the device between on-chip memory, off-chip storage, and the cache; and the ability for multiple processors to share data structures and code. These are not just functions of the processor itself but also of the on-chip interconnect fabric.

As this fabric becomes increasingly critical to the performance of the device, it becomes increasingly complex to design and more sensitive to small changes in configuration and implementation. To mitigate this, Arm has recently put considerable effort into designing on-chip interconnect and other system components, such as System MMU, cache, and DMA controllers. Many partners use these components when configuring and implementing their devices.

However, as systems become more complex, the engineering cost of putting them together becomes more significant. Arm is able to mitigate that with its range of Compute SubSystems (CSS), which are ready-built and verified by Arm.

Note that while subsystems are available for other Arm cores (see Corstone™, for example), this text concentrates only on those that include Neoverse processors. These details are relevant to silicon designers but not necessarily directly relevant to software developers, so these details are included here only for completeness.

5.6.1. Neoverse CSS V3

The CSS V3 platform is configured and verified for high-performance cloud, HPC, artificial intelligence and machine learning workloads. It supports up to 64 Neoverse V3

cores, and includes a customizable memory subsystem and allows AI accelerators to be connected easily. The system supports up to 12 channels of (LP)DDR5 memory, up to 64-lane PCIe or CXL I/O, and high-speed die-to-die connections.

5.6.2. Neoverse CSS N3

This platform is based around Neoverse N3 and is targeted for networking, 5G, and infrastructure edge deployment.

CSS N3 supports between 8 and 32 Neoverse N3 cores per die, and includes on-chip and off-chip interfaces for (LP)DDR5 memory, up to 32–lane PCIe or CXL I/O, and high-speed die-to-die connections.

5.6.3. Neoverse CSS N2

Neoverse CSS–N2 supports up to 64 Neoverse N2 cores, and is targeted for an advanced 5 nm manufacturing process. Neoverse CSS N2 CSS interfaces for (LP)DDR5 memory, PCIe and CXL I/O, plus SBSA certification and a platform software model.

	V1	V2	V3	N1	N2	N3	E1
Architecture	v8.6–A	v9.0–A	v9.2–A	v8.5–A	v9.0–A	v9.2–A	v8.5–A
ISA[9]	A64 A32/T32	A64 A32/T32	A64 only	A64 A32/T32	A64 A32/T32	A64 only	A64 only
SVE	Y	V2+AES	V2+AES	N	Y	Y	N
Crypto	Y	Y		Optional	Optional	Optional	Optional
MPAM	Y				Y	Y	
Pointer Authentication	Y				Y		
Cluster	Direct–connect	Direct–connect	Direct–connect	Up to 4 CPUs	Direct–connect	Direct–connect	Up to 8 CPUs
Physical Address[10]	48–bit	48–bit	48–bit	48–bit	48–bit	48–bit	44–bit
L1 I Cache L1 D Cache	64 kB 64 kB	64 kB 64 kB	64 kB 64 kB	64 kB 64 kB	64 kB 64 kB	32 kB 64 kB	32–64 kB 32–64 kB

9 ISA = Instruction Set Architecture. Where support for the A32 and T32 instruction sets is shown, this is only at EL0 in all cases.

10 This refers to the physical width of the address bus in bits.

L2 Cache	1 MB or 512 kB	1 MB or 2 MB	2 MB or 3 MB	256 kB–1 MB	512 kB–1 MB	128 kB–2 MB	Optional 64–256 kB
L3 Cache				Optional 512 kB–4 MB			Optional 512 kB–4 MB
Security	TrustZone®	CCA	CCA	TrustZone®	TrustZone®	CCA	TrustZone®
MTE		Y	Y			Y	
RME			Y			Y	
BRBE			Y				
RAS			Y		Y	Y	Y

5.7. Feature Summary

The table shows a summary of features for all the current Neoverse cores that are most relevant to software developers. It is not an exhaustive list—please refer to Arm's website for up-to-date details.

5.7.1. Correlation with x86 Features

The predominant alternative architecture in use in the data center is x86, primarily in devices supplied by Intel and AMD. In a similar way to the Arm architecture, the x86 architecture has a number of capability extensions that drive platform choice. When evaluating a potential Arm solution, it is useful to identify the broad correlation between these features across the architectures.

The table below is not an exhaustive list of extensions in either architecture, but concentrates on those which are visible to, and of interest to, software developers. For instance, features associated with virtualization would only be of interest to developers and vendors of hypervisors. Features for accelerating cryptographic operations would be of interest only to cryptographic libraries.

Extension	Description	Use Cases	Arm Equivalent[11]
MultiMedia eXtension MMX (1996)	SIMD[12] instructions to enhance multimedia performance	Image and audio processing	NEON (Armv7–A)
Streaming SIMD Extensions SSE (1999) SSE2 (2001) SSE3 (2004)	SIMD instructions to improve performance of 3D graphics and scientific simulations	3D graphics, scientific simulations	NEON (Armv7–A)
Advanced Vector eXtensions AVX (2011) AVX2 (2013) AVX–512 (2016)	Extended SIMD instructions to support high–performance computing and vector[13] processing	Machine learning, scientific computing	Scalable Vector Extension SVE (Armv8.2–A) SVE2 (Armv9–A)
Advanced Encryption Standard New Instructions AES–NI (2008)	Accelerates AES encryption and decryption operations in hardware	Secure data encryption and decryption	Cryptography Extensions (Armv8–A)
AMD Virtualization AMD–V (2006) Intel Virtualization Technology Intel VT–x (2006)	Enables hardware–assisted virtualization[14] for running multiple operating systems	Virtual machines, cloud computing	ARM Virtualization Extensions (Armv8–A)
Advanced Matrix eXtensions AMX (2021)	Accelerates matrix operations for artificial intelligence and machine learning workloads	Artificial intelligence, machine learning, high–performance computing	Scalable Matrix Extension SME (Armv9.2–A)

11 Exact equivalence is not implied here. The suggested Arm extension is the closest equivalent, which may provide similar capability through a different mechanism.

12 Single Instruction Multiple Data—instructions that are capable of operating simultaneously on multiple data items.

13 A vector is a data structure holding multiple data items—usually of the same type—stored in a contiguous area of memory.

14 Virtualization is a technology which allows multiple operating systems and multiple applications to co–exist on a single processor, secured and isolated from each other.

6. Available Implementations

The Neoverse™ range of cores has been licensed by many of Arm's partners, ranging from established players in the high–performance computing (HPC) and cloud space, to start-ups looking to innovate in the technology industry. These licensees take Arm's designs and build differentiated devices for their respective target markets.

There are two principal routes to access any particular Neoverse hardware platform in order to develop and execute software applications.

Many Neoverse-based devices are available to purchase on the open market, either as bare chips or fitted to a development board. Depending on the manufacturer or supplier, these will come with a varying level of software support. This may be just boot code, firmware, and operating systems, or it may include standard or proprietary application software. The choice of platform will be influenced by the intended end use for the system.

However, many developers choose to access cloud-hosted platforms. This has many advantages, especially that physical hardware is not required. Without the need to own and maintain in-house hardware, it is easier to upgrade to later and higher-performance platforms if the need arises.

As with so much about developing on Arm, the choice is yours, and there is a huge variety of choices.

This chapter first lists the silicon implementations of Neoverse cores that are currently available, then the available cloud instances deployed on Arm silicon. Many of the silicon devices listed are available directly to the developer via cloud-hosted systems deployed by major cloud vendors. Some of them, though, are developed for proprietary use and might not be available to purchase on the open market.

The reader should be aware that this list of licensees, developers and providers is constantly and rapidly changing, as more licensees enter the market and existing licensees enhance their product offering. The list is also not exhaustive, because there may be licensees who have started development but not yet made their activities public.

Although there is a sometimes bewildering array of platforms available, there is also a degree of standardization, which allows for an increasing degree of interoperability between platforms and software components.

6.1. SystemReady

One consequence of the extensive flexibility that Arm's licensees have in turning processor designs into finished devices is the wide diversity of the resulting platforms. But this could easily result in a very fragmented software market.

To combat this, Arm introduced the SystemReady program. SystemReady defines firmware and hardware standards to enable a consistent cross-platform baseline for software developers, equipment manufacturers, and vendors. Choosing a SystemReady-certified platform should enable a wider choice of software.

SystemReady has been developed over a number of years in partnership with the industry and is currently supported by over 45 partners. Initially aimed at the server market, SystemReady now covers a much broader range of platforms, including SystemReady LS for hyperscalers, SystemReady ES for embedded systems, SystemReady IR for embedded Linux, and SystemReady SR for servers and workstations. All of these build on an underlying set of system specifications, which cover boot code, system components, and hardware features. This provides a stable, standardized platform for operating systems, allowing a degree of interchangeability and a "just works" out-of-box experience.

For Neoverse platforms, the relevant bands are SystemReady LS and SystemReady SR.

SystemReady is managed by Arm, in collaboration with its partners. Certifications are issued by Arm to compliant platforms.

6.2. Silicon Implementations of Neoverse Cores

This section only presents an overview. For detailed and up-to-date specifications for all the listed products, please refer to the manufacturer's website.

6.2.1. Ampere

Ampere was founded in Santa Clara, California, in 2017, with the goal of developing Arm-based server processors. Its early products were based on the X–Gene processors from Applied Micro (Ampere acquired the processor design team from Applied Micro in 2017). Later products have used Ampere Altra—a semi-custom Neoverse N1, built to Armv8.2–A. However, the latest products use Ampere's own in–house AmpereOne core. The AmpereOne product line, built to Armv8.6–A, scales to 256 cores per chip.

Ampere's processors are deployed in the cloud by Oracle, Google, Microsoft, HPE, and others.

6.2.2. Annapurna Labs

Annapurna Labs was founded in Israel in 2011. Since 2015, it has been a wholly-owned subsidiary of AWS. Its main product line, Graviton, is now at version 4. Early versions were built on Cortex–A72 cores from Arm; later versions have been built on Neoverse N1 and lately Neoverse V1 and Neoverse V2. The latter implements Armv9.0–A and incorporates the latest advances in security.

Graviton cores are deployed in the cloud by AWS. Graviton4 is a 96-core Neoverse V2 device made available in a number of AWS EC2 instances in the cloud.

6.2.3. Google

Google's Axion processors are based on Neoverse V2, implementing Armv9.0–A. Announced in early 2024, Axion–based servers are SystemReady-compliant, and are available via a number of Google Cloud services, including Google Compute Engine, DataProc, DataFlow, Cloud Batch, and Google Kubernetes Engine.

6.2.4. Marvell

Marvell acquired Cavium in 2018, inheriting the ThunderX and Octeon processor lines. Early Octeon processors implemented the MIPS64 architecture, switching to Arm with the introduction of ThunderX in 2014.

Octeon TX2 is deployed in a range of high–performance infrastructure processors, based on a multi-core Cortex–A72 implementation.

Octeon 10 is a family of DPUs[15] targeted at dataplane acceleration and high-speed packet processing for 5G wireless infrastructure. Octeon 10 is built around a multi-core Neoverse N2 architecture with 8–24 cores.

6.2.5. Microsoft

Microsoft's Azure Compute Platform cloud service provides solutions based on Ampere Altra and on Microsoft's in-house Cobalt 100 processor. Cobalt 100 is built using the Arm Neoverse CSS N2, facilitating migration between this and other SystemReady-compliant platforms.

Arm–based instances are deployed in the following families:

- B–Family (general-purpose compute on Ampere Altra)
- D–Family (general-purpose compute on Cobalt 100)
- E–Family (memory-intensive workloads on Cobalt 100 or Ampere Altra)

6.2.6. NVIDIA

NVIDIA announced the Grace CPU in 2021, targeted at high-performance compute and AI workloads in the datacenter.

Grace incorporates 72 Neoverse V2 cores and implemented Armv9.0A. The Grace CPU is available in several implementations including:

- NVIDIA Grace C1 (single-die, 72 Neoverse V2 cores), targeting traditional datacenter workloads
- NVIDIA Grace CPU Superchip (two-die, 144 Neoverse V2 cores) targeting HPC workloads
- NVIDIA Grace–Hopper and Grace–Blackwell Superchips, designed for high-performance on AI training and inference workloads

15 A Data Processing Unit (DPU) is a programmable processing element which is optimized for high–speed processing and transmission of data.

6.3.Cloud Implementations

This section presents only overview information. For detailed and up-to-date specifications for all the listed products, please refer to the provider's website.

Across much of the cloud ecosystem, the standard unit of configuration and pricing is the "virtual CPU", often abbreviated to vCPU. The majority of vendors use this unit in their descriptions. Arm Neoverse processors are single-threaded, currently with Neoverse E1 being the only exception, so each virtual CPU corresponds to a single physical CPU. Multi-threaded cores from providers such as AMD or Intel may implement multiple vCPUs on a single physical core. Oracle (see below) employs a slightly different nomenclature.

The vendors listed are distributed across the world. Some operate only in particular geographies, and some may have limited offerings in certain locations. As usual, check with each vendor's website for up-to-date availability information.

6.3.1. Google Cloud

Google Cloud makes Arm instances available in a number of ways, all through the General-purpose Machine Family within Google Cloud Compute Engine.

The Tau T2A series is powered by 64-core Ampere Altra processors (based on Neoverse N1) and is Google Cloud's most cost-effective general–purpose compute range, suitable for low-traffic web servers and virtual desktops.

The C4A series is powered by Google's in-house designed Axion processor, based on Neoverse V2 and is suitable for high-performance general-purpose compute, high–traffic web servers, and multi-media streaming.

Based on Ampere Altra

t2a–standard – general–purpose compute, 1–48 vCPUs, 4 GB per CPU

Based on Google Axion

c4a–standard—general-purpose compute, 1–72 vCPUs, 4 GB per CPU

c4a–highcpu—compute-optimized, 1–72 vCPUs, 2 GB per CPU

c4a–highmem—memory-optimized, 1–72 vCPUs, 8 GB per CPU

6.3.2. Microsoft Azure Compute Platform

Arm processors power a number of virtual machines in the Compute Platform family. These include:

Bpsv2—cost–effective general-purpose compute, Ampere Altra, 2–16 vCPUs, up to 4 GB per CPU (low–traffic web servers, small databases, develop and test platforms)

Dpsv6—high-performance general-purpose workloads, Cobalt 100, 2–96 vCPUs, up to 4 GB per CPU, also available with local disk storage

Dpsv5—non-memory-intensive workloads, Ampere Altra, 2–64 vCPUs, 2 GB per CPU, also available with local disk storage

Epsv5—memory-intensive enterprise workloads, Ampere Altra, 2–32 vCPUs, up to 8 GB per CPU, also available with local disk storage

6.3.3. Oracle

Oracle Cloud Infrastructure (OCI) offers a combination of configurable Virtual Machine (VM) instances and Bare Metal (BM) instances. These are supported on two classes of Arm platform, both implemented on Ampere processors:

Standard.A1—based on Altra

Standard.A2—based on AmpereOne

OCI makes available Oracle's full developer stack, including Oracle Linux, Java, and MySQL, on either of the two Arm platforms. Irrespective of which platform you choose for your development, Oracle's method of calculating core count is slightly different from most other vendors. In place of the widely used vCPU unit, Oracle use the term "Oracle

CPU" (OCPU), which always equates to a single physical processor, which may support more than one execution thread. For non-Arm instances, which support multi-threading, one OCPU equates to one physical core, usually supporting two execution threads (other vendors may refer to this as *two* vCPUs). The Oracle Arm A1 instances support one physical core and one execution thread per OCPU; the Oracle Arm A2 instances support two physical cores and two execution threads per OCPU.

Oracle refers to the various configurations of virtual machines as "shapes". The following Arm shapes are supported:

VM.Standard.A1.Flex—1–80 OCPUs (OCPU=1 core, 1 thread), 1–64 GB per OCPU

VM.Standard.A2.Flex—1–78 OCPUs (OCPU=2 cores, 2 threads), 1–64 GB per OCPU

BM.Standard.A1—Bare metal Ampere Altra Q80–30, 3.0 GHz (OCPU=1 core, 1 thread)

6.3.4. Amazon Web Services

All Amazon Web Services (AWS) Arm instances are based on the Graviton cores developed by Annapurna Labs, and are made available via the Amazon Elastic Compute Cloud (EC2). The Graviton–based Arm instances are highlighted as a cost-effective choice in terms of performance per dollar. While not based on Neoverse, it is worth noting that AWS also makes available instances for Apple Mac hardware, powered by Arm-compatible Apple silicon (such as the Apple M1 and M2).

All the Arm instances use the Amazon Nitro lightweight hypervisor for improved security and performance. Most are available in bare-metal form as well as with various configurations of memory, local and remote storage, networking bandwidth, etc.

As expected, there is a wide range of configuration options, optimized for various categories of workload, including:

General Purpose

M8g—Graviton4, highest-performance general–purpose compute, 1–192 vCPUs, up to 4 GB per core

M7g—Graviton3, general-purpose compute, 1–64 vCPUs, 4 GB per core

M6g—Graviton2, balanced compute, memory and networking for a range of workloads, 1–64 vCPUs, 4 GB per core

T4g—Graviton2, burstable general-purpose workloads, 2–8 vCPUs, up to 4GB per core

Compute Optimized

C8g—Graviton4, high-performance compute–bound workloads, 1–192 vCPUs, 2 GB per core

C7g—Graviton3, compute-intensive workloads, 1–64 vCPUs, 1–2 GB per core

C7gn—Graviton3E, high-performance packet–processing, 1–64 vCPUs, 1–2 GB per core

C6g—Graviton2, cost-effective performance, 1–64 vCPUs, 2–4 GB per core

C6gn—Graviton2, high throughput networking, 1–64 vCPUs, 2 GB per core

Memory Optimized

R8g—Graviton4, high-performance, 1–192 vCPUs, 8 GB per core

R7g —Graviton3, memory-intensive workloads, 1–64 vCPUs, 8 GB per core

R6g—Graviton2, cost-effective high-performance, 1–64 vCPUs, 8 GB per core

X8g—Graviton4, best price-performance, 1–192 vCPUs, 16 GB per core

X2gd—Graviton2, lowest cost per GB, 1–64 vCPUs, 16 GB per core

Storage Optimized

I4g—Graviton2, best price per TB storage, 2–64 vCPUs, 8 GB per core

Im4gn—Graviton2, best price-performance, 2–64 vCPUs, 8 GB per core

Is4gen—Graviton2, best SSD density, 1–32 vCPUs, 6 GB per core

HPC Optimized

Hpc7g—Graviton3E, compute-intensive HPC, 16–64 vCPUs, 128 GB total

6.3.5. Alibaba Cloud

Alibaba Cloud's Elastic Compute Service (ECS) provides a number of Arm instances. Some are based on the Ampere Altra processor, while others employ a variety of processors from Chinese vendors—some of which are not commercially available to purchase. Most of the Chinese instances are only available in China. All advertise high efficiency and cost-effectiveness.

Based on Yitian 710 (Armv9.0–A, developed in–house and not commercially available)

g8y—general-purpose compute, 1–128 vCPUs, 4 GB per core

c8y—compute-optimized, 1–128 vCPUs, 2 GB per core

r8y—memory-optimized, 1–128 vCPUs, 8 GB per core

Based on Ampere Altra

g6r—general-purpose compute, 1–64 vCPUs, 4 GB per core

c6r—compute-optimized, 1–64 vCPUs, 2 GB per core

ebmgn6ia.20xlarge—fixed-configuration, bare-metal instance, 80 CPUs, 256 GB

Based on Kunpeng920 (Armv8.2–A, manufactured by HiSilicon)

s5–kp—shared instances, 2–64 vCPUs, 1–8 GB per core

g5s–kp—dedicated instances, 2–56 vCPUs, 1–8 GB per core

g5m–kp—dedicated instances, 2–88 vCPUs, 1–8 GB per core

g5x–kp—dedicated instances, 2–120 vCPUs, 1–8 GB per core

Based on FT processors (Armv8.0–A, manufactured by Phytium)

s5–ft–k—FT–2000+ processor, shared instances, 2–64 vCPUs, 1–8 GB per core

s6–ft–k—FT–S2500 processor, shared instances, 2–64 vCPUs, 1–8 GB per core

g5s–ft–k—FT–2000+ processor, dedicated instances, 2–56 vCPUs, 1–8 GB per core

g6x–ft–k—FT–S2500 processor, dedicated instances, 2–120 vCPUs, 1–8 GB per core

6.3.6. Equinix

Equinix provide cloud services and IT infrastructure support worldwide. They offer bare-metal instances based on Ampere Altra processors as part of this service.

c3.large.arm64—Ampere Altra, bare-metal instance, 80 cores, 256 GB memory

6.3.7. Scaleway

Scaleway's Cost–Optimized service is based on the Ampere M128–30 processor and is offered as an energy efficient and cost-effective alternative to other instances. The Ampere M128–30 is a 128-core processor built to Armv8.2–A.

COP–ARM—Ampere M128–30, bare-metal instance, 2–32 vCPUs, 8 GB per core

6.3.8. Hetzner Cloud

The Hetzner cloud compute offering includes instances based on Ampere Altra processors.

CAX—Ampere Altra, shared instances, 2–16 vCPUs, 2 GB per core

6.3.9. Tencent

Tencent's Cloud Virtual Machine product includes a single Arm-based instance, based on the Ampere Altra processor.

Standard SR1—Ampere Altra, general-purpose compute, 1–64 vCPUs, 2–4 GB per core

6.3.10. Gcore

The GCore Virtual Machine product makes instances available supporting either Linux or Windows, based on the architecture Armv8.2–A Ampere Altra Max processor.

a1–Standard—optimized for cloud workloads, 1–32 vCPUs, 2–4 GB per core

a1–cpu—high-performance cloud workloads, 2–32 vCPUs, 1 GB per core

6.3.11. GleSYS

GleSYS's Dedicated Server product includes the following dedicated Arm instances, based on Ampere processors.

Small.arm64—Ampere Altra Q80–26, 80 vCPUs, 64 GB memory

Medium.arm64—Ampere Altra M96–28, 96 vCPUs, 256 GB memory

Small.arm64—Ampere Altra Max M128–30, 128 vCPUs, 1 TB memory

7. Market Positioning and Competitive Analysis

The Neoverse™ cores represent a new market position for Arm® technology.

As you have seen in the context of Arm's history, the early commercial success of Arm was based on achieving very high volumes. Royalty rates were deliberately set low, so large volumes were necessary to sustain sufficient income for the company. This model worked very well for the mobile phone market—the market in which Arm first achieved significant success. With the widespread adoption of mobile phones by the public across many geographies, this market achieved very high volumes but remained very price sensitive. The traditional Arm "virtues" of cost-effectiveness, power efficiency, and flexibility of deployment led to Arm's success in this market. The enormous volume growth achieved by mobile phone vendors in the 1990s and 2000s powered matching growth in Arm's fortunes.

Arm subsequently moved into other markets, such as digital cameras, modems and routers, storage devices, and smart consumer devices. These markets displayed similar economics. The devices are predominantly powered by batteries, so the highly power efficient Arm processors were a compelling choice. High volumes coupled with price sensitivity made these markets ideal targets for Arm, and the company achieved great success in many of them.

The move into the data center market, however, is markedly different. Here, volumes are orders of magnitude smaller, while unit prices are much higher. The key considerations are, therefore, quite different.

Power efficiency, unit cost, and processing power are all still important, but for different reasons and with different priority. Additional factors—such as per-core performance and scalability—become more important.

7.1. Cost–Effective Performance

The primary requirement for data center compute devices is sufficient processing power. The x86 devices, which have dominated this market for considerable time, are clearly capable of sustained high performance. To be seen as a viable alternative, Arm needed to

deploy processors capable of running similar workloads. The Neoverse processors were designed specifically to achieve this. Modeling showed, for instance, that coherent instruction caches increased performance significantly in multi-processor systems such as the typical systems in a data center. In addition, high-bandwidth on-chip and off-chip interconnect is vital for moving the very large amounts of data involved in typical data center workloads. In order to work with very large datasets, the ability to address large amounts of memory is also important.

The Neoverse cores were specifically designed to these considerations. Benchmarks—both by Arm, and by independent vendors—have shown that Neoverse processors can compete with alternative platforms while remaining cost-effective and consuming less power.

7.2. Sustainability

In earlier markets, the need to conserve battery life drove ever greater power efficiency in Arm processors. A wide range of sleep, deep-sleep, and hibernation power modes allowed the processors to enter and leave power-saving modes quickly and efficiently. In the data center, power consumption matters for a different reason: huge numbers of processors are deployed in close proximity, needing sustained high performance for long periods of time, so heat dissipation and total system power consumption become critical.

There are several ways of dealing with heat dissipation, including the use of active cooling systems, and a reduction in the power consumption of individual devices. The cooling systems deployed in a typical data center are complex and expensive, and themselves consume significant power. It is therefore helpful to concentrate on reducing the power consumption of the devices themselves. While this reduces the cost of running them, it also allows deployment of simpler and most cost-effective cooling solutions. This reduces both the build cost and running cost of a typical data center.

In today's environmentally conscious world, the reduction of total energy consumption is an increasingly important design goal, making Arm processors a more attractive solution. Their energy efficiency can help offset the rising energy demands of data centers driven by the growing use of compute-intensive artificial intelligence and machine learning workloads.

7.3. Scalability

Scalability—the ability to keep performance approximately proportional to the number of cores—is a key requirement for data center operators. Arm addresses this, and allows its customers to address this, in multiple ways.

Recall that Arm provides designs for processors and their associated silicon infrastructure. Arm's partners can deploy these in many different sizes and configurations. The number of cores and how they are interconnected provides huge flexibility in the capability of the finished device. This allows Arm's partners to provide devices targeted at a very wide range of deployments—from single-processor to many-processor. As you have seen, devices are currently available with up to 256 cores in a single chip. This allows partners to target a wide range of price and capability points across the market. Many of these devices are plug- or pin-compatible, allowing smaller chips to be replaced with larger ones as demand increases.

The scalability enabled by these design choices also allows devices to be targeted at deployment from the data center to the edge of the network, retaining architectural and software compatibility across the range.

7.4. Total Cost of Ownership (TCO)

TCO is an aggregate of several factors: the initial cost to build individual devices and complete data centers; power and other running costs for the data center as a whole—including the need for cooling systems; and costs associated with deploying software applications.

Arm-based systems allow reductions in all these individual costs, enabling a data center solution that is both cheaper to build and cheaper to run.

7.5. Ecosystem Advantages

One of the great strengths of Arm has always been its commitment to working closely with as many partners as possible to build a broad and deep ecosystem of software and tooling support. The Arm ecosystem has been regarded for some time now as the largest around a single architecture the industry has ever seen.

The data center industry has long been based on x86 architecture platforms and the huge body of standard software available for them. Traditionally, it has been regarded as expensive to port necessary software components—including firmware, operating systems, middleware, and applications—to any new platform. Arm and its partners have worked hard in recent years to ensure that the majority of necessary components are supported on Arm platforms and have good performance compared to competing solutions. The porting costs are now minimal, and the size of the ecosystem is growing all the time, leading to the prospect of better and cheaper support in the future.

Part 2—The Software Stack

8. Diverse Use Cases Need Diverse Cores

The Neoverse™ range of cores spans a wide breadth of performance points, capabilities, and applications. Neoverse V represents the highest possible performance and is used for high performance compute (HPC) in the cloud. Neoverse N trades some performance for scalability and greater power efficiency when deployed in multi–core applications in infrastructure. Neoverse E is optimized for data throughput and is typically deployed towards the edge of the network—for example, in a smart NIC. In addition, Neoverse E and Neoverse N support similar software, allowing common applications to be run on both relatively easily.

The Neoverse range is further expanded and diversified by a wide variety of custom cores, designed and implemented by partners of Arm®. These cores have different and specific balances between performance, power efficiency, scalability, connectivity, and cost-of-ownership.

8.1. Supported Software

Because there are many diverse use cases for Neoverse systems, Arm also supports a large number of software packages. Arm maintains a list of these on its website, here: https://www.arm.com/developer–hub/ecosystem–dashboard/servers–and–cloud–computing. The list is constantly being updated as the number of supported packages evolves over time.

At the time of writing, this list contains over eight hundred software packages, including commercial software as well as free and open source software (FOSS), across a wide range of product categories, including operating systems, databases, HPC, security, web hosting, artificial intelligence and machine learning, networking, storage, and more.

In the category of operating systems, there are twenty-six products listed. Of these, twenty are FOSS—including mainstream Linux offerings such as Red Hat, Ubuntu, SUSE, Fedora, Debian and others. The commercial offerings include Windows, as well as enterprise–grade versions of Red Hat and SUSE.

Of particular interest to migrating developers is the fact that the largest category of supported packages are cloud-native development tools.[16] The list includes offerings from popular vendors such as CircleCI, Apache, Azure, Daytona, Docker, GitHub, Harness, Kubernetes, Nomad, and Oracle. Alongside this is a comprehensive set of languages and development tools, including compilers, debuggers and profilers. The size and coverage of this list means that the majority of developers will be able to migrate to a Neoverse platform without changing their tooling solution.

8.2. Example Platform

For the software developer seeking to deploy applications on a Neoverse processor that is accessed via a cloud instance, the target device will probably be based on Neoverse V or Neoverse N, implementing one of the Armv9–A architectures. The following discussion makes this assumption.

To focus the discussion further, we will refer to a Reference Design (RD) based on one of Arm's Neoverse Compute Subsystems (CSS). Each CSS is provided by Arm to its partners in order to shorten the development time and to reduce the engineering cost, for partners to get to market with a complete device. Importantly, this allows partners to focus their in-house resources on customizing the device for their own specific environment and workloads—increasing the amount of differentiation they can offer. These CSS products also come with reference software stacks—from boot firmware to hypervisors and Linux ports—which are made available via a public GitHub repository.

Additionally, Fixed Virtual Platform (FVP) models are available. These are configurable software simulation models of real hardware that allow software to be developed, executed, tested, and validated in the absence of a physical platform. This removes the need for access to physical devices, and also allows software to be written for platforms that are in development but not yet physically available.

16 "Cloud–native" refers to a development environment in which the development tools (compilers etc) execute on the same, or an identical, platform to which the developed application will be deployed.

For silicon developers, the impact of implementation configuration choices can be tested during the development of the device. However, please note that execution speeds on a software simulation will be significantly slower than a corresponding hardware platform.

Specifically, we will refer to the Neoverse N2 Reference Design, which is based on the Neoverse CSS N2 platform. The Neoverse N2 CSS is a highly configurable design that offers significant scalability to the licensee. It supports up to 64 Neoverse N2 processors, together with ancillary components, cache, memory interfaces, interconnect, and interfaces for connection to on-chip and off-chip components. The size and configuration of many of these components is chosen during the design and manufacturing process by the licensee.

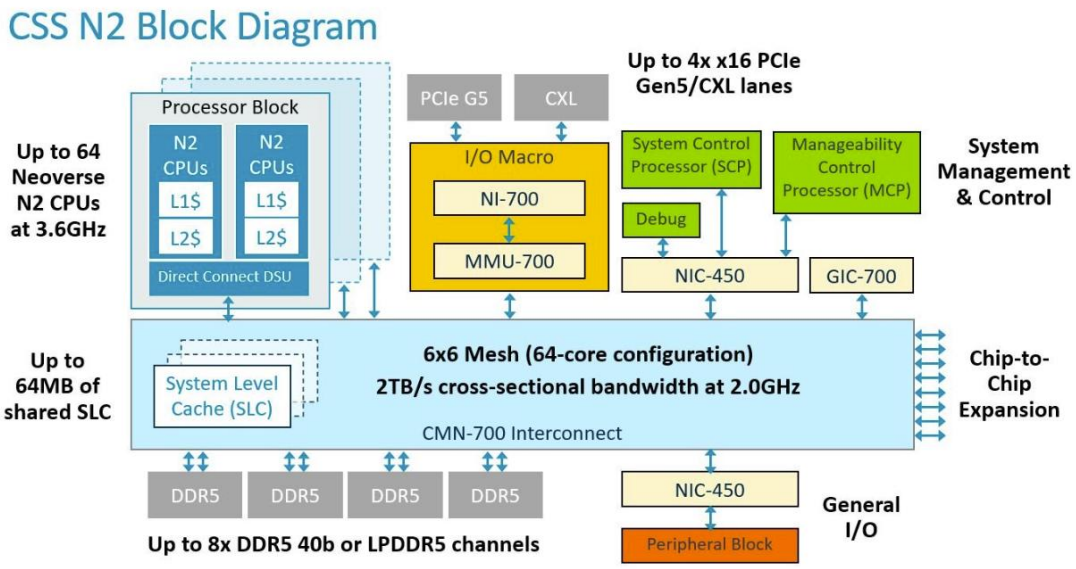

Figure 5: Neoverse CSS N2 Block Diagram

A diagram of Neoverse CSS N2 is shown in 5. Components shown include:

Processor blocks

In the delivered configurations of Neoverse CSS N2, each processor block contains a single Neoverse N2 processor, together with a private L1 I–cache, a private L1 D–cache, and a private unified L2 cache. Each of these components is connected to the rest of the

system via a Dynamic Shared Unit (DSU), which contains interface bridges, power management logic, and debug logic. The Neoverse N2 CSS is capable of supporting 24, 32 or 64 processor blocks.

Coherent Mesh Network Neoverse (CMN–700) interconnect
This is the main system interconnect infrastructure, connecting all the system components in a mesh topology. It contains the shared system level cache, and provides interfaces to on-chip and off-chip I/O, PCIe, on-chip DRAM, and die-to-die connectivity.

System Control Processor (SCP)
Manageability Control Processor (MCP)
Two Cortex®–M7 processors, which provide management and control of the device, power monitoring of each individual processor and of the system as a whole, the external management interface (SCMI) and security support.

A diagram of the Neoverse N2 Reference Design is shown in Figure . The Neoverse N2 Reference Design is compliant with Arm Server Base System Architecture version 6.0, indicating compatibility with many standard software products.

The Neoverse N2 Reference Design implements the Neoverse CSS N2 with the following configuration choices:

- 32 instances of the processor block—each including a single Neoverse N2 processor with a 64 kB private L1 I–cache, a 64 kB private L1 D–cache, and a 1 MB private, unified L2 cache;
- a third–party DDR memory controller; and
- 32 MB System Level Cache in the interconnect.

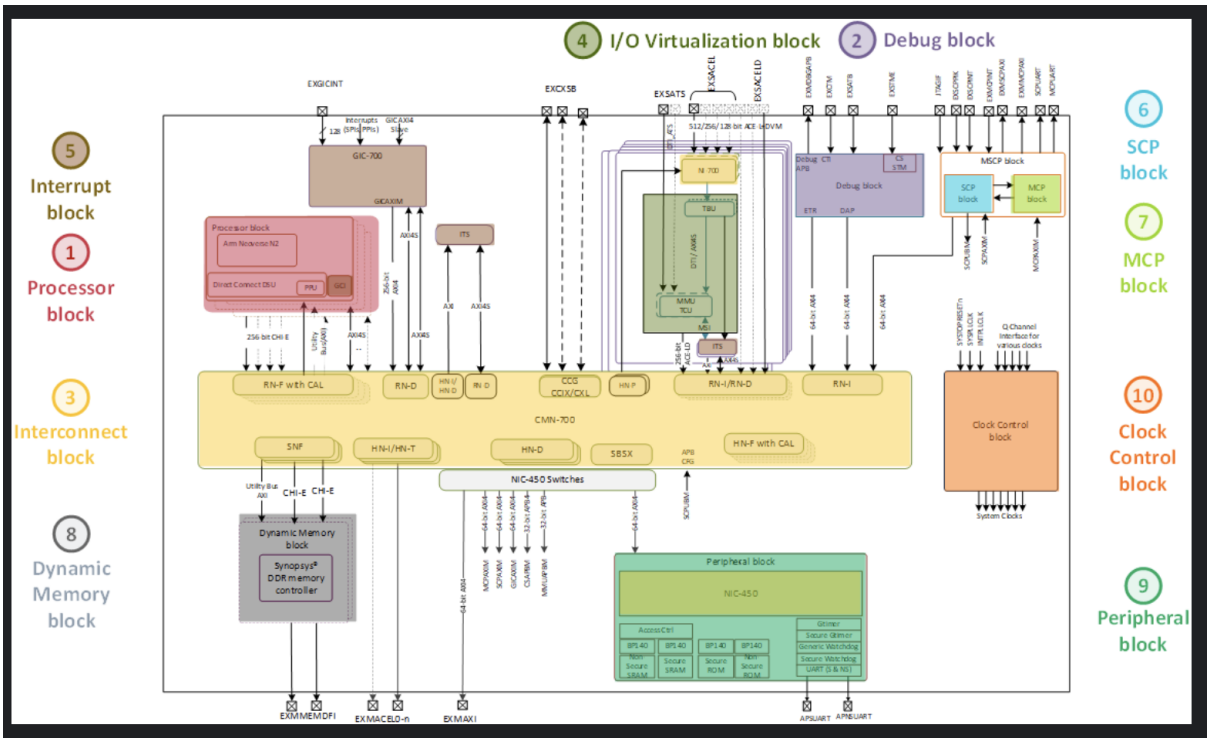

Figure 6 – Neoverse N2 Reference Design

For readers with an interest in system-on-chip architecture, the final chapter in this section contains a brief overview of many of the connectivity components listed above.

This discussion won't cover in detail the software running on any of the support processors (SCP and MCP), nor the (rather complex) boot process needed to get the device from power-on to a usable state running a Linux kernel. Further detail about these topics, and the platform itself, can be found on Arm's website.[17]

17 https://neoverse-reference-design.docs.arm.com/en/latest/about/software_stack.html

9. Security Considerations

Note: This section describes the Confidential Compute Architecture (CCA). This technology has been developed by Arm® in recent years and has been released to partners in the latest processor core designs. CCA is supported by a range of software models upon which software can be developed and tested. Wider market adoption of this technology requires the coalition of many partners, including regulators and service providers.

Security has become an increasingly important design consideration in recent years. For the cloud computing industry, it has become a foundational issue, chiefly for these reasons:

- Cloud resources are accessible from anywhere by anyone on the internet.
- The same physical resources are frequently shared by multiple users.
- Resources are often dynamically allocated on demand.
- Software uploaded and executed by users is generally not vetted.

The cloud computing industry requires systems to be accessible via the internet. This introduces the risk of unauthorized access and use. These open systems can no longer hide behind access restrictions—no matter how sophisticated—and instead must incorporate robust protection against misuse by potential malefactors who may have gained some level of access to the system itself.

In a cloud computing environment, multiple users may be sharing access to a single physical processor—or cluster of processors—running arbitrary software that hasn't been vetted. It is important to isolate users from each other adequately, so that malefactors can't prejudice, interfere with, or observe the activities of other users on the same system.

To address this, modern security architectures typically rely on some form of "compartmentalization". This ensures that if a malefactor does gain access to the system in some way, the compartments restrict what they can do once inside the system. This also protects against actions by legitimate users of the system, who may—whether deliberately or inadvertently—execute software that is prejudicial to other system users.

Such compartmentalization architectures usually rely on hardware support built into the processor itself. For example, Arm's Confidential Compute Architecture (CCA) is supported by the latest Armv9–A cores, and TrustZone® is supported by all application–class cores since Armv7–A.

However, not all software can run in such a compartmentalized environment. In particular, code that is part of the boot process must have essentially unfettered access to the system in order to initialize hardware components, launch device drivers, and set up the secure environment before it passes control to a hypervisor or one or more operating systems. The hypervisor or operating systems are then responsible for loading and launching applications in a secure and protected manner to provide the necessary isolation. In any case, boot code executes before any security systems have been initialized. We shall look at the boot sequence in more detail.

Another important principle is the need to minimize the proportion of the system that must be "trusted". That minimal component can then be tested and verified to a very high degree of assurance, upon which the rest of the security rests.

In particular, it is important to recognize that common software components—such as the operating system and hypervisor—cannot be trusted to the same extent. They are simply too large and complex to be robustly verified and certified.

Referring to the system structure shown in Figure 8, notice that the entire Linux kernel, together with any application workloads running on it, is regarded as untrusted and executes in the Non-Secure World. Security architectures like CCA and TrustZone remove the need to trust the operating system or hypervisor. Going beyond the separation provided by TrustZone, Arm CCA allows the hypervisor to exert control over any virtual machines but removes the right of access to any code, state, or data information used by or within that virtual machine. This allows developers to deploy workloads securely without needing to trust the software infrastructure supporting them.

On the Application Processor (AP), only a small body of software—the "Secure Runtime Services"—executes in the Secure World.

On systems that support RME (see Figure 8) the majority of the drivers, together with the code responsible for switching between the Secure and Non-Secure Worlds, are executed in Root state. User workloads can either execute in the Non-Secure World on top of the Linux kernel, or in compartmentalized realms in Realm state.

The following points are worth noting:

- A workload executing in a Realm is invisible not only to other workloads in Realm state but also invisible to the operating system.
- The body of software executing in Root and Secure states is small, bounded, and can be verified prior to execution.
- User–generated workloads, running in Realms, are isolated so that they can neither interact with each other nor with the rest of the system.

Hardware debug features must be carefully considered and controlled, because they require extensive access to on-chip resources and internal information. All Arm Neoverse™ Reference Designs implement some form of debug authentication. This requires external debuggers to identify and authenticate before the system will provide access to internal static and dynamic information. The debug access to AP, MCP, and SCP are separately authenticated. Some systems, including the Arm Neoverse Reference Designs, internally disable some debug features if the debugger cannot successfully be authenticated.

10. A Typical Software Stack

The software stack that executes on the Application Processor (AP)—the Neoverse™ processor is only one part of the story, because the AP is only one of several processors in the system. To support the AP, management and control tasks are often delegated to support processors. The example shown in Figure 7 shows a typical system configuration of an Application Processor (AP), System Control Processor (SCP), and Manageability Control Processor (MCP).

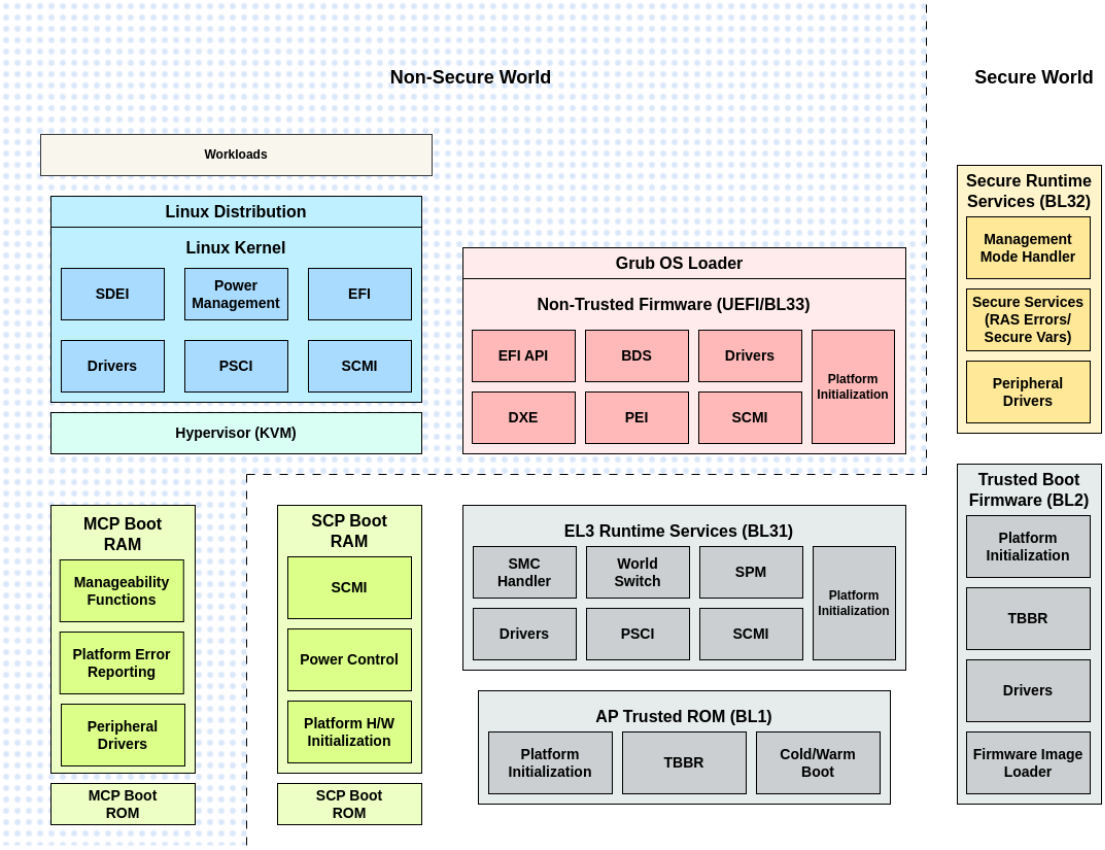

Figure 7 – High–level software illustration of a Neoverse Reference Design.

The slightly more complex, example in Figure 8 shows the configuration when Confidential Compute Architecture (CCA) is implemented. This requires the addition of Realm Software (executing on the AP) to manage the Realm Control Extension hardware.

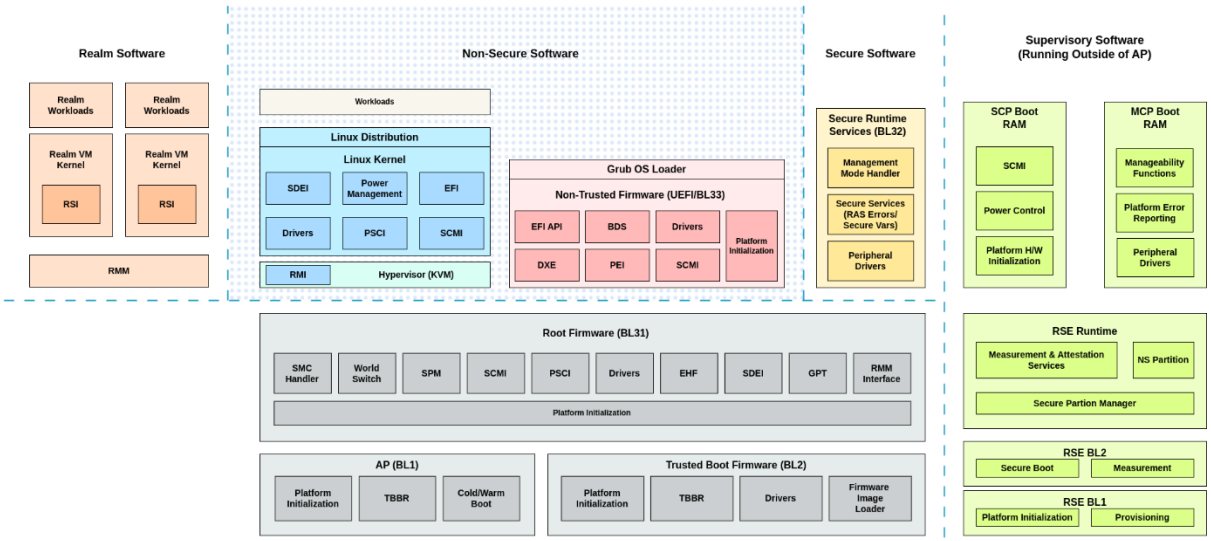

Figure 8 – High–level software illustration of a Neoverse Reference Design with RME.

The AP, SCP, and MCP are physically distinct processors—although in some systems, a separate processor executes the Runtime Security Engine functions. Each processor executes a separate software image and communicates with other processors using on–chip interconnect pathways. (The details of how this communication occurs and how it is implemented need not concern us here.)

The distribution of tasks between these components is typically as follows:

Application Processor (AP)	Non–Secure	Workloads[†] Operating system (typically Linux)[Error! Unknown switch argument.] Hypervisor (typically KVM) Boot loader (typically UEFI) Non–trusted firmware
	Secure	System boot code Trusted boot firmware Secure run–time services (including secure devices drivers and error handlers) EL3 (hypervisor) run–time services

†† There may be multiple operating systems—each running multiple workloads—under the control of a single hypervisor running on the Application Processor.

	Realm	Realm Management Software
Manageability Control Processor (MCP)	Non–Secure	Platform management functions Peripheral drivers System Control & Management Interface (SCMI)
System Control Processor (SCP)	Secure	Power control Hardware initialization System clocks On–chip interconnect Memory controllers PCIe controllers
Runtime Security Engine[18] (RSE)	Secure	Boot control Provisioning state machine

In general, the SCP handles internal monitoring, management, and control, while the MCP handles the management connection with the world outside the device, particularly the interface with the management application in a server environment.

The software executing on these two processors is provided by the equipment manufacturer and can generally be regarded as immutable by end users. It is typically based on reference software provided by Arm®.[19]

We now look at the boot sequence—beginning from reset and typically ending with the launch of the operating system.13

18 In some systems, the functions of the RSE are carried by the SCP. This simplifies the hardware architecture somewhat.

19 While Arm provides implementations of these components as part of its Reference Designs, this software should not be treated as of "production quality". It is intended merely as an example for device manufacturers.

11. The Boot Sequence: Hardware to Operating System

As might be expected, the boot sequence in a complex, multi-processor system is inevitably intricate. It requires a chain of secure steps, designed to give a high level of confidence in the security of the running system. This is often referred to as the "Root of Trust"—just like how trees are anchored into the soil by their root system. If you sever the roots, the tree will die; if you break the "Root of Trust", the security of the entire system collapses.

As protection against malicious boot software, any boot code component that is dynamically loaded is typically supplied in an encrypted form and must be decrypted and verified before being executed. Each stage carries out its designated functions before downloading the subsequent stage, verifying it, and then transferring control to it. This approach requires a tamper-proof mechanism for storing the necessary encryption and validation keys.

Prior to the boot process, during chip and device manufacturing, a separate provisioning process installs the chip and device keys, configuration data, manufacturer keys, and information.[20] The initial boot code (BL1_1) is built into on-chip ROM during manufacturing; the second stage boot code (BL1_2) is programmed into one-time programmable (OTP) memory during device provisioning. This provisioning process is a once-only, one-way process that installs code and keys so that they are securely stored and cannot be subsequently changed.

Following provisioning, subsequent resets will start execution from the on-chip secure boot ROM. The first stages are under the control of the SCP and MCP, which validate their respective initial software images. Once the MCP and SCP have started, the SCP boots the AP. All this takes place initially on the primary CPU on the primary chip. Once it is complete on that processor, secondary processors and chips are then booted.

20 This is analogous to the storage of keys in external Trusted Platform Modules, a technique used in many system architectures.

It is important to note that the entire process proceeds, step by step, by loading and executing validated software images. The earliest of these images are inserted during manufacture and are immutable thereafter. This is the foundation of the "Root of Trust" mentioned above. Similarly, at each stage, validation is carried out using immutable keys, which are inserted during manufacture and are unique to the device.

In overview, the sequence, shown in Figure 9, is as follows:

01 MCP and SCP

a. MCP and SCP boot from their respective boot ROMs.
b. MCP waits until the SCP has established PLL lock.
c. MCP and SCP load images from on-chip RAM or off–chip flash into TCM.
d. MCP and SCP validate the loaded images.
e. MCP and SCP jump to TCM.
f. MCP and SCP exchange keys to establish mutual trust before continuing.
g. SCP enables the Application Processor's clock circuits.
h. SCP powers up one core (typically CPU0).

02 Application Processor (AP)

a. AP boots from the BL1 image in secure ROM.
b. AP BL1 loads and validates BL2 image from Flash, and transfers control to it.
c. AP BL2 loads and validates BL31 image from Flash, and transfers control to it.
d. Successful completion of BL31 boot signaled to SCP.
e. AP BL31 configures the interrupt framework, power management interface, and secure partition manager.
f. AP BL32 initializes Secure Partition Manager (EL3) and returns to AP BL31.
g. AP BL31 transfers control to UEFI (Non–secure EL2).
h. AP UEFI transfers control to the operating system (OS).
i. OS requests power-on of secondary CPUs on primary chip.
j. OS requests power-on of CPUs on secondary chip(s).

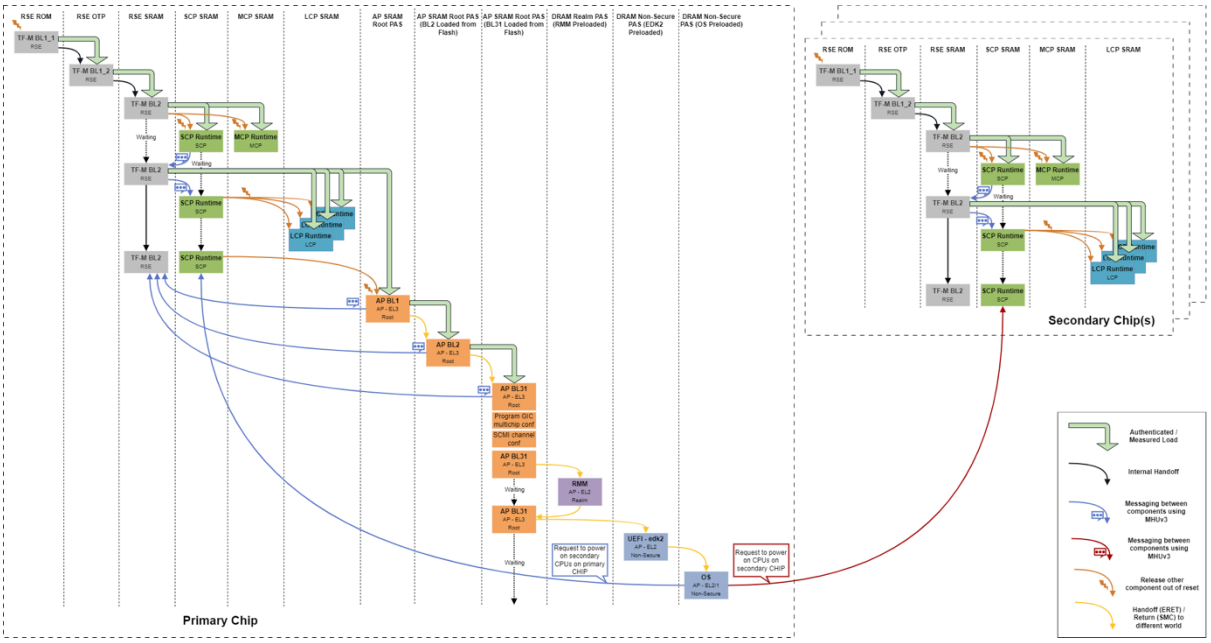

Figure 9 – Boot Sequence Flow Diagram

12. Device and Platform Discovery

It should be apparent by now that no two Neoverse™-based platforms are the same! While this may sound like a disadvantage, this is a manifestation of one of the key strengths of the Arm® ecosystem. Since Arm's silicon partners aren't permitted to change the behavior of the processor itself, nor the behavior of key peripherals and on-chip infrastructure, the way these components function is consistent across all Arm-based platforms based on any given version of the architecture. Arm ensures this by insisting on compliance testing for all products developed using Arm technology.

Beyond this, the licensee can introduce any amount of differentiation in their end product. It is this differentiation that—for Arm's partners—represents a considerable benefit of using Arm technology. Freed from the cost of developing their own processor and related intellectual property (IP), they can concentrate resources on their own product-specific IP. For end-users, the ability to rely on consistent behavior across a huge range of very diverse products allows much more efficient re-use of software, and better expertise across products, companies, and industry sectors.

However, this diversity means that if a software product is used across several platforms, it requires a method for determining the details of the hardware on which it is executing, so that its behavior can be tailored accordingly. At the most obvious level, this would include device drivers, which will necessarily differ from platform to platform.

Fortunately, the industry has standard mechanisms for dealing with this.

The firmware—provided by a combination of Arm, device manufacturer, product developer, and software developer—abstracts away the vast majority of hardware specificity. This, on its own, means that most software products will "just work" on a given Arm platform.

Firmware typically uses one of two methods for adapting to the hardware:

Device Tree

This is a description of the hardware platform in a standard format. This format is widely

used by Arm-based Android devices, in embedded development, and systems using the U–Boot. It is no longer widely used for server-class deployments.

Advanced Configuration & Power Interface (ACPI)

This is essentially an abstract description of the hardware, encoded in a set of tables that contain information about the hardware itself, as well as software elements (such as device driver components) that are incorporated into the system at run time. ACPI tables are less restrictive than Device Trees.

Every device manufacturer produces a set of ACPI tables for their platform, and these are in an industry-standard format. They are particularly used by systems based on the UEFI standard.

Initially developed for x86-based systems, ACPI has supported Arm systems since 2011.

Of the two possibilities, ACPI is more common and more capable. It is preferred by Arm and other major vendors of processor-based platforms, and used by all the major Linux distributions. The format is governed by the UEFI Forum.

Application software developers don't need any knowledge of how this system works. They can simply rely on the majority of system software using it and therefore being usable across a wide range of platforms—both Arm and non-Arm.

13. Operating Systems & Hypervisors

(For an updated list of software packages supported on Arm, refer to: https://www.arm.com/developer-hub/ecosystem-dashboard/servers-and-cloud-computing)

The majority of applications use a standard operating system. Although other options are possible, this operating system will likely be some form of Linux and we shall make this assumption for the remainder of this discussion.

The choice of Linux distribution is not necessarily influenced by the choice of an Arm-based platform. Since the majority of mainstream distributions are supported on Arm, and have been for some time, there is essentially the same choice for Arm developers as there is for users of other architectures. The same criteria apply when choosing any particular distribution: stability, middleware support, security considerations, compatible development tools, commercial licensing terms, etc.

It is important to remember that the operating system is not the only software executing on the system. It relies upon layers of supporting software which provide the interface between the platform and the operating system kernel.

Bear in mind, also, that the Application Processor (AP) is not the only component in the system that executes software. It relies on support functions carried out by the SCP and MCP, which execute separate firmware. The SCP is responsible for low-level system management, and the MCP is responsible for manageability functions.

To recap the more detailed discussion in §11, the supporting firmware executing on the AP—loading and supporting the operating system kernel—includes:

- Arm Trusted Firmware BL1
  BL1 is contained in an on-chip ROM, fixed at time of manufacture
- Arm Trusted Firmware BL2
  BL2 is loaded from external non-volatile storage into on-chip RAM

– Arm Trusted Firmware BL31
  BL31 manages the Power State Coordination Interface (PSCI), Secure monitor framework, and Secure partition manager
– Secure runtime services BL32|
  BL32 is runtime software that runs under the management of the Secure partition manager
– Bootloader BL33
  BL33 is the final component in the boot sequence, typically a UEFI-compliant bootloader providing the interface between the firmware and the operating system

More information about these components can be found in the Arm Neoverse™ N2 Reference Design Technical Overview.

The Arm Neoverse N2 Reference Design Linux kernel is provided to demonstrate the platform features, and does not implement sufficient functionality for typical use cases. The rest of this section introduces the Linux implementations that are likely to be found in widely available Neoverse systems. You will need to check which kernels are available for the particular platform that you plan to use. Alternatively, if you have already selected a particular kernel, then you will need to choose a platform on which it is supported.

All of these software components are available from the Reference Design Github repository.[21]

The main choice to make at this point is whether to use a vendor-specific Linux distribution, or to use a cross-platform distribution. We discuss both below. The choice is driven by one or more of the following:

– New or ported software
  If you are porting an application from another platform, and your current distribution is supported on the destination platform, you may wish to retain the same Linux distribution to minimize disruption when porting.

21 As "Reference Designs" these should be treated as examples rather than production–ready software. Device and platform manufacturers will have their own implementations of this functionality which will be distributed with those devices and platforms.

- Hardware support
  Applications that rely on particular hardware features that are present in only some platforms may wish to opt for a vendor-specific or platform-specific distribution in order to ensure the best and most up-to-date support for those features.

- Licensing terms
  Changing distribution will almost certainly involve changing supplier and adopting new commercial and licensing terms. If you are already using a particular distribution, sticking with it may be a sensible choice for this reason. If you are developing a completely new product, then this would not apply.

- Stability and support
  Many developers opt for a longstanding and well-supported distribution from the open marketplace, in order to benefit from the stability that comes with maturity, and the support that comes with a commercial product. The cost is obviously weighed against the benefits here.

- Ease of use and availability
  Most device manufacturers and cloud hosting companies make a particular distribution, or a selection of distributions, available with their product. Choosing one of these is not just the "easy" option—these are likely to be supported directly by the supplier and be better targeted to their specific platforms. In particular, they will often be the first to support new features. Licensing and commercial terms for these platforms may also be simpler as they may be bundled with the terms for using the platform itself.

13.1. Cross–Platform: SUSE, Debian, RedHat, Ubuntu

On Microsoft Azure bare-metal instances, RedHat RHEL and Suse SLES are the options for the operating system.

14. Middleware

In a typical software application development environment, middleware is the glue that connects the applications to the operating system and the underlying hardware capabilities. For example, middleware allows applications written in different languages to communicate with each other and to share common underlying services. Middleware largely hides the details of the underlying platform from applications that wish to use the services it provides.

Middleware may be highly integrated and specific, such as a game engine that provides audio, video, animation, and game-play services to a gaming application. Or it may be fundamentally distributed and general-purpose, such as a web server providing database services to front-end applications that may be running on a different operating system on a physically remote system. It may be as simple as a driver for a specific type of hardware sensor, or as complex as a real-time, secure, distributed transaction manager for a banking system.

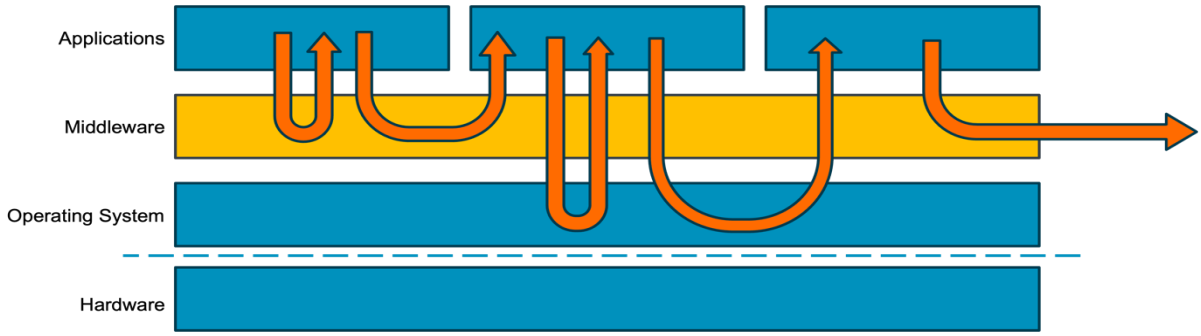

Figure 10 – The Place and Architecture of Middleware

Middleware can be described as the plumbing that allows disparate software components to communicate with each other, while hiding the details of how that communication actually happens.

Below are some examples of different kinds of middleware.

- Database
  A database access service (such as Hadoop or MySQL) allows applications to query a database using a standard query language, such as SQL.

- Runtime Environment
  Many programming languages (such as Python, Ruby or Java) need to execute in a standard environment at runtime. This environment makes available a standard set of services which the language needs.

- Networking
  Applications rarely run in complete isolation, and almost always need to communicate with other applications. They may be executing on the same platform, or on physically distinct and possibly remote platforms, on different processors. A networking layer (such as NFS or Cloudflare) provides these connection and data transport functions while hiding the underlying nature of the connection.

- Security
  For a variety of reasons, applications often need access to secure and standard means of encrypting data, either for local use or for transmission to other applications. Security middleware (such as OpenSSL or Bitdefender) handles this, as well as related functions such as key generation and secure key exchange.

The choice of middleware components can have significant effects on the performance and functionality of the final system. The criteria for choosing middleware on an Arm®–based platform are essentially the same as for any other platform.

You should consider the following factors:

- Commercial or non–commercial licensing terms
  This might include rights for redistribution, modification or re–use.
- Platform support
  Not all middleware components are supported on all platforms, so the choice will be limited by what is available on your chosen platform.

- **Cost**
  Licensing, use and distribution costs vary greatly between suppliers. The selection must fit the financial business case made for the application.
- **Compatibility**
  Some components are incompatible with others, or are not supported on some operating systems. It can be complicated to choose a selection of components that are all compatible.
- **Support services**
  The availability and quality of support vary greatly across the industry. In general, commercial offerings will come with a level of support; FOSS offerings may not.
- **Functionality and performance**
  You should evaluate which options meet the required features at the needed performance level. These factors should be addressed in the application requirements specification.

15. Cloud–Native Tooling

In cloud–native development, applications are built in the cloud, to run in the cloud. This requires that all the development tooling and infrastructure be available in the same target cloud environment.

It was not always this way.

In the earliest days of computing, all development, integration, test, debugging, and deployment took place on a single machine (or a single type of machine). Since computers were large, rare, inflexible, and extremely expensive, it was not generally feasible to develop applications on one machine and deploy them on another. Applications were developed on the machine on which they were deployed (or on an identical machine, in the case of an application developed by a vendor for deployment to a number of clients).

The advent of minicomputers and personal microcomputers changed this paradigm and made the process more flexible and more efficient. Using cross-compilers, it became possible for multiple developers to write software on their own machines before components were compiled for the target machine. Since the development and deployment machines were usually of very different architectures, this meant that the deployed software could not be executed on the development machines (without some form of emulation).

These days, compute resources are provided in the cloud, and are flexible, accessible, scalable, and easily manageable. It is again possible to develop, integrate, test, and deploy in a single end-to-end environment.

As well as convenience and efficiency of development, cloud-native has other advantages.

Greater reuse—it is much easier and more efficient for multiple applications to make use of common middleware and shared resources, since all are in the cloud.

Compartmentalization—individual components can be individually containerized, being connected via shared middleware into the complete system. This makes it easier to isolate and fix problems, and enables a divide-and-conquer approach to system design, performance optimization, and scalability. To fix isolated problems, only the affected components need to be replaced, and this can be done "in place". Additionally, components can be individually scaled to meet capacity requirements.

Simplified supply chain—with components being built to execute on the same underlying platform, using the same middleware APIs, there is less work involved to integrate components from a diverse supply chain.

Scalability—development resources can be provisioned and deployed instantaneously as required, eliminating the need to purchase and maintain expensive development platforms.

Reduced Development Cost—without the need to purchase and maintain development kit in the locations where developers are working, development time and cost are reduced.

High Availability and Resilience—it is no longer the job of the developer to maintain access to developer hardware. The cloud vendor is responsible for making the agreed resources available, at the agreed time, with the agreed service level. This reduces risk for developers.

Reproducibility—technologies such as Docker allow software to be developed and executed within containers. These containers can then be perfectly reproduced on the target platform. This allows software to be tested on the local platform in an environment that perfectly matches the target environment where the software will be executed.

Further, a modern DevOps (CI/CD) development flow relies on a high degree of automation throughout the develop–integrate–deploy–test cycle. Clearly, it would be simpler and more efficient if all the necessary automation tools are available in the same development environment, executing on the same platform.

While there are many options for suites of automation tools, many are well supported on Arm platforms, including Git, Docker, Kubernetes, Splunk, Nagios, Datadog, Maven, Jenkins, and many more.

16. Applications

There are two main routes that developers might take when seeking to deploy an application on an Arm-based Neoverse™ platform. First, development of a new application specifically targeted at Neoverse. Second, porting of an existing application from another architecture.

In the case of developing from scratch for Neoverse, the preceding discussions will help with selecting the right platform, choosing an operating system, integrating middleware, and finding and deploying the appropriate tooling for your development.

If you are porting from another architecture, there are some additional considerations to bear in mind.

Arm is a widely supported compiler target

Development for Arm is mainstream now, and the majority of compilers and related development tools support Arm as a target architecture. When specifying your target platform, it is important to be as specific as possible. There are two reasons for this.

Firstly, each new version of the Arm architecture introduces new features and enhancements—from new instructions to entirely new capabilities such as CCA. In order to make full use of the features of your chosen platform, you need to specify it as accurately as possible. If you specify the basic architecture version, such as Armv9–A, you can rely on newer features as well as the core architecture. If possible, be even more specific and specify the exact version, e.g. “Armv9.1–A”.

Second, processors that implement the same version of the architecture (and thus execute the same instruction set with identical functionality) may do so using differing internal microarchitectures. This may include differences in pipeline structure, cache architecture, memory interface speed and bandwidth, and on-chip interfaces. Without details of this information, the compiler cannot optimize for greatest instruction throughput, for instance, nor lay out data structures for either lowest memory use or greatest speed of access. So, specifying the exact processor is the best option, e.g. “Neoverse N2”.

Nearly all mainstream libraries and components are available on Arm

Given the wide support for Arm, your choice of libraries and middleware is likely to be available. Therefore, consider which ones have been optimized for your target platform, which are available on suitable licensing and commercial terms, and which might be better supported by your chosen cloud service provider.

Watch out for hardware–specific software elements

Applications originally written for other architectures may make use of machine-specific features. The most obvious example of this would be modules that make use of a specific instruction set via intrinsic functions or inline assembly language. This is often done to access instructions that the compiler cannot emit, or which it cannot utilize efficiently. Such sections will clearly need to be either removed, rewritten, or substituted with a suitable library.

Be aware of the flexibility in the Arm memory model

Arm processors, since Armv7, implement a "weakly ordered" memory model. This is essential for highest performance, as it allows the compiler and the machine to re-order memory accesses to maximize instruction throughput and make the most efficient use of available memory bandwidth. At the microarchitecture level, the processor pipeline has significant freedom to reduce, and even eliminate, instruction dependencies and data dependencies in the instruction stream. This is often highly specific to the exact processor being used—hence the advice above to specify the processor as exactly as possible.

So it is important to ensure that appropriate memory areas are used for specific purposes, For example, memory-mapped hardware devices must be placed in Device memory; for efficient execution, executable code must be placed in Normal memory.[22] The correct use of the "volatile" keyword in C is particularly important when accessing devices or shared memory areas.

It is important to ensure that any memory regions in which access order is important (including devices, semaphores, mutexes, and shared message buffers) are carefully and correctly classified.

The issue of "atomicity" needs careful consideration if necessary.

---

22 For a discussion of the different types of memory and their associated properties, see Chapter B2 in the Arm Architecture Reference Manual for A–profile architecture (ARM DDI 0487K). The discussion there is highly technical!

Don’t be caught out by differences such as arithmetic rounding and precision

While processors of different architectures often contain instructions that carry out the same basic function, such as floating-point multiplication, the exact way in which these behave in corner cases may differ. There are, for instance, various different strategies for arithmetic rounding: towards zero; to the nearest integer; or towards infinity. This can lead to different results from arithmetic calculations, especially when intermediate results are involved! For instance, a particular instruction sequence might or might not round intermediate results while performing an algorithm.

Additionally, the precision of the variables and the functionality of the instructions may result in different results.

Read the specification carefully and test your calculations.

In reality, there are almost no major wheels which you need to re-invent, due to the wide range of tried and tested libraries and middleware components already available for Arm, so don’t waste time writing or re-writing any code that is already available from an existing source.

And, like a good software eco-warrior, always Reuse and Re-cycle!

This is where the scope of this text draws to a close and gives way to the extent of your imagination!

17. System–on–chip Components

This section contains a brief overview of some of the system-on-chip components that are included in the Neoverse™ N2 Reference Design. A detailed understanding of how these functions work, and the reason for which they are included, is not required when working with application-level workloads. Their initialization, configuration and management are handled entirely by software executing either on the SCP or within the software stack on the AP—but this may be of interest to some readers.

17.1. Generic Interrupt Controller (GIC–700)

The interrupt logic is based on the Arm® Generic Interrupt Controller (GIC–700) and supports the management and distribution of interrupts. It handles Shared Peripheral Interrupts (SPIs), Locality-Specific Peripheral Interrupts (LPIs), Private Peripheral Interrupts (PPIs), Message-Signalled Interrupts (MSIs), and Software-Generated Interrupts (SGIs).

The architecture is distributed, providing private and common interrupt resources to all the processors in the system. The interface is fully virtualized, allowing interrupts to be delivered to virtualized workloads without the intervention of the hypervisor.

The distributed architecture of the GIC–700 consists of a group of centralized components, together with a set of distributed components that are replicated for each core (or cluster of cores) in the system. The central components consist of a single Distributor (GICD), an Interrupt Translation Service (ITS) to translate message-based interrupts from external sources such as PCIe Root Complexes, and a SPI Collator.

For each core (or cluster of cores), a GIC Cluster Interface (GCI)—sometimes called a Redistributor—manages PPIs and SGIs. The distributed GCIs can be powered down, along with their associated cores, separately from the GICD. They also connect to the System Control Processor to support wake-up signaling when powering up individual cores.

17.2. System Memory Management Unit (MMU–700)

In a virtualized system, each core in the system may have its own virtual address space, and each application running on each core may, in turn, have its own virtual address space. All these separate virtual address spaces need to be mapped to a single physical address space, which is used to access shared and external components. Such a system involves frequent context switches between the different addresses when switching between processors and between workloads on each processor. These switches involve considerable configuration overhead. Without some mitigation, these would absorb a significant proportion of system performance. A System Memory Management Unit is designed to overcome this specific problem.

The System Memory Management Unit (MMU–700) allows fully virtualized access to peripherals. The unit contains distributed address translation logic, which allows peripherals to be accessed using virtual addresses simultaneously in different contexts. This removes the need to reconfigure the system address translation regime when switching between virtualized workloads, and allows multiple cores to access shared peripherals simultaneously without the need to reprogram address translations for each context.

The MMU–700 consists of two main components: a Translation Control Unit (TCU); and one or more Translation Buffer Units (TBUs).

The TCU controls and manages the process of address translation. The TCU is responsible for accessing (walking) translation tables in system memory to retrieve translation information when required. The TCU contains internal caches for recently used translation information in order to reduce the number of page table accesses required.

Each TBU carries out the address translations. When a TBU receives a request for an address translation, if possible it performs the translation using configuration cached in its internal Translation Look–Aside Buffers (TLBs). If it does not find a matching cached translation, it requests the TCU to retrieve one. Each TBU may be connected to one or more external bus masters. To improve performance in individual cases, more than one TBU may be connected to a single master.

17.3. Network Interconnect (NI–700)

The NI–700 Network Interconnect is a highly-configurable, high frequency, low latency on-chip system-level interconnect, capable of connecting across power and clock domain crossings. In the Neoverse N2 Reference Design, the NI–700 connects to the external I/O masters and provides expansion interfaces for CMN–700.

17.4. Coherent Mesh Network (CMN–700)

CMN–700 is a mesh-based coherent interconnect with multi-chip support using the CCIX standard. The mesh architecture supports high bandwidth data transfer between system components. It supports data coherency across the system, allowing greater and more efficient utilization of private and shared caches.

CMN–700 is specifically designed and proportioned to meet the needs of compute platforms used in high-end networking and enterprise compute applications.

The basic structure deployed by CMN–700 is of a rectangular network of Crosspoints (XPs). XPs are connected in two dimensions to other XPs in a mesh structure. The number of XPs and the vertical and horizontal dimensions of the mesh are configurable. In the mesh, each XP is connected to up to six other devices: XPs in the inner part of the mesh connect to four neighboring XPs and up to two devices; XPs on the edge of the mesh connect to three neighboring XPs and up to three devices; XPs at the corner of the mesh connect to two neighboring XPs and up to four devices.

Certain nodes within the mesh may incorporate parts of the system-wide cache. This is a “last-level” cache, and it is coherent across the mesh.

Other nodes may connect variously to I/O coherent AMBA devices, ACE5–Lite devices, PCIe subordinates, or Coherent Multichip Link (CML) interfaces.

CMN–700 can be configured with multiple asynchronous clock domains.

17.5. Network Interconnect (NIC–450)

The NIC–450 Network Interconnect is a library of components that can be used to build a scalable and configurable network-on-chip interconnect. At its heart is the NIC–400 interconnect, consisting of switches, bridges and interface blocks that can be used to connect bus masters and slaves in a cascading, switched network. It supports a variety of AMBA protocols.

In the Neoverse N2 Reference Design, the NIC–450 connects to system peripherals, other on-chip components, and other subsystem components. Unlike the CMN–700, NIC–450 is not a coherent interconnect.

17.6. Dynamic Shared Unit (DSU)

The Dynamic Shared Unit (DSU) provides connection from a single core, or cluster of cores, into the rest of the system. Highly configurable, it provides an optional shared L3 system cache, snoop control and coherency logic, power control, together with a suite of coherent and non-coherent bus master interfaces, and debug logic.

Although the DSU is capable of hosting a number of cores in a coherent cluster, in the Neoverse N2 Reference Design it is configured in “Direct Connect” mode. In this mode, it hosts only a single core, and the optional L3 cache (together with the associated snoop control and coherency logic) is not implemented. Instead, the core is connected directly into the on-chip interconnect, via a bridge, and to the shared, coherent L3 cache within the CMN–700. This configuration not only reduces on-chip area but also reduces latency for transactions between the core and the interconnect.

The DSU also supports the Realm Management Extension (RME) security architecture.

Part 3 – Arm Neoverse Software and System Design

18. Introduction

18.1. Brief Summary of Parts One and Two

The first two parts of this book provide an overview of the origins, history and characteristics of Arm's Neoverse processor cores. While a detailed understanding of history may not be a requirement when looking to deploy your own application, such an overview can provide valuable insight into the reasoning behind some of the design decisions that have gone into making Neoverse such a success in its target market.

In particular, a focus on factors such as power efficiency, adaptability and customizability, and cost-efficiency have resulted in a range of processing solutions that are uniquely suited to the huge diversity of application to be found in the cloud today.

Earlier material also covered the design objectives behind the security solutions adopted in Neoverse. These are intended to address problems which are in many ways unique to the cloud environment, such as shared ownership and usage models, the need to constantly reload and reconfigure the software environment, and the significant diversity of operating systems and hypervisors that are in use.

Crucially, the "clean sheet" that Arm provided with the Armv8-A architecture (first implemented in Cortex-A and Cortex-X processors) has allowed Arm's partners to develop Neoverse systems that are not burdened by the need to support decades of legacy hardware and software.[23] This unlocks benefits in efficiency and total cost of ownership (TCO) for users.

23 Armv8-A provides backwards compatibility for legacy software via the AArch32 mode. This mode is optional in Armv8-A cores, allowing backwards compatibility either to be provided transparently (by implementing AArch32) or eliminated completely (simply by not implementing it). Since the A64 instruction set supported by AArch64 is a ground-up design, backwards compatibility is neither necessary not provided in this mode.

18.2. The Benefits of Deploying on Arm Platforms in the Cloud

The benefits identified of deploying on Arm platforms in the cloud can be summarized as follows:

- An extensive and growing range of supported software components, middleware, operating environments and development tools.
- Easy access to a wide variety of cloud instances, providing a diversity of performance and price point.
- An architectural focus on power efficiency, which is more sustainable and offers lower TCO.
- A diversity of design and manufacturing options offers licensees the ability to tune for specific use cases, leading to greater scalability that can grow with your needs.

18.3. Common Arm Use Cases in the Cloud

There are extremely few use cases that cannot be supported "out-of-the-box" on modern Arm processing platforms. However, there are several common classes of application for which Arm is particularly suitable.

Modern Arm processors are designed to be easily deployed with multiple cores on a single processor. The memory architectures that they use are optimized for multicore access to large amounts of shared on-device memory, and the caches are designed for efficient sharing of frequently used data between cores.

The latest Armv8 and Armv9 architectures incorporate features that are designed to facilitate rapid context switches between multiple applications on the same core, making these devices particularly suitable for microservices, containerized and multi-tenant environments, where rapid switching between multiple contexts is crucial.[24]

24 Features aimed at fast context switching include efficient interrupt entry and exit sequences which automatically save and restore context, and use of process and context identification information in the memory system to obviate clearing caches and memory management units on a context switch.

The on-chip power and bus architectures used by Arm platforms allows individual cores to be powered up and down quickly and efficiently, making for highly power-efficient implementations which can rapidly scale in response to changing processing demands.

These factors make Arm processing platforms particularly suitable for high-traffic websites, and applications which need to support multiple simultaneous connections.

The inherent power efficiency of Arm platforms makes them suitable for deployment at the network edge, nearer to the IoT devices which are often connected to and supported by cloud applications. These IoT devices are commonly built on Arm architecture processing platforms, and the commonality between an Arm-based cloud application and Arm-based IoT devices allows for the possibility of testing entire Edge-to-Cloud systems on a common platform in the cloud.[25]

18.4. Objectives

The objectives of this document are to give a high-level overview of the steps and processes involved in designing, developing, and deploying an application in a cloud environment on an Arm Neoverse platform. On completion, the reader should be aware of what is involved, where to find software components, how to select a particular platform for deployment, where to find appropriate development tools, and how to test and deploy the final application.

This will be helpful for those seeking to migrate existing applications as well as those looking to develop from scratch. It will also be of use for both teachers and students in an academic setting.

While this document cannot claim to be comprehensive, it seeks to provide sufficient information for further research to inform the design, development, and deployment process.

25 See http://www.corellium.com for one example of such virtualization technology for Arm devices.

18.5. Structure of This Document

Early parts of this document provide guidance on how to characterize the proposed application. This covers subjects such as performance requirements, memory and storage evaluation (size and speed), licensing and billing, testing, and deployment.

These factors inform the choice of platform from the many which are available, also the choice of components and tools that will be incorporated into the process.

This is followed by an examination of three example applications. These are chosen partly because they are common use cases, but also because they provide the opportunity to illustrate typical decision processes which are extensible to other application domains.

The three example applications are:

- A simple web-hosting use case;
- A video transcoding application;
- A framework for hosting and serving AI models.

In this section, extensive reference is made to practical resources published on Arm's website. These contain detailed instructions for building and executing the example systems. A link to the key resources concerned is included at the head of each section.

18.6. Pre-Requisites

It is possible (and useful) to read this document without access to any of the components or tools used in the examples. However, much greater value will be gained by the reader who has access to a suitable environment in which these can be tried out as practical exercises. Pointers are provided to full instructions for readers who are able to do this.

18.7. Further Reading and Learning Resources

Arm and its ecosystem partners have generated much high-quality documentation and training material. This list highlights some of the resources that are available from Arm

itself.

Learning Resources

- https://learn.arm.com/learning-paths/servers-and-cloud-computing/

  Learning resources on servers and cloud computing. These cover tools, application areas, containers, databases, ML, etc.

- https://www.arm.com/markets/computing-infrastructure/arm-cloud-migration

  The Arm Cloud Migration Program is a central resource for migrating any application to an Arm platform. This includes resources for "doing-it-yourself" as well as pointers to third-party migration services and the option to connect with Arm experts who can provide help and advice.

- https://developer.arm.com/servers-and-cloud-computing/arm-cloud-migration

  Tools, tutorial and use cases to help with migrating workloads to Arm platforms.

Libraries and Components

- https://developer.arm.com/ecosystem-dashboard/

  The Arm Software Ecosystem Dashboard is a comprehensive and constantly updated list of over 900 software components available for Arm platforms. The list can be sorted and filtered for specific application areas, licensing terms etc.

- https://www.arm.com/markets/artificial-intelligence/software/kleidi

  The Arm Kleidi Libraries are a set of libraries optimized by Arm for optimal performance across a wide range of Machine Learning, Artificial Intelligence, and Computer Vision application domains. The libraries leverage Neoverse, NEON, SVE, and SME features when available on the target platform. They have already been integrated into many mainstream products.

- https://developer.arm.com/ai

  Resources, models, frameworks, tools, and libraries to help with optimizing AI workloads across all Arm-based platforms, including Neoverse in the cloud.

- https://developer.arm.com/servers-and-cloud-computing/ai-cloud

Specific leaning resources to help cloud application developers at all levels and in all application domains.

– https://developer.arm.com/servers-and-cloud-computing/arm-simd

  Learning resources to help with making best use of the SIMD capabilities of Arm platforms, including NEON, SME, SVE, and SVE2. Also, specific information on migrating code which is implemented for x86 platforms to A64.

19. Writing or Migrating?

KEY RESOURCE: ARM MIGRATION OVERVIEW (https://learn.arm.com/migration)

An overview of how to migrate application to Arm Neoverse with links to additional resources to support software developers.

19.1. Migration Overview

While the end result of your project will be a working system executing on an Arm platform, the place from which you start has an effect on the process you will use to get there.

When developing an application "from scratch" a number of key decisions will need to be made. Some of these are "functional", e.g., what does the system do, and how does it behave? Others are better referred to as system properties or constraints, for example, how it must scale, how many transactions it must be capable of handling in unit time, etc. Both of these decisions will affect what components are most appropriate. This might include choosing the operating system, selecting a database engine, and so on.

When migrating an existing application to an Arm platform, many of these decisions will already have been made as part of the original development. For instance, if a database engine is needed, one will have been selected based on the requirements of the application. It will then have been integrated into the software stack. Since the majority of popular database engines are supported on Arm platforms, it is likely that this selection need not be revisited and the existing database engine may be retained in the migrated application. This will simplify development, integration, and testing as much prior work can be reused. The key step here is to determine whether the existing database engine is supported on the selected Arm platform.[26]

26 See the Software Ecosystem Dashboard for Arm at https://developer.arm.com/ecosystem-dashboard/

Similarly, key decisions about operating system choice, task decomposition, storage requirements, etc., may be retained during the migration. Depending on how much of an existing system can be retained, much of the early parts of the design process can be skipped.

However, a change of platform will necessarily result in a change of performance characteristics. The change in the underlying processor, memory architecture, and connectivity which will inevitably be involved will all have an effect, large or small, on the performance of the system. Some time, therefore, will need to be allocated to benchmarking the working application, then to optimizing some components as necessary.

For further information on how to successfully migrate an application to an Arm platform, refer to the Arm Migration Overview mentioned above. This document outlines two common approaches to migration:

- "top-down", in which the entire stack is built on the new platform and build errors are successively fixed until the stack build is complete and runs correctly.[27]
- "bottom-up", in which the stack layers are built one-by-one from the bottom to the top, fixing errors at each stage before continuing to the next one.

These two approaches are illustrated in Figure 11 and Figure 12.

27 This should not be confused with the similarly named "Arm Topdown Methodology" which is a telemetry-based methodology for analyzing and optimizing performance on Arm devices. See https://developer.arm.com/documentation/109542/02 for more information.

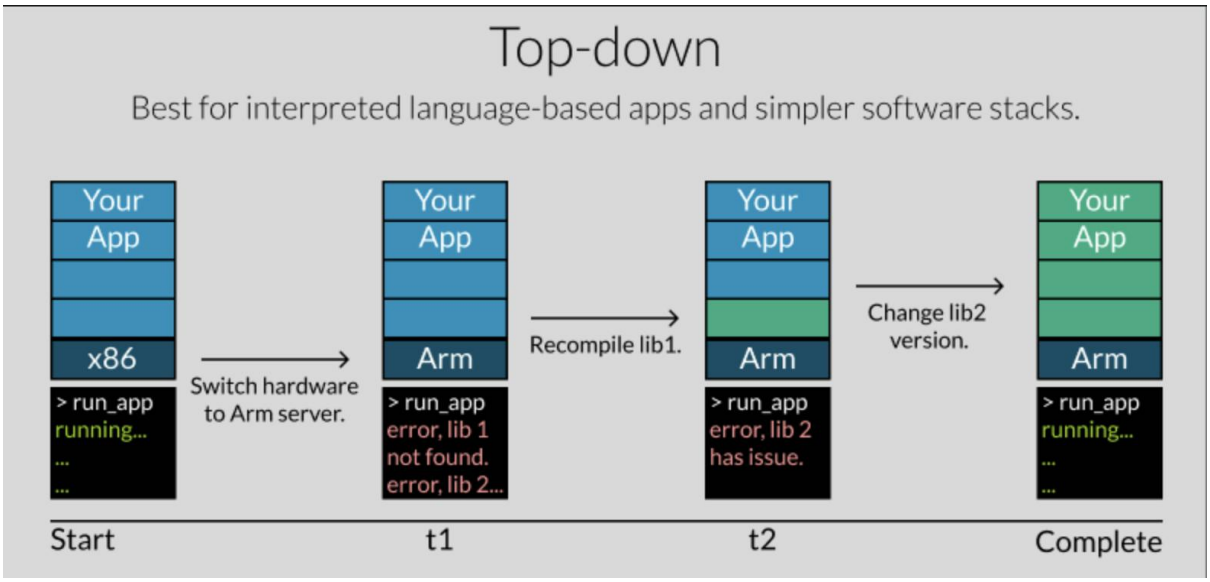

Figure 11 - A top-down migration approach (How can I migrate applications to Arm Neoverse? | Arm Learning Paths)

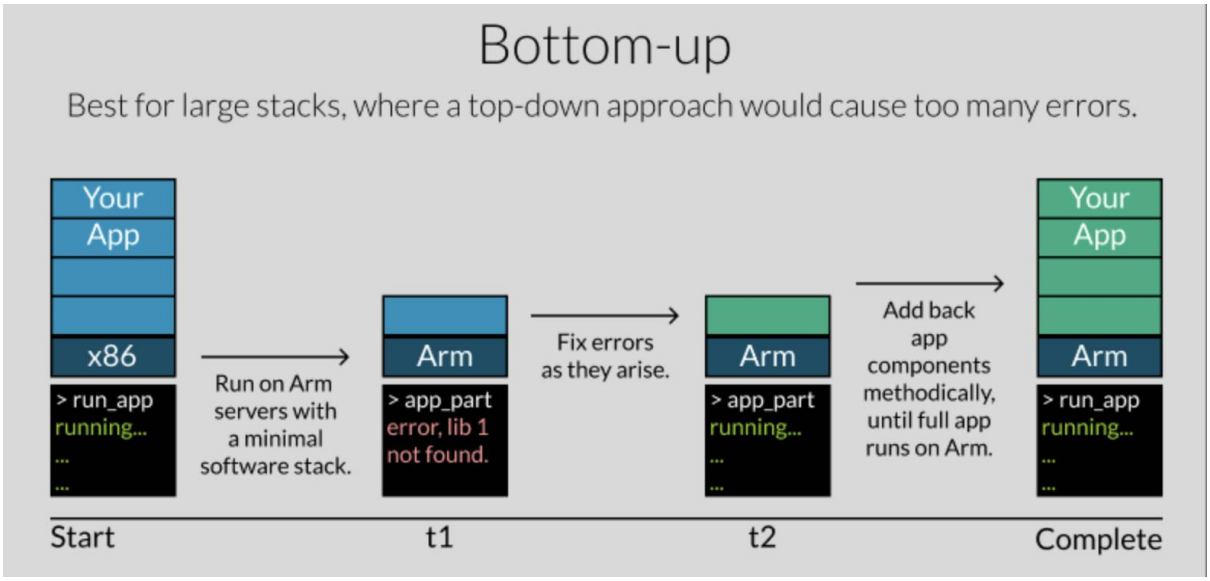

Figure 12 - A bottom-up migration approach (How can I migrate applications to Arm Neoverse? | Arm Learning Paths)

In general, the larger and more complex the stack, the more likely that the bottom-up approach will be more suitable. For such a stack, attempting to build all in one go in a top-down approach might result in so many errors that the way forward is not obvious.

It may be the case that your system is implemented in a distributed manner, i.e. different layers/functions are built separately and deployed to different platforms which communicate and cooperate with each other. In this case, it is much easier to rebuild the system in steps by selecting separate stand-alone components which can be rebuilt and then deployed to the new platform while leaving other components unchanged. Over time, the entire application can be re-compiled and re-deployed piece by piece. In this case, it often makes sense to start with the simplest components in order to pipe-clean the new build process.

Don't forget that, regardless of how much or how little of the system has changed during the migration, regression testing will always be necessary!

19.2. Introducing the Arm MCP Server

KEY RESOURCE: THE ARM MCP SERVER (https://developer.arm.com/servers-and-cloud-computing/arm-mcp-server)

KEY RESOURCE: LEARNING PATH - AUTOMATE X86-TO-ARM APPLICATION MIGRATION USING ARM MCP SERVER (https://learn.arm.com/learning-paths/servers-and-cloud-computing/arm-mcp-server/)

Arm recently announced the Arm Model Context Protocol (MCP) Server. This leverages the open MCP standard to integrate external tools and knowledge sources into your AI

assistant.[28] For more details and a tutorial on installation and usage of the tool in a variety of automated development workflows, see the resources linked at the head of this section.

28 See the press release here: https://developer.arm.com/community/arm-community-blogs/b/ai-blog/posts/introducing-the-arm-mcp-server-simplifying-cloud-migration-with-ai

20. System Design Considerations

When designing an application, the intended function of the system, in other words "what will it do?", is usually the primary focus – this is outside the scope of this document. There are several other properties of the system which must also be considered, and which are within scope here. These relate to properties of the system in addition to its intended function.

For instance, questions such as "how fast must it respond to input?", "how many transactions is it expected to complete in a given time?", "what is an acceptable error rate in responses?". To this list, we can add other properties such as the amount of memory (both program memory and data memory) that is required.

Sometimes, these factors can be derived prior to writing any code (e.g., from statistical analysis of known request rates and response times, or from user-experience testing), or they can be calculated from static inspection of the software (e.g., instruction and cycle counting in key algorithms). In other cases, running code must be benchmarked (with or without instrumentation) to measure the relevant values. If measured or derived figures are out of the acceptable range, then, in what may be an iterative process, some aspects of the design or implementation of the application may have to be revisited.

20.1. Performance Requirements

The term "performance" is most often equated with speed of execution but, in reality, it is much broader than that. While raw execution speed is often an important parameter in a design, there are other properties of a system which need to be considered. In this section, we look at the properties of the system that shape these decisions most:

- Is the system performance dependent on processing power or memory speed?
- Can the system be parallelized to take advantage of the ability to execute multiple software threads simultaneously?
- What is the required throughput of the system, for example, in terms of transactions per unit time?
- What is the required response time of the system to external input? And what constitutes a valid response in terms of the external interface?

Bear in mind that Arm processors are, in general, single-threaded and that data center workloads tend to scale more linearly on such platforms. Since SMT cores experience increased contention for shared resources at higher utilizations, on such platforms it is common to set a CPU usage threshold of c.50% utilization before triggering automatic scaling to more processor cores.[29] On non-SMT Arm Neoverse platforms, it is safer to set the threshold somewhat higher, e.g., 70-80%, thus saving on per-core licensing costs.

Figure 13 below shows some of the factors which may be at play in the determination of suitable performance goals and metrics.

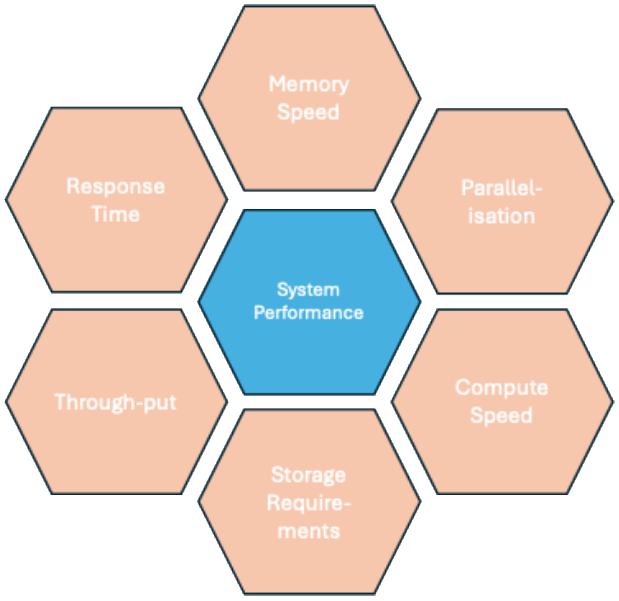

Figure 13 - Some factors which impact performance goals

When you have identified the required performance metrics, and the factors which affect them, create realistic usage scenarios which can be used to validate your assumptions and then benchmark and prove the performance of the final system.

29 For background on this, see: https://developer.arm.com/community/arm-community-blogs/b/servers-and-cloud-computing-blog/posts/reassess-cpu-utilization-on-x86-and-arm

20.1.1. Compute-Bound or Memory-Bound

Since time is money, in many senses, higher execution speed is likely to result in a more efficient use of compute resources which must be paid for (whether in purchase cost or rental fees).

However, the speed of the processor is not the only factor that affects the time it takes to carry out the functions of a system. Other factors are also important, such as, for example: the time it takes to retrieve data from a database, the time taken to receive input and transmit output across a communications link, or the performance of attached storage (which might be on-chip, off-chip, or off-line).

When selecting the most appropriate execution platform for a system, it is important to determine whether the performance of the system depends on the speed of the processor or the speed at which it can access data. The former is referred to as "compute-bound", the latter as "memory-bound". We touch on the related issue of "network-bound", in which the limiting factor is network bandwidth, in §20.20.3 below.

If a system is compute-bound, then a platform with higher execution speed will likely result in improved performance; if memory-bound, high-performance memory and larger memory bandwidth may have a larger effect. In the first instance, consider the nature of the functions that the system is carrying out. If it is simply retrieving and returning data from a relatively simple database, then memory speed is likely to be the limiting factor in performance; if it is carrying out some form of processing or computation on the data, either after retrieval or prior to storage, then (depending on the complexity of the computation) compute speed may be more important. Figure 13 illustrates the relationship between these concepts.[30]

30 "Computational intensity" is a multi-variable metric which aims to quantify the amount of computation effort required to complete a given task, as compared to the amount of effort expended in moving data around. This might include factors such as memory bandwidth and power consumption as well as the raw CPU cycles required. As such, it is more illustrative of the real compute resource requirement than a simple measure of CPU cycles.

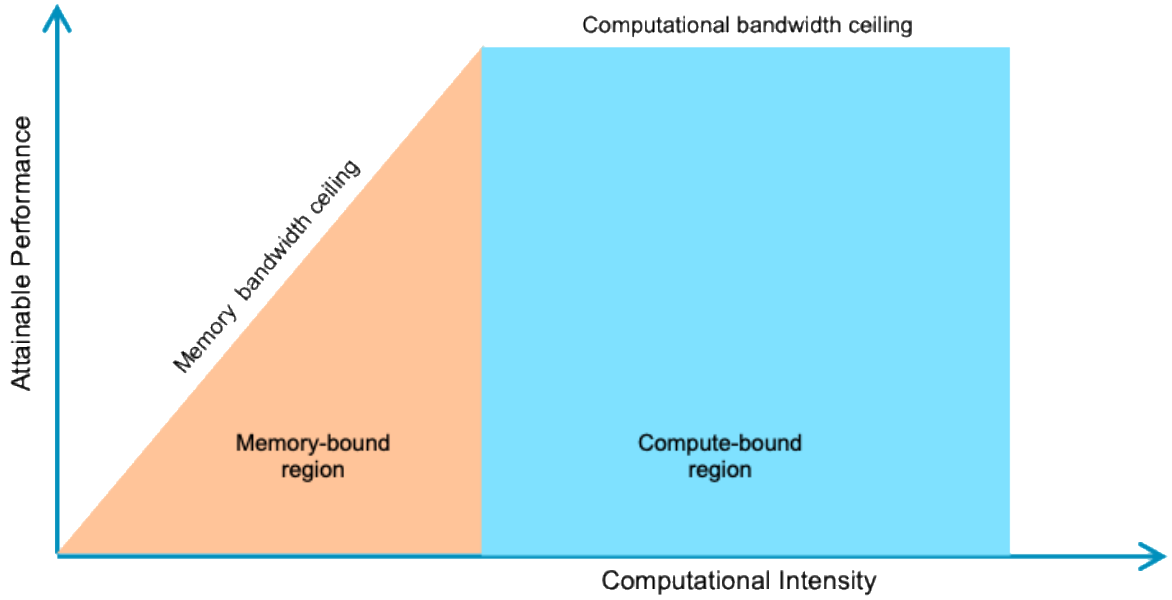

Figure 14 - Compute-bound vs Memory-bound

In practical terms, some form of profiling or modeling is likely the best way to characterize the limiting factor in your system. For instance, a profiler will be able to report how much time the processor spends "idle", waiting for input from internal or external sources.[31] If the processor is rarely idle, then it is likely that the system is compute-bound and increasing the available processing power will improve the performance of the system. Conversely, if the processor is spending considerable amounts of time waiting then it is possible that the system is memory-bound and faster memory interfaces may improve performance (however, note that memory latency is not the only factor affecting processor idle time – waiting for input from internal or external sources is an equally likely explanation).

More detail on profiling is given in §26 below.

31 See §25and §26 below for more detail on profilers and other optimization tools.

20.1.2. Parallelizability

Modern compute platforms are designed to handle multiple tasks simultaneously. The Arm-based Neoverse processors found in many cloud platforms are particularly efficient at doing this. They achieve this by employing one or more of multi-tasking, multi-threading, and multi-processing.

Multi-tasking

Multi-tasking operating systems, such as Linux or Windows, can switch rapidly from one software task to another. This gives the illusion of being able to execute multiple tasks simultaneously even though there is only a single processor. At any one time, the processor is executing instructions from one of the tasks in the system. A task switch occurs in response to a number of possible events. These include:[32]

- the currently executing task reaches the end of its defined function;
- the currently executing task cannot continue because it is waiting for input from another task;
- the system must wait for a response from an external agent;
- the operating system determines that the currently executing task has had a sufficient share of processor resources and initiates a switch to another task;
- an external event results in a trigger to switch to a task which can process that event.

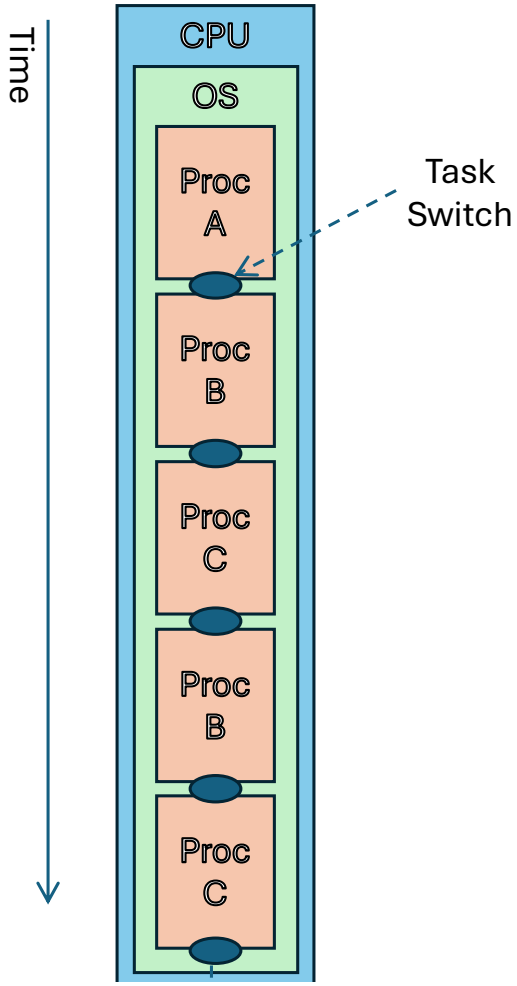

Figure 15 - Multi-tasking

In such a system, there is a single core which can only execute a single instruction sequence at any one time. Rapid switching from one task to another creates the illusion of simultaneous execution of multiple software tasks. The speed of these switches, which are entirely software-controlled, determines the depth of the illusion. Neoverse processors employ several design features that make task switches fast and efficient.

32 These scheduling strategies for task switching are the same across Arm and other architectures.

Figure 15 illustrates the switching between processes in a single-processor, single-threaded multi-tasking system. There is a single instance of the Operating System, executing on a single processor. Only one process is executing at any one time. Task selection and switching is done by the Operating System.

Hardware Multi-threading

A multi-threaded processor has execution hardware which can switch extremely rapidly between two (or very occasionally more) execution threads. This switching happens at a granularity that is too fine for software control, e.g., when a particular instruction is blocked from completing due to the need to wait for a response from the memory system, or to wait for the result of another instruction. In such cases, the execution unit can rapidly switch from the blocked thread to another which can execute immediately. This feature allows the processor to execute more than one thread simultaneously, in effect behaving as two distinct processors running on a single hardware instance. The cost is paid in larger and more expensive execution hardware.

In the main, Neoverse cores are single-threaded (one thread per core), which, in general, yields better per-core performance than multi-threaded competitor architectures. At the time of writing, the Neoverse E1 is the only processor in the Neoverse family to support hardware multi-threading, and is not available as a public cloud instance.

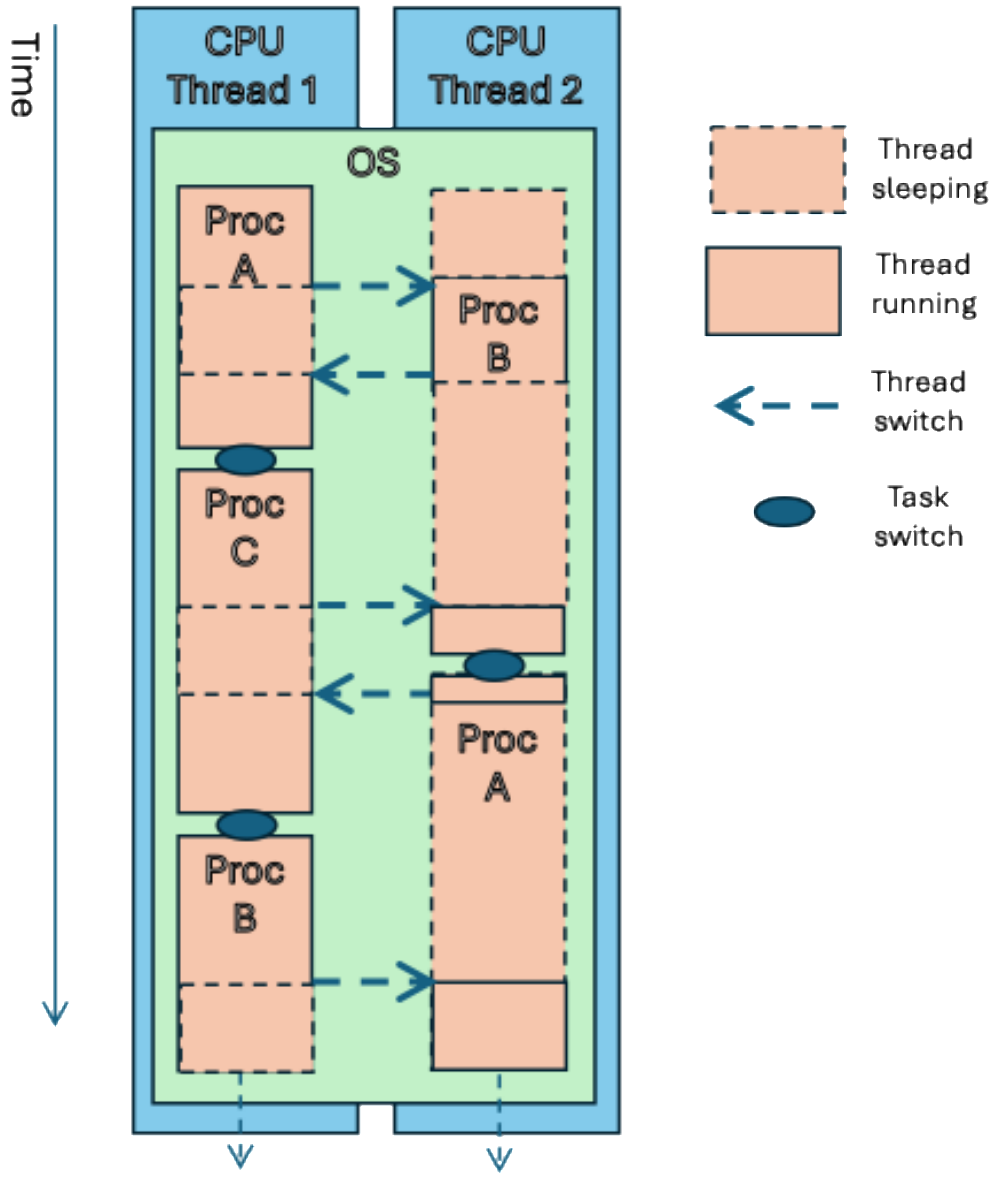

Figure 16 – Hardware Multi-threading

Figure 16 shows a simple example of hardware multi-threading on a single CPU under a multi-tasking operating system. A single CPU runs a single instance of a multi-tasking operating system. At hardware level, the CPU can maintain two threads, each of which can be mapped to a separate process by the operating system. Switches between threads

are managed by the CPU, at a level of granularity below the operating system. Switches between tasks are managed by the operating system.

At any one time, one of the two threads is being executed by the CPU.

Multiprocessing

A multiprocessing system distributes the distinct software tasks comprising a system across multiple physical processors. The performance of such an approach relies heavily on the efficiency with which these separate processors can share data. Generally, a single instance of the operating system will execute across multiple processors, allowing them to dynamically share software tasks and data.

All current cloud processing platforms are built from chips containing more than one core. The number of cores per chip varies, considerably but each core is identical. The on-chip interconnect and memory architecture used by Neoverse implementations make it possible to share code and data between processor cores on the same device very easily and efficiently. There is clearly some increased latency involved in accessing data, which is in shared memory, but intelligent caching strategies improve this considerably.

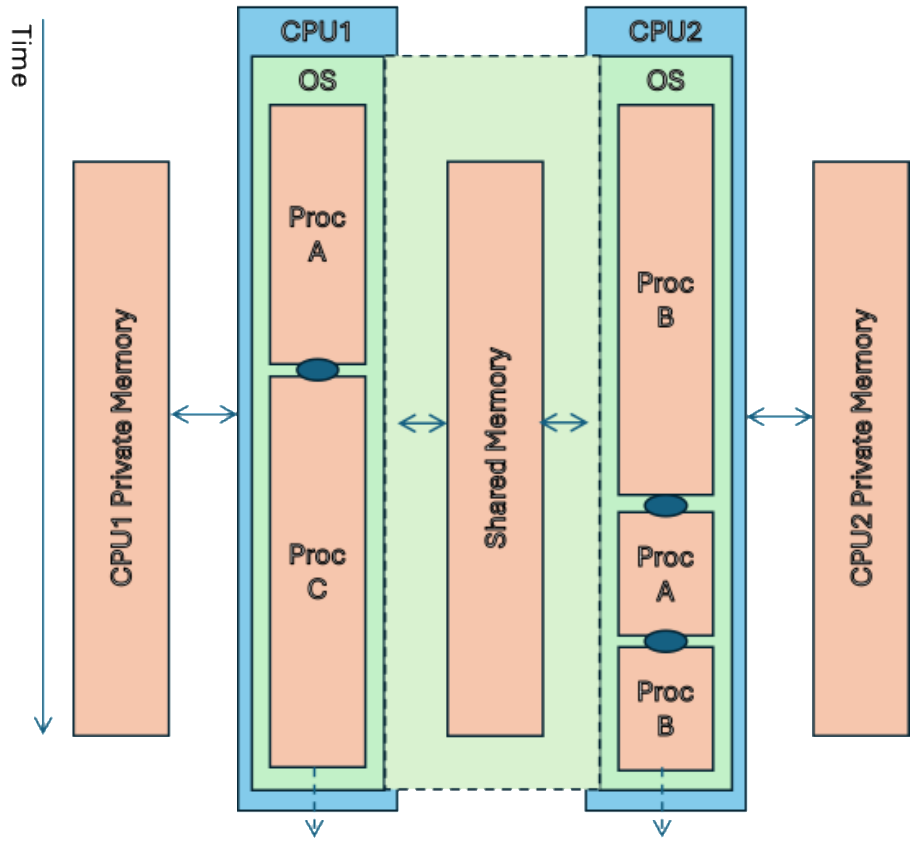

Figure 17 - Symmetric Multi-processing

Figure 17 shows an example of symmetric multi-processing. What is, in effect, a single instance of the operating system executes across a number of processors simultaneously. Although executing independently, they behave as a single instance because they share the operating system data structures. Tasks, from a pool of executable tasks, are allocated dynamically to each processor.

All of the above means that ensuring that a system is capable of decomposition into a set of distinct tasks, which can, to some extent, execute in parallel, and can have a dramatic effect on performance.

It is important to recognize that some parts of a software system may be inherently non-parallelizable, while others may lend themselves naturally to it. Cryptographic hashing algorithms, for example, are often designed with the intention of making them very difficult to parallelize.

There has been much material written on this topic throughout the history of the computing industry and many of the problems are well understood. In particular, pay attention to Amdahl's Law (originally formulated in 1967), which analyzes the cumulative effect of adding more compute resource to a system.[33]

20.1.3. Throughput

Throughput refers to the volume of work the system can carry out in a specific time period. This might be measured in various ways, e.g., transactions-per-second, requests-per-minute, pixels-per-second, and so on.

To establish the required throughput of the application, it is necessary to construct typical usage scenarios. This might be done by estimating the total user population, how many requests each user might make in a given time period, the response time for each request, the total compute resource required for each transaction, etc. From these, a theoretical target for throughput can be derived.

If an existing, operational system is available, then real usage data can be recorded and replayed to give an accurate idea of the performance of the new system.

Remember that the maximum throughput of a system is not dependent only on the available processing power.

See §25 and §26 below for more information on how these characteristics might be measured. In addition, there is an Arm Learning Path which provides a worked example of benchmarking and tuning network performance.[34]

---

33 You can find Amdahl's original paper here: https://dl.acm.org/doi/10.1145/1465482.1465560

34 Microbenchmark and tune network performance with iPerf3 and Linux traffic control: https://learn.arm.com/learning-paths/servers-and-cloud-computing/microbenchmark-network-iperf3/

20.1.4. Responsiveness

The responsiveness of the system relates to how quickly the system responds to external input triggers. This will be made up of several components, each of which will have different acceptable bounds and will be affected by different aspects of the system.

Consider the process of purchasing an item from an online retailer. Greatly simplified, this involves the following steps:

01 Locate and view the home page of the retailer.

02 Search for the desired item and put it in your shopping basket.

03 Checkout and pay.

04 Receive the delivered item.

The responsiveness of the system appears to the user in four distinct stages:

1. **The time to locate and view the home page**
   The user expects this response quickly as confirmation that the website is available and working, and that it is now waiting for the user to initiate a purchase. This response time is entirely dependent on the web application itself.[35]

2. **The time to find the desired product and put it in the basket**
   The user will wait longer for this response as the website is clearly functioning and will eventually return a response. The response time is dependent entirely on the web application, particularly on the time taken to search product databases.

3. **The time to complete payment**
   The user will wait even longer for this as they have now committed to the purchase and often have low expectations for a speedy response at this stage. The response time is entirely dependent on the payment services provider and may involve, for example, two-factor authentication. No amount of optimization in the web application is likely to improve response time here. The only way to improve it would be to purchase a higher level of service guarantee from the payment service provider.

35 Ignoring any contribution due to transmission delays.

4. **The time to receive the delivered item**
   Clearly, this is entirely out of the hands of the web developer and dependent completely on the performance of third-party shipping companies.

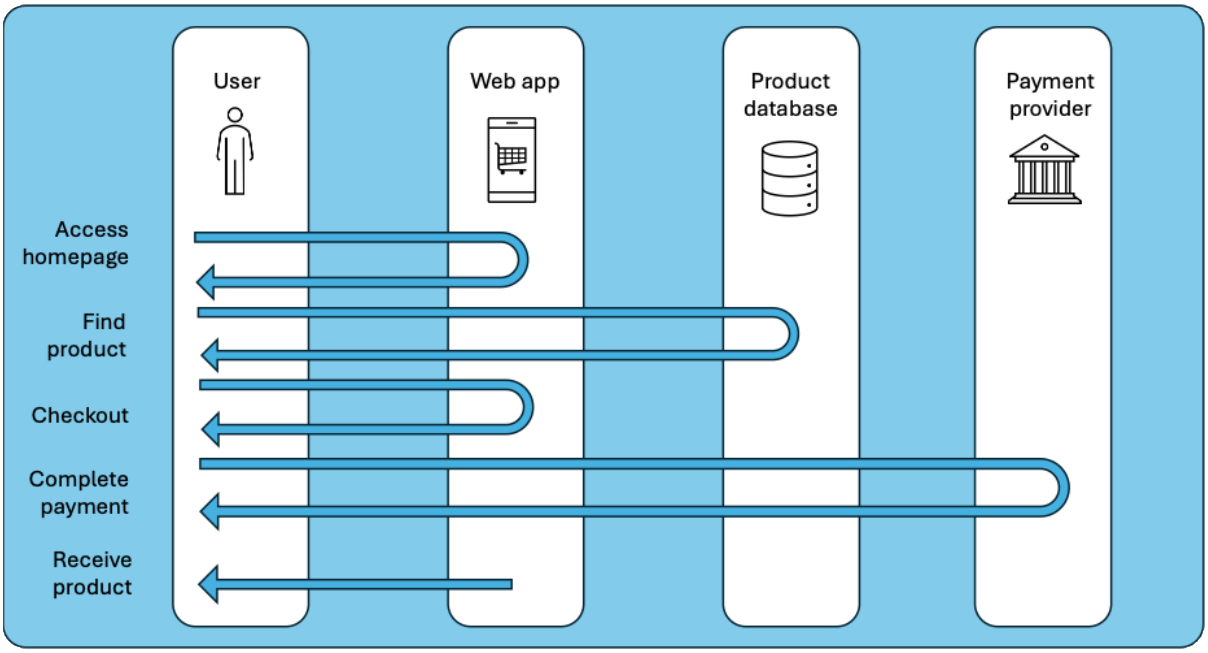

Figure 18 - Making a purchase

Figure 18 illustrates this process.

In a similar way, the responsiveness of your proposed software system should be broken down into those components that are dependent on the performance of your system alone, those that are partially dependent on it, and those that are not dependent on it at all. It is important to recognize that users will have different expectations of what is perceived as acceptable for each of these and that your system only has control over some of them—some matter more than others.

In the hypothetical purchasing scenario, the system designer should focus most optimization effort on the initial response time of the home page (since this is the key

response to the user and is the most dependent on the local system), and next, focus on the time to locate the required product (since this is less important to the user and local optimization has limited effect). The payment process depends significantly on a third-party payment processing system. The response time here is not under the developer's complete control, however, some local optimization might be productive. The shipping process is determined entirely by third parties, so cannot be optimized at all.

20.2. Storage Requirements

The principal factors in determining storage requirements are:

- the amount and type of data needed;
- the speed at which the data can be accessed;
- the patterns in which the data is accessed during execution;
- whether access patterns are dominated by reading or writing data.

Consideration must also be given to whether a database engine is required and, if so, which database architecture is the most appropriate. This decision will have far-reaching implications for the performance and functionality of the final application.

20.2.1. Speed

You will want to consider some or all of the following factors:

- the time taken to access a data item in response to a transaction;
- the necessary responsiveness of the system (see §20.1.4);
- the transaction rate (see §20.1.3),

Some of this estimation can be done as part of the system design, at which time estimates will be available for information such as the transaction rate and responsiveness. It is likely, though, that some measurement will be necessary using the executing application itself. For example, a profiler can be used to count memory transactions and estimate how long each one takes.

Data can also be divided into that which must be accessed quickly (for instance, data on which the initial user response depends), and that which can be accessed more slowly.

The former may need to be maintained in an in-memory database, while the latter could be allocated to offline disk storage. Available processor platforms will generally have a fixed amount of fast on-chip storage.

Consider, too, the likely access pattern. For instance, an image held in a frame buffer will be accessed in a predictable but non-linear pattern, while an audio stream will be accessed in a strictly linear fashion. The former will benefit from being held in a large enough area of fast memory and will likely benefit from efficient caching, since many data items will be accessed multiple times; the latter can likely be held in a relatively small linear buffer and caching may well be less relevant since each data item will only be accessed once.

20.2.2. Size

Some or all of the following factors will come into consideration:

- the amount of data required to service each transaction;
- the type and size of each data item;
- the amount of new data generated by each transaction;
- the transaction rate (see §20.1.3);
- the deletion rate of expired data;
- the retention policies which dictate for how long data must be stored;
- whether multiple copies of the data need to be stored for integrity and backup reasons.

These factors (estimated or measured) can be used to derive the base amount of storage, and the rate at which it grows or diminishes.

20.2.3. Choosing a Database Engine

Not all systems need or benefit from a database engine. For instance, a file server or an image transcoder do not need the structured data retrieval capabilities of a database. However, it is likely that all but the simplest systems will rely on some kind of structured data which is used in generating responses to user requests. Usually, there will be a need to be able to write as well as read the data. User transactions, as well as accessing existing data, will typically also generate new data which will need storing in the database.

The type of database engine selected will be driven by the type, structure (or lack of structure) and volume of the data being stored, as well as how it is to be queried, accessed and updated. Highly structured data will likely fit an SQL database engine better, while data which is essentially a collection of random, unrelated items may suit a different engine altogether.

When selecting a database engine, there may be external regulatory requirements that drive the selection. For instance, highly-regulated fields such as finance, medical, etc., may mandate some degree of ACID compliance:

- **Atomicity**
  Individual transactions can be treated as indivisible, i.e., either all elements of the transaction are completed or none of them are. This is important for applications such as accounting and banking.

- **Consistency**
  This ensures that the database is always in a consistent and stable state. This has bearing, for instance, on the ability of the database to survive system failure while retaining its integrity. Also, that multiple agents accessing the database simultaneously will receive consistent results.

- **Isolation**
  This refers to the requirement that individual transactions do not interfere with or affect other transactions that may be occurring at the same time. This may be crucial for any database where multiple agents are accessing it at the same time in response to concurrent user requests which are processed separately in a multi-processing system.

- **Durability**
  This describes the ability of the database to survive various kinds of system failure e.g., power loss, or software malfunction. On a restart, the database must be in a stable and consistent state, ready to continue operation. Audit trails must make it possible to determine which transactions were completed and which were not before the interruption.

Database engines typically fall into two categories: SQL or NoSQL.[36]

An SQL database engine implements a relational database in which data is stored in structured tables consisting of rows and columns of data items. The layout of the data, the way in which items relate to each other, and the way in which the database can be searched (referred to as the "schema") are generally fixed at design time. SQL databases are generally ACID compliant. They are accessed using the standard Structured Query Language (SQL). Examples of SQL databases include MySQL, PostgreSQL, and SQL Server.

A NoSQL database refers to one in which the data is stored in a less structured and usually flexible, dynamic schema, for example as a graph or tree. The type and format of data that can be stored is often more varied than in an SQL database, and the schema can be changed dynamically in response to changing storage requirements. For example, different nodes in a graph structure may hold completely different types of data to neighboring nodes. ACID compliance is not universal and, if available, is sometimes harder to prove when trying to meet regulatory requirements. Examples include MongoDB, Redis, Cassandra, and DynamoDB.

A brief summary of the differences between SQL and NoSQL is given in the table:

Feature	SQL	NoSQL
Data structure	Tables (rows/columns)	Key-value
Schema	Fixed	Dynamic
Query language	SQL (standard)	Varies
ACID compliant	Generally, yes	Varies
Scalability	Within existing scheme	Flexible and scalable

36 SQL refers to "Structured Query Language", a standard language used for interrogating, accessing and modifying relational databases and is used by all "SQL" databases; NoSQL indicates a database which is not relational and does not support SQL for access.

SQL may be preferred if:

- the data exhibits a high degree of structure;
- complex joins and queries are to be used when accessing data;
- data integrity and consistency, e.g., ACID compliance, are important;
- the application domain is financial, medical, or legal.

NoSQL may be preferred if:

- the schema, layout and type of data stored is likely to change dynamically;
- the data is less structured and individual data items are not necessarily or obviously connected to other items;
- the application domain is real-time analytics, sensor data, or social feeds.

Extensive guidance on database selection and configuration exists. All of the major cloud service providers offer advice on database selection and many have in-house or preferred database engines which are optimized for their platforms.

The majority of popular database engines are available for Arm platforms.[37]

20.3. Networking/Communications

Like compute power, network resource is a chargeable commodity when working in the cloud. System designers need to pay careful attention to how much networking capacity is needed to avoid either under-provisioning or paying for unused bandwidth. In addition to cost considerations, it is important to ensure that networking bandwidth is not the limiting factor in application performance ("network-bound"). No amount of performance optimization on the source code will help improve the performance of a network-bound application, whose performance is dependent on the amount of network bandwidth available to it.

37 See the Software Ecosystem Dashboard for Arm at https://developer.arm.com/ecosystem-dashboard/

Factors which affect the amount and type of networking capability include:

- **Bandwidth**
  This can be measured using real or estimated values for the transaction rate and amount of data to be transferred per transaction.

- **Latency**
  The responsiveness of the application can depend on the latency of the network connections used. Consider also whether network transmissions are particularly time-critical (e.g., time-series sensor data, video streams), or not (e.g., financial transactions, chat rooms). Time-sensitive data will require networking capability with known and low latency.

- **Security**
  The type and sensitivity of the data being transferred over a network in or out of the system will dictate the level of security that is appropriate. High-security algorithms often consume a great deal of processing power and time (therefore increasing latency and reducing responsiveness), so be careful not to over-provision. Bear in mind any regulatory requirements which may be relevant to the application domain (e.g., financial transactions, medical data etc.). It may be necessary to deploy a VPN solution such as Strongswan, or an encryption solution such as Zerotier.[38]

- **Identity Management and Authentication**
  Some application domains may require careful attention to the identity and permissions of those accessing the system. Consider whether a bespoke, private identity management and authentication capability is required, or whether the system can leverage an existing, external solution.

- **Resilience**
  Mission-critical applications may need to consider implementing some kind of redundancy or alternative provision to increase resilience in the event of system failure or external attack. External solutions, such as Cloudflare, may be considered.[39]

---

38 https://docs.strongswan.org and https://github.com/zerotier/ZeroTierOne#getting-started

39 Build and Install on Arm Servers: https://learn.arm.com/learning-paths/servers-and-cloud-computing/zlib/setup/

Official documentation: https://github.com/cloudflare/cloudflared/tree/2024.6.1?tab=readme-ov-file#installing-cloudflared

20.4. Other Considerations

There are many other considerations which will influence the design of an application. These are generally beyond the scope of this document but may include some or all of the following examples.

Quality of Service

As well as parameters such as performance, latency and throughput, Quality of Service (QoS) covers such considerations as reliability, resilience, availability, and redundancy. When selecting a deployment platform, these may influence your choice of provider, location, connectivity, etc. All providers will publish their guarantees for availability, amongst others, and their policies for resilience in the event of failure.

Deployment

When considering a provider and a platform, financial and commercial factors may come into play. For instance, the geographical region in which you need your application to be available to users may influence the region in which you seek out a suitable provider. Not all providers are able to provide service in all locations, and you may elect to deploy to multiple providers to cater for your market.

Commercial/Financial/Legal

As well as the Non-Recurring Expense (NRE) incurred in developing your application, there will be ongoing costs for hosting, maintaining, and servicing it post-deployment. This will influence the selection of provider, platform, and billing model. In addition, some software components, unlike those used in the examples in this document, are not free to use and incur license and/or usage fees.

21. Options for Building and Deployment

KEY RESOURCE: SOFTWARE ECOSYSTEM DASHBOARD (https://developer.arm.com/ecosystem-dashboard/)

There are several options for putting together the desired software stack and deploying the result to a Neoverse platform.

The easiest option is simply to install pre-built images. For example, on a system running Ubuntu, this can be done by using the Advanced Package Tool command, e.g., "sudo apt" to install the package direct from the repository.

The majority of packages supported by Arm are available via this route, which allows stacks to be assembled and configured quickly.

However, in order to optimize the software for particular platforms and use cases, it is sometimes necessary to build the components from source. For the majority of open source components, this is a straightforward process and documentation will usually be found either on the host website or in the relevant GitHub repository.

Building components from source offers greater flexibility but is more involved and takes greater effort time and compute resource. Building a large code base from source may take many hours (at least the first time). A sensible approach would be to use pre-built components to test feasibility and pipe-clean the chosen configuration before choosing to build from scratch if further optimization proves to be necessary.

The approach taken in the following sections, which deal with specific example applications, is to use pre-built components where available, and then to provide pointers to instructions for building from source should that be necessary.

Containerized application development is an approach, which is gaining significant traction in the industry. Containers such as “docker” simplify the process of building an application on one architecture and targeting it for execution on another. “Multi-architecture” Docker images contain multiple versions of the same application targeted at different execution platforms, allowing a single application image to be executed on a diverse range of platforms without modification.

For instance, an application might be built on an Arm-64-based platform running Linux. The build process can output a single, containerized image which can be deployed, for example, to a platform based on a different ISA architecture and running Windows, or to an Arm64-based platform running Linux, or to an Arm64-based platform running macOS.

The ability to decouple the need to build and deploy specifically and separately for each architecture greatly simplifies development and deployment pipelines.

22. Example Application One – Web Hosting

KEY RESOURCE: LEARN HOW TO DEPLOY NGINX (https://learn.arm.com/learning-paths/servers-and-cloud-computing/nginx/**)**

This learning path is an introduction for engineers who want to use Nginx on Arm.

KEY RESOURCE: LEARN HOW TO DEPLOY MYSQL (https://learn.arm.com/learning-paths/servers-and-cloud-computing/mysql/**)**

This learning path is an introduction for engineers who want to deploy MySQL on Arm.

In this example, we will examine how to deploy a basic web server, capable of serving a simple SQL database.

Further information and detailed instructions on how to carry out some of these steps can be found in the resources listed above.

22.1. Pre-requisites

You will need at least one Arm-based instance, either from a cloud service provider or an on-premises server.

22.2. High-Level System Design

The number of possible component combinations from which a web server stack might be assembled is huge and the possibilities quickly become too many to count.

"LAMP" is an acronym that refers to a template software stack for a "standard" web server.

L – Linux

A – Apache

M – MySQL

P – Perl, PHP or Python

Such a stack is illustrated in Figure 19.

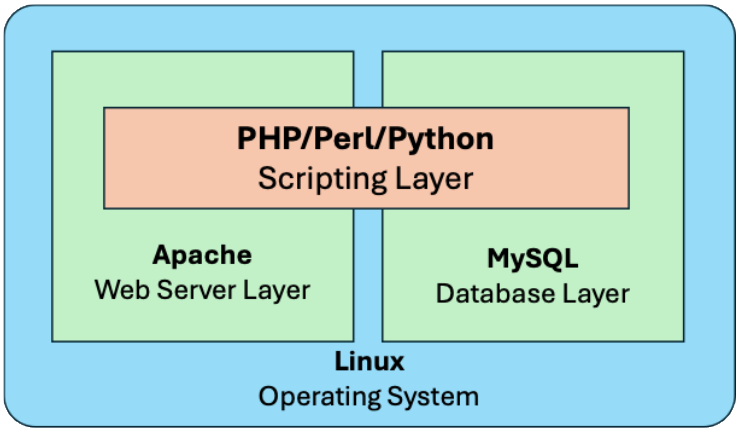

Figure 19 - Example LAMP Stack

Many web server deployments are based on this, or on variants of it in which components are substituted with alternative packages. For instance, "WAMP" refers to a stack in which Linux has been replaced with Windows as the underlying operating system. Similarly, "LEMP" refers to a system in which Apache is replaced with Nginx. This latter configuration is the one used here.

Nginx (pronounced "engine x") is free and open source software that is used by a large and growing proportion of deployed servers (as of 2025, Nginx has the largest share of deployment for any single application in cloud use cases).[40] It is most commonly used as a web server but can also be deployed as a reverse/mail proxy, load balancer, or HTTP

40 March 2025 Web Server Survey, conducted by Netcraft, https://www.netcraft.com/blog/march-2025-web-server-survey

cache. The source code is available on GitHub and supports a number of commonly used operating systems. It is lightweight compared to alternative components, configurable and extensible.

22.3. Platform Selection and Configuration

The Arm example referenced above does not place any specific requirements on the execution platform. The system as built is simple enough not to require a high-performance platform.

For a "real-world" system, the choice of platform is influenced by one or more of:

- Company policy
  Your employer (or customer) may have reasons for favoring or selecting a particular platform provider. For instance, there may be existing licensing arrangements or company policies which encourage or even mandate one provider.
- Software component support
  As we have seen, the majority of components are well supported and maintained on a wide range of Arm platforms. However, if your choice of platform is influenced by other factors (see above), then the choice may be restricted to the components available on that platform. Similarly, the provider may have preferential support for particular components, e.g., Oracle offers Oracle Linux as a standard install on its platforms.[41]
- Platform availability
  As noted earlier, not all platforms are available in all regions. Depending on the geographical deployment of your system, this may dictate some elements of the platform choice.
- Nature of application
  If your system is primarily retrieving and serving data from a database, then a platform which is optimized for memory access may be appropriate. If your system is carrying out significant computation on the data in transit, then a more powerful platform may be a better choice.

41 https://docs.oracle.com/en-us/iaas/oracle-linux/home.htm

22.4. Component Selection[42]

Component	Supported on Arm	Version to use	Source
Nginx	Since 2014	1.20.1 or later	https://nginx.org/en/download.html
MySQL	Since 2021	8.0.38 or later	https://dev.mysql.com/downloads/
PHP	Since 2020	8.0.0 or later	https://www.php.net/downloads.php

Links to Arm and third-party documentation can be found in the Arm Software Ecosystem Dashboard on Arm's website.

The links in the table above are to open source versions of the relevant components. In many cases, commercial versions are also available.

22.5. Build, Install, Deploy

KEY RESOURCE: PHP INSTALLATION: (https://www.php.net/manual/en/install.unix.php)

This is the standard documentation for installing PHP on Unix systems.

KEY RESOURCE: NGINX INSTALLATION: https://nginx.org/en/linux_packages.html

This is the standard documentation for installing Nginx on Unix systems.

KEY RESOURCE: MYSQL INSTALLATION: https://dev.mysql.com/doc/refman/8.4/en/linux-installation.html

This is the standard documentation for installing MySQL on Linux systems.

42 See the relevant entries in the Software Ecosystem Dashboard for Arm at https://developer.arm.com/ecosystem-dashboard/

The key decision here is whether to use pre-built components or to build them from source. The latter approach involves more development work but offers greater flexibility in optimizing the component for your platform and for your application.

The referenced example deployment of Nginx documents both approaches.

A pragmatic approach may be to use pre-built components rapidly to put together a functional system, then to re-build from sources if testing and benchmarking indicates that specific optimization or customization is required.

For widely-used components such as MySQL, many of the cloud providers support their own deployment paths which can greatly simplify the deployment. For instance, Amazon RDS is a MySQL-compatible database engine optimized for AWS cloud platforms.[43] If you are using an AWS platform, selecting Amazon RDS for the database engine may simplify the deployment process considerably (and there may also be price and performance benefits).

22.5.1. Install Nginx

These instructions assume that you are using Ubuntu. Instructions for other variants of Linux, including proprietary versions from cloud service providers, can be found at https://nginx.org/en/linux_packages.html.

Install pre-requisites:

```
sudo apt install curl gnupg2 ca-certificates lsb-release ubuntu-
keyring
```

43 https://aws.amazon.com/rds/

Import an official nginx signing key so apt can verify the packages authenticity. Fetch the key:

```
curl https://nginx.org/keys/nginx_signing.key | gpg --dearmor \
    | sudo tee /usr/share/keyrings/nginx-archive-keyring.gpg
    >/dev/null
```

Verify that the downloaded file contains the proper key:

```
gpg --dry-run --quiet --no-keyring --import --import-options import-
    show /usr/share/keyrings/nginx-archive-keyring.gpg
```

The output should contain the full fingerprint 573BFD6B3D8FBC641079A6ABABF5BD827BD9BF62 as follows:[44]

```
pub   rsa2048 2011-08-19 [SC] [expires: 2027-05-24]
    573BFD6B3D8FBC641079A6ABABF5BD827BD9BF62

uid  nginx signing key <signing-key@nginx.com>
```

Note that the output can contain other keys used to sign the packages.

To set up the apt repository for stable nginx packages, run the following command:

```
echo "deb [signed-by=/usr/share/keyrings/nginx-archive-keyring.gpg]
    \
    http://nginx.org/packages/ubuntu `lsb_release -cs` nginx" \
    | sudo tee /etc/apt/sources.list.d/nginx.list
```

44 If the key has changed since publication of this document, the fingerprint may be different to that shown.

If you would like to use mainline nginx packages, run the following command instead:

```
echo "deb [signed-by=/usr/share/keyrings/nginx-archive-keyring.gpg]
     http://nginx.org/packages/mainline/ubuntu `lsb_release -cs`
     nginx" \
     | sudo tee /etc/apt/sources.list.d/nginx.list
```

Set up repository pinning to prefer the chosen packages over distribution-provided ones:

```
echo -e "Package: *\nPin: origin nginx.org\nPin: release
     o=nginx\nPin-Priority: 900\n" \
     | sudo tee /etc/apt/preferences.d/99nginx
```

To install nginx, run the following commands:

```
sudo apt update

sudo apt install nginx
```

22.5.2. Check the nginx version and build configuration

It can be useful to have details of exactly which version has been installed. The useful Nginx "version" command will give you all this information together with a set of compiler build options which you can use to rebuild from source should that prove necessary in future.

Run the following command:

```
nginx -V
```

This command will produce output similar to the following.[45] The "`--with-cc-opt`" output shows the compiler flags which can be used to rebuild from source. The output also shows which version of OpenSSL has been incorporated into the build.

```
Nginx version: nginx/1.18.0

built with OpeSSL 3.0.2 15 March 202

TLS SNI support enabled

configure arguments: - - with-cc-opt='-g -O2 -ffile-prefix-
     map=/build/nginx-glNPk0/nginx-1.18.0=. -flto=auto
```

Enable and start nginx:

```
sudo systemctl enable nginx

sudo systemctl start nginx
```

Instructions for rebuilding from source can be found here:

https://learn.arm.com/learning-paths/servers-and-cloud-computing/nginx/build_from_source/

The same example on Arm's website also shows how to then configure Nginx as a static file server, reverse proxy, or API gateway.

45 The output shows information about the actual version which has been installed. If what you see differs from what is shown here, this merely reflects that a newer version is installed.

22.5.3. Install MySQL

The following instructions are extracted from the full detailed documentation for MySQL, which can be found at https://dev.mysql.com/doc/refman/8.4/en/

Locate and download the appropriate version from MySQL APT repository at https://dev.mysql.com/downloads/repo/apt/.

Install the downloaded package. You will need to replace the version information with the relevant information for the package if you have downloaded a different version. You may also need to specify a pathname for the package if it is not in the current directory.

```
$> sudo dpkg -i ./mysql-apt-config_0.8.34-1_all.deb
```

The installation process will query which versions and which optional components you would like to install. If you are unsure, accept all of the default options.

Update the package information:

```
$> sudo apt-get update
```

Install MySQL with APT:

```
$> sudo apt-get install mysql-server
```

Additional components can now be installed using the same procedure. Details are in the documentation referenced above.

The above instructions will also install the `mysql` monitor and you can now use the CLI tool `mysql` to create, connect to, and manipulate databases.

22.5.4. Install PHP

Run the following commands to update the package lists and install PHP. The module for connecting PHP and MySQL is installed.

```
sudo apt update

sudo apt install -y php-fpm php-mysql
```

22.5.5. Configure Nginx to use PHP

Using your favorite text editor, edit the default configuration file at `/etc/nginx/sites-available/default` to add the following:

```
server {

    listen 80 default server;

    listen [::]:80 default_server;

    server_name _;


    index index.php index.html index.htm index.nginx.htm;


    location / {

        try_files $uri $uri/ =404;

    }


    #pass PHP scripts to FastCGI server
```

```
    location ~ \.php$ {
        include snippets/fastcgi-php.conf;
        fastcgi_pass unix:/run/php/php8.1-fpm.sock;
    }

    location ~ /\.ht {
        deny all;
    }
}
```

Restart Nginx:

```
sudo systemctl restart nginx
```

22.5.6. Test the Stack

To test the stack, create a simple PHP file and make sure that the page renders properly.

Create a file named info.php in the /var/www/html containing a simple phpinfo call.

```
sudo chmod -R 777 /var/www/html

echo "<?php phpinfo(); ?>" >> /var/www/html/info.php
```

Visit: http://localhost/info.php and the PHP info page should appear confirming the details of the installation.

22.5.7. Key takeaways

All the necessary components for this application are easily accessible as open source and can easily be installed on any given Linux system. While all are available as binaries, they can also be built from source if necessary. Integration and assembly of the software stack is largely achieved by simple scripting since the APIs are standard. Arm's Learning Path provides a simple framework to get you started.

23. Example Application Two – Video Transcoding

KEY RESOURCE: RUN X265 ON ARM SERVERS (https://learn.arm.com/learning-paths/servers-and-cloud-computing/codec/**)**

This Learning Path is an introduction for software developers who want to build and run an x265 codec on Arm servers and measure performance.

In this example, we will build and test a video transcoder application. This will take a short video file and encode it using a standard encoding scheme. The resulting encoded file is significantly smaller than the original, with minimal loss of image quality.

23.1. Pre-Requisites

You will need at least one Arm-based instance, either from a cloud service provider or an on-premises server.

This example assumes that the platform is running Ubuntu (20.04 or later), though the technique should work with any mainstream Linux variant. We will also be building the codec direct from source code and we will use `gcc` and `cmake` for this (instructions for installing these are included and may be ignored if they are already installed).

23.2. High-Level System Design

The codec used for this example is x265. This is an open-source H.265/HEVC codec, designed by the Motion Picture Experts Group (MPEG). Although, when compared to its predecessor H.264, it requires greater processing power when running, H.265 offers greater data compression for the same video quality. Recent efforts have optimized this

package for Neoverse platforms, particularly those platforms which support the NEON instruction set.[46]

The source code repository is hosted on Bitbucket (bitbucket.org).

23.3. Platform Selection and Configuration

In addition to the considerations outlined in §22.3 in relation to the webserver example, note that a codec application will likely, by its very nature, require:

- greater input and output bandwidth for the incoming and outgoing video stream;
- more and faster local storage for frame buffers;
- greater, and more specialized processing power.

These will be important considerations when selecting a platform.

Additionally, H.265 is capable of making use of parallel processing in a way in which its predecessor cannot. Specifically, the input frame is divided into regions which can be processed entirely separately, on separate processors which share the same frame buffer. This means that multicore platforms should perform better for H.265.

23.4. Component Selection

Component	Supported on Arm	Version to use	Source
x265	Since 2020	3.4 or later	https://bitbucket.org/multicoreware/x265_git/wiki/Home
Gcc	Since 2018	15.0 or later	Available on all Linux distros
cmake	Since 2021	3.19.3 or later	Available on all Linux distros

46 NEON, a 128-bit wide SIMD instruction set, has been an optional component of the Arm architecture since Armv7-A. At the time of writing, all available Neoverse processors, as well as Armv8 and Armv9 processors built by licensees, support NEON.

Links to Arm and third-party documentation on these components can be found in the Arm Software Ecosystem Dashboard on Arm's website.

The links in the table above are to open-source versions of the relevant components. In many cases, commercial versions are also available.

23.5. Build, Install, Deploy

23.5.1. Install `gcc`

The following instructions assume that you are using Ubuntu. Instructions for other variants of Unix can be found at https://learn.arm.com/install-guides/gcc/native/.

```
sudo apt update

sudo apt install gcc g++ -y

sudo apt install build-essential -y
```

Confirm that the installation is complete and determine the version:

```
gcc –version
```

23.5.2. Install `cmake`

```
sudo apt update

sudo apt install wget git cmake cmake-curses-gui build-essential -y
```

23.5.3. Download and Build `x265`

Clone the repository from bitbucket.org

```
git clone https://bitbucket.org/multicoreware/x265_git.git

cd x265_git/build/linux
```

Set default flags for the build, then run `make` to build the codec.

```
./make-Makefiles.bash

Make
```

23.5.4. Download Test Video Streams

Video streams which can be used for testing the codec are taken from the publicly available Google User Generated Content (UGC) dataset. This dataset comprises some 1500 video clips, totaling 500 minutes of content of varying quality, bitrate, resolution, and content. We will use the 360P and 1080P files.

```
wget https://storage.googleapis.com/ugc \
     dataset/original_videos/Sports/360P/Sports_360P-02c3.mkv

wget https://storage.googleapis.com/ugc \
     dataset/original_videos/Sports/1080P/Sports_1080P-0640.mkv
```

23.5.5. Run the Codec and Measure the Performance

The following command runs the codec on 50 frames of 360P video:

```
./x265 --preset ultrafast --frames 50 Sports_360P-02c3.mkv \
     --input-res 640x360 --fps 24 --output outfile.265
     --frame-threads 1 --no-wpp --pools ','
```

The following command runs the codec on 50 frames of 1080P video:

```
./x265 --preset ultrafast --frames 50 Sports_1080P-0640.mkv \
     --input-res 1920x1080 --fps 24 --output outfile.265 \
     --frame-threads 1 --no-wpp --pools ','
```

Note the following options which are used to control the level of concurrency employed by the codec:

--frame-threads 1

This option specifies that one frame at a time is processed, with no concurrency. Due to the improved motion compensation, quality will generally be slightly higher and compression slightly better. However, performance is reduced as multi-processing and/or multi-threading cannot be used. Other values are possible: a value of 0 instructs the program to auto-detect and select an appropriate level of concurrency; values between 1 and 16 set the required level. Experimentation will show the optimal level for your platform. You should find that, beyond a certain point, memory use will increase but performance will not.

--no-wpp

This disables Wavefront Parallel Processing (WPP), preventing the codec from beginning to process a new row before previous rows have been completed. Removing it (or specifying `--wpp`) will increase potential parallelism at the expense of a slight reduction in compression efficiency.

--pools

This option has (fairly complicated) syntax allowing the user to specify the number of threads allocated to each node. The default (used here) specifies that no thread pool is created.

More detail on these and the many more command line options can be found in the official documentation at: https://x265.readthedocs.io/en/master/introduction.html.

The output from the command will display what parameters were used (including default values for those not specified on the command line), together with metrics indicating the performance of the codec. The most useful metrics are usually reported in the last line:

```
Encoded 50 frames in 13.74s (3.64s fps), 15933.41 kb/s, Avg QP:
34.47
```

This line includes the:

- number of frames encoded;
- total time taken;
- calculated “frames per second” figure (fps);
- average data throughput in kilobytes/second (kb/s);
- average Quantization Parameter (QP), an indication of the compression efficiency - higher QP indicates more aggressive quantization, leading to greater compression but lower quality.

It is relatively easy to adjust several of these parameters and observe the differences in performance. However, to go further than this, some form of code optimization will be required. There will be more to say on this subject in §26 below.

23.5.6. Key Takeaways

Video transcoding is a highly complex and compute-intensive process. However, easily accessible standard components make this technology easy to deploy. Standard installation tools and build scripts make building from source easy for any platform. To carry out tests, sample files are available online.

24. Example Application Three - AI Model Serving

KEY RESOURCE: DEPLOY A RAG-BASED CHATBOT (https://learn.arm.com/learning-paths/servers-and-cloud-computing/rag/**)**

This Learning Path is an introduction for software developers, ML engineers, and those looking to deploy production-ready LLM chatbots with Retrieval Augmented Generation (RAG) capabilities, knowledge base integration, and performance optimization for Arm Architecture.

In this example, we will build and test a simple Retrieval Augmented Generation (RAG)-enabled chatbot. We will use an open source LLM which has been optimized for Arm.

24.1. Pre-requisites

Due to the higher processing power requirements of LLM software, the requirements for the target platform are somewhat more demanding than the earlier examples. You will need access to an Arm64 platform with at least 16 cores, 8GB of RAM and 32GB of disk space.

The example assumes that you are running Ubuntu Linux.

24.2. High-Level System Design

The llama.cpp which is installed in this example makes use of Arm's Kleidi AI libraries, which are optimized for acceleration of Computer Vision (CV) and Machine Learning (ML) applications on Arm platforms.

You can find out more information about KleidiAI here:
https://www.arm.com/markets/artificial-intelligence/software/kleidi

24.3. Platform Selection & Configuration

You will need access to an Arm64 platform with at least 16 cores, 8GB of RAM and 32GB of disk space.

24.4. Component Selection

Component	Supported on Arm	Version to use	Source
Python	Since 2012	3.11.0 or later	Available on all Linux distros
llama-cpp-python	Since 2018	15.0 or later	abetlen.github.io/llama-cpp-python/whl/cpu
llama.cpp	Since 2021	3.19.3 or later	Installed with the above

Links to Arm and third-party documentation on these components can be found in the Arm Software Ecosystem Dashboard on Arm's website.[47]

The table above refers to the open source versions of the relevant components used in the example. In many cases, commercial versions are also available.

24.5. Build, Install, Deploy

24.5.1. Install Build Tools

The instructions for installing gcc and cmake can be found in §23.5.1 and §23.5.2.

24.5.2. Install Python

Run the following commands to install Python and relevant dependencies.

47 See https://developer.arm.com/ecosystem-dashboard/

```
sudo apt update

sudo apt install python3-pip python3-venv cmake -y
```

24.5.3. Create a Requirements File

Using a text editor, add the following to `requirements.txt`.

```
# Core LLM & RAG Components

langchain==0.1.16

langchain_community==0.0.38

langchainhub==0.1.20


# Vector Database & Embeddings

faiss-cpu

sentence-transformers


# Document Processing

pypdf

PyPDF2

lxml


# API and Web Interface

flask

requests

flask_cors
```

```
streamlit


# Environment and Utils

argparse

python-dotenv==1.0.1
```

24.5.4. Create, Activate and Install the Python Environment

Run the following commands:

```
python3 -m venv rag-env

source rag-env/bin/activate

pip install -r requirements.txt
```

24.5.5. Install llama.cpp

The following command installs the llama.cpp backend, included in the Python binding.

```
pip install llama-cpp-python --extra-index-url \
     https://abetlen.github.io/llama-cpp-python/whl/cpu
```

24.5.6. Download and Build the Model

```
mkdir models

cd models

wget https://huggingface.co/chatpdflocal/\
     llama3.1-8b-gguf/resolve/main/ggml-model-Q4_K_M.gguf



cd ~

git clone https://github.com/ggerganov/llama.cp

cd llama.cpp

mkdir build

cd build

cmake .. -DCMAKE_CXX_FLAGS="-mcpu=native" -DCMAKE_C_FLAGS="-
     mcpu=native"

cmake --build . -v --config Release -j `nproc`


cd bin

./llama-quantize --allow-requantize ../../../models/\
     ggml-model-Q4_K_M.gguf ../../../models/\
     llama3.1-8b-instruct.Q4_0_arm.gguf Q4_0
```

The first set of commands downloads the model source, the second set of commands builds the library (to run on CPU only, with no GPU acceleration), and the third set quantizes the model.

Quantization is a technique used to reduce the file size and memory usage for a particular model. In this case, 4-bit quantization is used. By default, the model uses 32-bit floating-point weights. These weights, as well as being large, require considerably more computing resource to handle. Using a 4-bit integer representation reduces the memory requirements as well as speeding up the arithmetic considerably as integer math can be

used. Quantization is always a trade-off between size and accuracy, since quantization loses some of the resolution of each weight.

24.5.7. Configure and Start the Backend Server

Using a text editor, create a file backend.py with the following content.[48]

```
import os

import time

import logging

from flask import Flask, request, jsonify

from flask_cors import CORS

from langchain_community.vectorstores import FAISS

from langchain_community.embeddings import HuggingFaceEmbeddings

from langchain_community.llms import LlamaCpp

from langchain_core.callbacks import StreamingStdOutCallbackHandler

from langchain_core.prompts import PromptTemplate

from langchain_community.document_loaders import PyPDFLoader,\
    DirectoryLoader

from langchain_text_splitters import HTMLHeaderTextSplitter,\
    CharacterTextSplitter

from langchain.schema.runnable import RunnablePassthrough

from langchain_core.output_parsers import StrOutputParser

from langchain_core.runnables import ConfigurableField
```

48 The source text for this script can be found at https://learn.arm.com/learning-paths/servers-and-cloud-computing/rag/backend/. It can be copied from there and pasted into the file.

```
# Configure logging

logging.getLogger('watchdog').setLevel(logging.ERROR)

logger = logging.getLogger(__name__)


# Initialize Flask app

app = Flask(__name__)

CORS(app)


# Configure paths

BASE_PATH = "$HOME"

TEMP_DIR = os.path.join(BASE_PATH, "temp")

VECTOR_DIR = os.path.join(BASE_PATH, "vector")

MODEL_PATH = os.path.join(BASE_PATH, "models/llama3.1-8b-
     instruct.Q4_0_arm.gguf")


# Ensure directories exist

os.makedirs(TEMP_DIR, exist_ok=True)

os.makedirs(VECTOR_DIR, exist_ok=True)


# Token Streaming

class StreamingCallback(StreamingStdOutCallbackHandler):

    def __init__(self):

        super().__init__()

        self.tokens = []
```

```
        self.start_time = None


    def on_llm_start(self, *args, **kwargs):
        self.start_time = time.time()
        self.tokens = []
        print("\nLLM Started generating response...", flush=True)


    def on_llm_new_token(self, token: str, **kwargs):
        self.tokens.append(token)
        print(token, end="", flush=True)


    def on_llm_end(self, *args, **kwargs):
        end_time = time.time()
        duration = end_time - self.start_time
        print(f"\nLLM finished generating response in
     {duration:.2f}\
            seconds", flush=True)


def format_docs(docs):
    return "\n\n".join(doc.page_content for doc in
     docs).replace("Context:", "").strip()


# Vectordb creating API
@app.route('/create_vectordb', methods=['POST'])
def create_vectordb():
    try:
```

```
    data = request.json
    vector_name = data['vector_name']
    chunk_size = int(data['chunk_size'])
    doc_type = data['doc_type']
    vector_path = os.path.join(VECTOR_DIR, vector_name)


    # Process document
    chunk_overlap = 30
    if doc_type == "PDF":
        loader = DirectoryLoader(TEMP_DIR, glob='*.pdf',\
             loader_cls=PyPDFLoader)
        docs = loader.load()
    elif doc_type == "HTML":
        url = data['url']
        splitter = HTMLHeaderTextSplitter([
            ("h1", "Header 1"), ("h2", "Header 2"),
            ("h3", "Header 3"), ("h4", "Header 4")
        ])
        docs = splitter.split_text_from_url(url)
    else:
        return jsonify({"error": "Unsupported document type"}),
400


    # Create vectorstore
    text_splitter = CharacterTextSplitter(
        chunk_size=chunk_size,
```

```
            chunk_overlap=chunk_overlap

        )

        split_docs = text_splitter.split_documents(docs)

        embedding = HuggingFaceEmbeddings(model_name="thenlper/gte-
    base")

        vectorstore = FAISS.from_documents(documents=split_docs,\
             embedding=embedding)

        vectorstore.save_local(vector_path)



        return jsonify({"status": "success", "path": vector_path})

    except Exception as e:

        logger.exception("Error creating vector database")

        return jsonify({"error": str(e)}), 500


# Query API

@app.route('/query', methods=['POST'])

def query():

    try:

        data = request.json

        question = data['question']

        vector_path = data.get('vector_path')

        use_vectordb = data.get('use_vectordb', False)



        # Initialize LLM

        callbacks = [StreamingCallback()]

        model = LlamaCpp(
```

```
        model_path=MODEL_PATH,

        temperature=0.1,

        max_tokens=1024,

        n_batch=2048,

        callbacks=callbacks,

        n_ctx=10000,

        n_threads=64,

        n_threads_batch=64

    )



    # Create chain

    if use_vectordb and vector_path:

        embedding = HuggingFaceEmbeddings(model_name=\
            "thenlper/gte-base")

        vectorstore = FAISS.load_local(vector_path, embedding,\
             allow_dangerous_deserialization=True)

        retriever =
vectorstore.as_retriever().configurable_fields(

            search_kwargs=ConfigurableField(id="search_kwargs")

        ).with_config({"search_kwargs": {"k": 5}})



        template =
"""<|begin_of_text|><|start_header_id|>system<|end_header_id|>

        You are a helpful assistant. Use the following context
to\
             answer the question.

        Context: {context}

        Question: {question}
```

```
        Answer: <|eot_id|>"""


        prompt = PromptTemplate(template=template,\
            input_variables=["context", "question"])

        chain = (

            {"context": retriever | format_docs, "question":\
                RunnablePassthrough()}

            | prompt

            | model

            | StrOutputParser()

        )

    else:

        template =
"""<|begin_of_text|><|start_header_id|>system<|end_header_id|>

        Question: {question}

        Answer: <|eot_id|>"""


        prompt = PromptTemplate(template=template,\
             input_variables=["question"])

        chain = RunnablePassthrough().assign(question=lambda x:
x) |\
             prompt | model | StrOutputParser()


    # Generate response

    response = chain.invoke(question)

    return jsonify({"answer": response})

except Exception as e:

    logger.exception("Error processing query")
```

```
        return jsonify({"error": str(e)}), 500


# File Upload API
@app.route('/upload_file', methods=['POST'])
def upload_file():
    try:
        file = request.files['file']
        if file and file.filename.endswith('.pdf'):
            filename = os.path.join(TEMP_DIR, "uploaded.pdf")
            file.save(filename)
            return jsonify({"status": "success", "path": filename})
        return jsonify({"error": "Invalid file"}), 400
    except Exception as e:
        logger.exception("Error uploading file")
        return jsonify({"error": str(e)}), 500


if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000, debug=True)
```

Run the following command to start the backend server.

```
python3 backend.py
```

The output should confirm that the server is now running. You are now ready to configure and start the frontend server.

24.5.8. Configure the Frontend Server

Using a text editor, create a file `frontend.py` with the following content.[49]

```
import os

import requests

import time

import streamlit as st

from PIL import Image

from typing import Dict, Any


# Configure paths and URLs

BASE_PATH = "$HOME"

API_URL = "http://localhost:5000"


# Page config

st.set_page_config(

    page_title="LLM RAG on Arm Neoverse CPU"

)


# Title

st.title("LLM RAG on Arm Neoverse CPU")
```

49 The text for this script can be found at: https://learn.arm.com/learning-paths/servers-and-cloud-computing/rag/frontend/. It can be copied from there and pasted into the file.

```
# Sidebar

with st.sidebar:

    st.write("## Model Settings")

    model = st.selectbox('Select LLM', ["llama3.1-8b-
     instruct.Q4_0_arm.gguf"])

    use_vectordb = st.checkbox("Use Vector Database")


# Initialize session state

if 'messages' not in st.session_state:

    st.session_state.messages = []

if 'vectordb_path' not in st.session_state:

    st.session_state.vectordb_path = None


# Vector Database Creation

if use_vectordb:

    st.sidebar.write("## Vector Database")

    # First select vector store type

    vector_store = st.sidebar.selectbox("Vector Storage Type",
     ["FAISS"])


    # Then select action

    action = st.sidebar.radio("Action", ["Create New Store", "Load \
     Existing Store"])


    if action == "Create New Store":

        source = st.sidebar.radio("Source", ["PDF"])
```

```
    if source == "PDF":

        uploaded_file = st.sidebar.file_uploader("Upload PDF",\
            type="pdf")

        if uploaded_file:

            files = {'file': uploaded_file}

            response = requests.post(f"{API_URL}/upload_file",\
                 files=files)

            if response.ok:

                st.sidebar.success("File uploaded successfully")


                db_name = st.sidebar.text_input("Vector Index
Name")

                if st.sidebar.button("Create Index"):

                    response = requests.post(

                        f"{API_URL}/create_vectordb",

                        json={

                            "vector_name": db_name,

                            "chunk_size": 400,

                            "doc_type": "PDF"

                        }

                    )

                    if response.ok:

                        st.session_state.vectordb_path = \
                             response.json()['path']

                        st.sidebar.success("Vector Index
created!")

                    else:
```

```
                        st.sidebar.error("Failed to create
vector \
                                Index")


else:  # Load Existing
    # Updated directory handling
    vector_dir = os.path.join(BASE_PATH, "vector")
    if os.path.exists(vector_dir):
        # Get all directories that contain FAISS index files
        dbs = []
        for root, dirs, files in os.walk(vector_dir):
            if "index.faiss" in files:  # Check for FAISS index
file
                # Get relative path from vector_dir
                rel_path = os.path.relpath(root, vector_dir)
                dbs.append(rel_path)
        if dbs:
            selected_db = st.sidebar.selectbox("Select Index",
dbs)
            st.session_state.vectordb_path =
os.path.join(vector_dir,\
                 selected_db)
            st.sidebar.success(f"Loaded index: {selected_db}")
        else:
            st.sidebar.warning("No existing indexes found. \
Please create a new one.")
    else:
        # Create vector directory if it doesn't exist
```

```
            os.makedirs(vector_dir, exist_ok=True)

            st.sidebar.warning("No indexes found. \
     Please create a new one.")


# Chat interface

if use_vectordb and action == "Load Existing Store" and dbs:

    if prompt := st.chat_input("Ask a question"):

        st.session_state.messages.append({"role": "user", "content":
     \
             prompt})


        # Display messages

        for msg in st.session_state.messages:

            with st.chat_message(msg["role"]):

                st.write(msg["content"])


        # Get response

        with st.chat_message("assistant"):

            response = requests.post(

                f"{API_URL}/query",

                json={

                    "question": prompt,

                    "vector_path": st.session_state.vectordb_path,

                    "use_vectordb": use_vectordb

                }

            )
```

```
            if response.ok:
                answer = response.json()['answer']
                st.write(answer)
                st.session_state.messages.append({"role":
    "assistant",\
                     "content": answer})
            else:
                st.error("Failed to get response from the model")
```

Run the following command to start the frontend server:

```
python3 -m streamlit run frontend.py
```

The output will indicate that the server is running and display a URL at which the app can be accessed via a browser.

24.5.9. Access the Web Application

Point your browser at one of the URLs displayed by the frontend server in the previous step.

24.5.10. Upload a PDF File as Source Material, Index it and Load the Resulting Index

Select a PDF file to use as the source material for the model and follow these steps to load it, create and index and load the index into the model.[50]

50 Sample output from this step can be seen at: https://learn.arm.com/learning-paths/servers-and-cloud-computing/rag/chatbot/

1. Open a web browser and navigate to the Streamlit frontend.
2. In the sidebar, select **Create New Store** under the **Vector Database** section.
3. By default, **PDF** is the source type selected.
4. Upload your PDF file using the file uploader.
5. Enter a name for your vector index.
6. Click the **Create Index** button.
7. Switch to the **Load Existing Store** option in the sidebar.
8. Select the index you created. It should be auto-selected if it is the only one available.

24.5.11. Interact with the LLM

You can now enter queries into the prompt field and observe the response from the LLM.

As you do this, you can see performance metrics output on the backend terminal. These provide information about the performance, efficiency and processing speed of the model.

24.5.12. Key Takeaways

This is the most complex of the three examples to integrate and deploy. Even so, much of the process is standard and the necessary scripts are easily generated from online templates.

25. Development Tools

25.1. Overview

In this section, we deal only with the selection of appropriate tools for a particular application development. Configuration of the tools will be covered in §26 on performance optimization. As mentioned there, the platform on which the application is deployed forms a significant component in the optimization process.

When looking to select appropriate tools for your development process, remember that the build environment and the execution environment may be different. For instance, a cloud-native development flow will develop, debug, optimize and deploy on the same platform, or at least platforms which, if not identical in every respect, share the same basic architecture; alternatively, you may be developing and compiling on one platform and then deploying to another (what is usually referred to as 'cross-compilation"), in which case you will be choosing a compiler that runs on one architecture and debugger/profiler tools that run on another.

When choosing tools that run on an Arm-based platform, refer to the list of supported components on Arm's website.[51]

25.2. Compiler

There are a large number of options when selecting a compiler for your development. The Software Ecosystem Dashboard for Arm gives a comprehensive list of them. In general, you will find that all the mainstream options are supported on Arm, either for native compilation or cross-compilation. Note carefully the information about which version of each tool supports which version of the Arm architecture.

In addition to many commercial and/or proprietary compilation toolchains, public domain tools are widely available for Arm.

---

51 Software Ecosystem Dashboard for Arm: https://developer.arm.com/ecosystem-dashboard/

Component	Supported	Version to use	Source
GNU (gcc)	Since 2018	15 or later	Available on all Linux distros
Clang	Since 2024	19.1.0 or later	Available for all Arm Linux distros

For high-performance use cases, consider Arm Compiler for Linux (ACfL). Packaged and tested by Arm, ACfL is based on the latest LLVM and contains some specific extensions which may benefit certain high-performance use cases. For more detail, including comparative performance metrics against specific benchmarks, see the relevant page on Arm's website.[52]

Many providers of Arm Neoverse cores also provide and support compilers which are tailored specifically to their products. For instance, NVIDIA supports optimized builds of the LLVM Clang compiler which exploit the specific architectural configuration of the Grace CPU as well as the particular implementation choices made by NVIDIA. See their website for more details.[53]

Regardless of which compiler you choose, it cannot be over-emphasized that configuration of the compilation process is very important. Failure to configure the compilation to match the target platform will result in sub-optimal performance. There is more detail on configuration of the compiler in §26.2.2 below.

25.3. Debugger

The debug facilities available on a Neoverse core depend on several factors:

52 Choosing Compilers for HPC on Arm: https://developer.arm.com/community/arm-community-blogs/b/servers-and-cloud-computing-blog/posts/choosing-compilers-for-arm-hpc

53 Clang for NVIDIA Grace: https://developer.nvidia.com/grace/clang

- the debug components supported by the architecture of the platform – while all architectures support program trace, the precise nature and scope of that support varies;
- the decisions made by the implementer as to which hardware debug components to include, how to configure them, and how to make them available to the developer (e.g., for security reasons some facilities may be restricted to securely-connected and authenticated debuggers);
- the support for these features implemented in the selected debug tool.

Arm cores, including Neoverse, support two basic methods of debug: self-hosted, and JTAG. Self-hosted debug employs a resident software monitor program that executes on the target device and communicates with the debugger to provide control and visibility of the application. JTAG debug requires physical connection of a hardware probe to the target system and, as such, is unlikely to be available in a cloud deployment.

Self-hosted debug is intrinsically "non-invasive" in the sense that it never halts the target processor (since the debug monitor executes on the target), instead diverting the execution flow away from the application under debug and into the monitor so that debug facilities can be provided.[54] In theory, self-hosted debug is capable of providing the ability to debug applications, operating systems, and hypervisors. In a cloud environment, it is likely that only application debug will be enabled, due to the need to prevent simultaneous and collocated users interfering with each other's operations when sharing cloud hardware.

More detail on how Arm cores implement self-hosted debug is available on Arm's website.[55]

Component	Supported	Version to use
Gnu Debugger (GDB)	Since April 2013	14.1+

54 Note, though, that since the execution flow is diverted to provide debug functions, the timing of application execution will be affected to some degree.

55 Learn the architecture – Aarch64 self-hosted debug –https://developer.arm.com/documentation/102120/0101/Overview

In addition to traditional debugging tools, there are several logging libraries and APIs which can provide in-execution logging and event tracking to aid debugging.

Component	Supported	Version to use
Google Logging Library (glog)	Since March 2019	0.7.0 or later
Log4cplus	Since April 2018	1.1.2 or later
Log4cpp	Since 2013	1.1.1 or later

25.4. Profiler

A profiler is a tool which helps identify the most frequently executed portions of an application. This can be especially useful when deciding how to deploy limited resource for optimization. Identifying the portions of the code that are executed most often helps to narrow down those code sections which should be the focus for the majority of optimization effort. It should be easy to see that saving a single cycle from a routine that is executed a million times in the course of a transaction is more impactful than removing a thousand cycles from a routine that is executed only a handful of times.

Several public domain tools are available.

Component	Supported	Version to use
gPerfTools	Since August 2015	2.10.80 or later
Perf	Since August 2018	As installed with apt[56]

By default, the GNU compiler does not generate information for analysis with profiling tools. When using the GNU `prof` tool, this must be enabled using the gcc compile flag `-p`. Profiling data for `gprof` can be generated using the gcc flag `-pg`.

25.5. Arm Performix

KEY RESOURCE: ANALYZE & OPTIMIZE WORKLOADS ON ARM NEOVERSE:
https://developer.arm.com/servers-and-cloud-computing/arm-performix

This page describes the Arm Performix tool and information on how to join the Early Access Program.

Arm Performix is a performance analysis tool specifically targeted at developers building and running workloads on Neoverse platforms. The tool collects performance data directly from performance counters built into the hardware at run-time, which is then presented in a form which enables swift determination of performance issues and bottlenecks.

The tool is made available to developers and can be downloaded from the page linked above.

56 The best version depends on the particular Linux distro you are using. apt will install the most appropriate version as part of the linux-tools package. Some versions of Ubuntu come with perf already installed.

26. Performance Analysis and Optimization

There is a wealth of material on this subject, both direct from Arm and from a wide range of other sources. The first linked resource below is to the Arm Learning Paths on the topic of "Performance and Architecture". While there may seem to be a daunting number of individual topics in that list, they will reward further study.

KEY RESOURCE: ARM LEARNING PATHS: https://learn.arm.com/learning-paths/servers-and-cloud-computing/?subjects=performance-and-architecture

This link is to an index of the set of Arm Learning Paths which are concerned with performance and the specifics of the Arm Architecture.

KEY RESOURCE: TUNING NGINX: https://learn.arm.com/learning-paths/servers-and-cloud-computing/nginx_tune/

This is a more advanced topic for software developers who want to use Nginx on Arm, covering the performance impact of kernel parameters, compilers and libraries, and Nginx configuration.

KEY RESOURCE: OPTIMIZATION METHODOLOGY NEOVERSE V1: https://community.arm.com/arm-community-blogs/b/servers-and-cloud-computing-blog/posts/arm-neoverse-v1-top-down-methodology

This Arm whitepaper explains the Arm Neoverse V1 Performance Analysis Methodology.

KEY RESOURCE: PROFILE GUIDED OPTIMIZATION (PGO)

https://learn.arm.com/learning-paths/servers-and-cloud-computing/cpp-profile-guided-optimisation/how-to-1/

This document is for developers looking to optimize C++ performance based on runtime behavior, using benchmarks and Profile Guided Optimization..

KEY RESOURCE: EXPLORE PERFORMANCE GAINS BY INCREASING THE LINUX KERNEL PAGE SIZE ON ARM: https://learn.arm.com/learning-paths/servers-and-cloud-computing/arm_linux_page_size/

26.1. Overview

"Optimization" is the process of improving a software application to make it more efficient and effective. There are some important things to note about this statement.

First, optimization is an iterative process. It is unlikely that you will achieve the best possible result at the first attempt. Your process will be one of continually modifying and measuring to determine which actions take you closer to the desired performance.

Second, this means that you will need to define some realistic and repeatable performance metrics which can guide each step of the process. This will entail the definition of realistic usage scenarios, together with a means to measure performance.

Third, the source code is not the only factor to consider. The platform on which it is running can have a significant effect on the performance of the application, as can the tools used to build it, and the libraries and other components from which it is constructed.

Finally, "performance" is a multi-faceted term. It is instinctive to think of performance being equated to execution speed. However, execution speed is not the only performance metric in which you might be interested. For instance, any or all of memory footprint, I/O bandwidth, code and/or data size, and power efficiency might be significant for you. We examined some of these aspects in greater detail in §20.1 above.

In this section, we take the assumption that code size is not a significant consideration. Data size is likely to be more important when deploying to a particular platform. However, matching data layout with algorithm design and memory architecture is likely to be more significant still.

For instance, when iterating over arrays, cached memory systems generally provide better performance if the innermost loop iterates over the rows. This leverages much better the performance benefits provided by caches. For more information on this subject, which can become complex, see standard material on loop switching, strip mining, tiling, and preloading as they relate to matrix operations on data held in cached memory systems.

When optimizing, it is important that the metrics generated from each separate execution are comparable. In other words, as far as possible, the operating environment should be consistent, the input parameters should be the same, and network traffic should be comparable with earlier runs. Tools such as `tcpreplay` can be used to record and replay network traffic, making it easier to compare results from each test run.[57]

Bear in mind that Arm, in common with all in the ecosystem, are constantly innovating and improving the performance and capability of their products. This makes it all the more important to ensure that you are working with the latest versions of other software components and development tools.

26.2. Things You Can Optimize

When starting on the process of optimization, it is tempting to go straight to the source code of your application and start there. Remember that your source code is only one component in the optimization equation and that many other factors affect the performance of your application. In this section, we consider some of those other factors.

26.2.1. Platform

As we have seen throughout this text, there is a huge variety of Arm-based platforms available, all with different characteristics. Choosing the right platform is key to a successful deployment (remembering that "success" is not just defined as having a

57 See https://tcpreplay.appneta.com/ for further information on this tool.

working application, but includes other factors such as commercial, regulatory, availability, etc.).

At this point, it may be worth revisiting §5 and §6, which cover the Neoverse cores themselves and the available implementations of them from a wide selection of providers. Recall that the properties of any particular implementation depend on choices made by the implementer, as well as the underlying Arm architecture used.

The example applications presented above all contain some discussion around the selection of a suitable platform for the specific application involved.

26.2.2. Tools

The pairing between platform and development tools is particularly important, as is the configuration of the chosen tools.

Again, pay particular attention to the discussion around architectural and implementational choices, particularly in §5.1.1 "Architecture, Implementation and Manufacturing". It should be clear that configuring the tools for the right architecture is only part of the story. Doing so will enable the tools to make use of all available instructions which may accelerate certain parts of your application. Configuring for the exact implementation you are using is also important as this gives the tools information about the microarchitecture of the platform. This includes details such as the pipeline structure, prefetch capabilities, superscalar characteristics etc. Knowledge of these details allows the tools to generate machine instructions in particular sequences and combinations, which will allow the core to reach higher instruction throughput in certain circumstances.

As a minimum, you need to configure the tools for the correct Arm architecture of the target.[58] Remember that each architecture has some optional features and you need to include these otherwise the compiler will not be able to make use of the additional instructions. Here are some examples:

---

58 We assume in this section that you are using GCC and the options shown are for that compiler and associated tools. Alternative tools will have similar options but may vary in the syntax used.

```
   -march=armv9-a        // Configures for a generic Armv9-A
processor

   -march=armv9-a+sve2   // Armv9-A with SVE2 extensions
```

It is even better to configure for the exact implementation being used. Here are some examples:

```
   -mtune=neoverse-v1    // Generic Neoverse V1

   -mtune=cobalt-100     // Microsoft Cobalt 100
```

A special case can be used when you are building and deploying on the same platform (the two examples shown are synonymous in the case where the compiler is able to determine the precise architecture and implementation of the host processor):

```
   -mcpu=native    // Build for the host CPU

   -mtune=native   // Build for the host CPU
```

There are many other modifiers for architecture and CPU which indicate the presence (or absence) of particular features. All are given in the official GCC documentation.[59]

There are many architecture and features modifiers documented in the GCC manual.

Some other options control the characteristics of the compilation process, such as degree and type of optimization. Here are some examples:[60]

---

59 The official documentation for GCC on AArch64 platforms is here: https://gcc.gnu.org/onlinedocs/gcc/AArch64-Options.html

60 Generic GCC optimization options are here: https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html

```
   -O0       // Disable most optimizations, can be useful when
debugging

   -O3       // Maximum optimization (levels 1 and 2 are also
available)

   -Ofast    // Optimize for speed

   -Og       // Optimize for debug


   -finline-functions          // Inline functions when appropriate

   -finline-small-functions    // Inline all small functions[61]


   -funroll-loops  // Unroll and peel all loops when possible[62]
```

26.2.3. Operating System

Linux exposes many configuration parameters, which can be changed via the `sysctl` interface. In general, the default values will have been selected for best performance on general workloads and there will be no need to change them. If changes are required, they are best carried out by advanced users.

One parameter which may be of greater interest when developing on an Arm platform is the page size in the virtual memory system. Arm processors support page sizes of 4 KB, 16 KB and 64 KB. Other architectures also support variable page sizes, but the available options are not necessarily as useful. In most Arm Linux distributions, the default size is 4 KB. This minimizes overhead in memory allocation but achieves this at a potential cost in TLB pressure and I/O transfer speed (DMA accesses, for instance, are required to restart every time a page boundary is crossed). The alternative page sizes of 16 KB and 64 KB may provide better performance for some workloads. While very few kernels support the 16 KB option, most support a change to 64 KB. If your workload deals with large,

61 "Small" in this context means all functions which are smaller than the associated call-return overhead.

62 Loops whose iteration count is determined at compile time will be completely unrolled. They will also be "peeled" (i.e., loops with a small, constant iteration count will be removed completely). The resulting code may run more quickly but will always be larger.

contiguous data structures in memory (e.g., frame buffers) then a switch to the larger page size can result in increased performance.

There is an Arm Learning Path which details how to change this parameter.[63]

26.2.4. Other Components

Remember that your source code is only one part of what is eventually compiled and linked into your final application. Other components such as libraries are included too.

The default is to use the standard libraries for your toolchain and target platform. If the build process is configured appropriately, then these will be chosen and included automatically.[64] However, specialist libraries exist to support particular types of application and platform. There are several possible sources for these.

Platform-specific

As we have learned, platforms differ greatly in their behavior and performance. Platform-specific libraries are optimized for a particular platform. Generally, these will be included by default if the tools are configured for the correct platform. Individual providers may make proprietary libraries tailored to their own platforms available and these should be used in preference whenever possible.

Vendor-specific

To support their CPU and platform offerings, vendors provide suites of software that are often quite extensive, including custom libraries, tools, etc. Again, where available, these should be used by default and selected individually if not included by the tools by default.

Application-specific

Some types of application make use of highly specific algorithms to achieve their goals. For instance, Artificial Intelligence, Computer Vision, and Machine Learning applications

63 See https://learn.arm.com/learning-paths/servers-and-cloud-computing/arm_linux_page_size/

64 See §26.2.2 above.

process data in a highly specific way using complex algorithms. Optimizing for these is difficult and often requires specialist knowledge of the algorithms involved and the performance characteristics of the platform. Fortunately, there are many libraries available which have already been optimized for a variety of Arm platforms. Examples include OpenCV[65] (an open-source library for Computer Vision applications), and Tesseract[66] (an Optical Character Recognition engine). Many are listed on the Software Ecosystem Dashboard for Arm.[67]

Arm maintains a number of optimized libraries - in particular, the Arm Kleidi libraries have been shown to improve performance for a wide variety of platforms and applications. Documentation and source for KleidiAI (optimized critical routines for AI workloads) and KleidiCV (a library of high-performance image processing functions) can be found on Arm's GitLab.[68]

26.2.5. Source Code

Once you have the right tools and libraries in place, and they are configured correctly for your target platform, your source code itself is the next thing to examine.

Beware that source code optimization is time-consuming, complex, involves repeated regression testing, and frequently requires detailed domain knowledge and expertise. Because it is time-consuming, it is important to focus efforts on the areas of the source code which will deliver the greatest returns. Identifying these areas is a major component of the optimization process.

Frequently, the most helpful tool is the profiler. A profiler helps to identify the parts of an application which are executed most, in which the application spends the majority of its time. It is likely that improvements to these areas will yield the greatest overall performance increase. A small improvement in a portion of the application which is executed many times will likely yield a greater performance change in the overall

65 OpenCV documentation is available here: https://docs.opencv.org/4.x/index.html

66 Tesseract documentation is here: https://github.com/tesseract-ocr

67 Here: https://developer.arm.com/ecosystem-dashboard/

68 Here: https://gitlab.arm.com/kleidi

application than a huge improvement (often achieved at great effort) in a part which is only executed a few times.

Profilers either make use of hardware profiling facilities (e.g., the performance counters which are implemented in many Arm cores) or add software instrumentation (via modifying the source code during compilation and/or by linking profiling-enabled libraries) in order to produce output indicating where an application spends its time.

An example from Arm is the Streamline Performance Analyzer.[69] See §26.2.6 below for a discussion on how a profiler can be used to inform the compilation step in what is known as "Profile-Guided Optimization".

How to optimize at source code level and in assembly language is beyond the scope of this document. This is almost always highly specific to the platform in use and is likely to result in non-portable source code.

In addition to resources published by device providers (such as the guide to Graviton referenced in §126.2.7 below), Arm makes optimization guides available for many of its cores. Details and documentation are available on Arm's website.[70] Be aware that the optimization strategies described in these documents are very low level and are often only relevant to those who are developing compilers and libraries for specific domains.

26.2.6. Profile-Guided Optimization

Profile-guided optimization is a standard technique which uses the profiler and compiler in combination better to utilize the optimizations available from the compiler. As mentioned above, one of the most difficult aspects of optimization is identifying the portions of source code which will give the greatest return for effort. A similar problem exists when trying to determine the best compiler optimization configuration. It might be easy simply to configure maximum optimization across the entire body of source code. However, this may result in a significant and unwelcome increase in compilation time (due to the extra work which the compiler has to carry out, possibly along with extra

69 Documentation here: https://developer.arm.com/Tools%20and%20Software/Streamline%20Performance%20Analyzer

70 See here: https://learn.arm.com/learning-paths/servers-and-cloud-computing/profiling-for-neoverse/additional-resources/

compilation passes), and may also result in a much larger image (for instance, due to a much greater amount of loop unrolling). It is much better to identify the portions of the application which would benefit from such aggressive optimization and then enable these options only for that part. A profiler can help to do this.

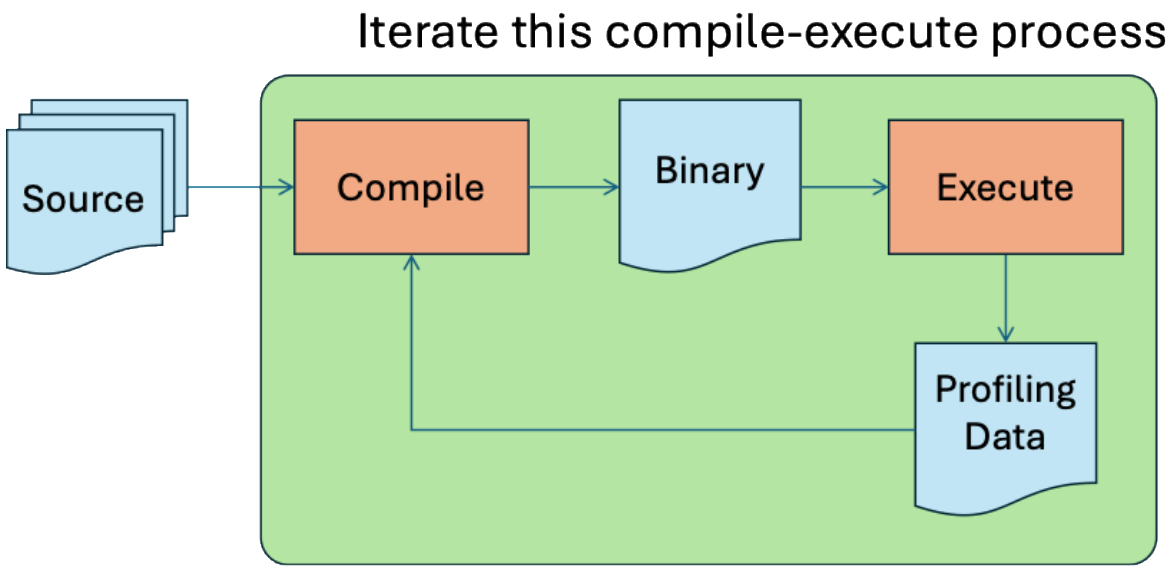

Figure 20 - Profile-guided Optimization

Figure 20 illustrates the process. Instrumented source code is compiled and then executed on the platform under the control of the profiler. This generates a file of profiling information which documents, among other things, the time spent in each portion of the application during the execution. This information is then fed back to the compiler and the compilation repeated. The compiler uses the profiling data to determine which optimizations are best to apply and where they are best applied. Although not always necessary, the compile-execute process can be iterated until no further gain is achieved.

Note that the execution environment and input data must be representative of real-world conditions for the profiling data to be relevant to real-world performance.

A profiler can also assist with coverage testing, providing evidence that all areas of the application have been exercised. This is often a requirement in regulated applications.

26.2.7. Vendor-Specific Optimization

Many vendors publish detailed guidance on optimization on their own proprietary platforms.

For instance, NVIDIA publish guidance on optimizing for the Grace CPU. See their website for further detail.[71] Similarly, AWS provide documented resources on optimizing for Graviton.[72]

71 Grace Performance Tuning Guide: https://docs.nvidia.com/grace-perf-tuning-guide/

72 Graviton Performance Runbook: https://github.com/aws/aws-graviton-getting-started/blob/main/perfrunbook/README.md

27. Summary

The purpose of this document is four-fold:

- To show how the Arm instances available in the cloud map on to common use cases;
- To discuss how to derive and measure performance characteristics;
- To provide worked examples which establish that the development processes when using Arm platforms are essentially the same as for competitor platforms;
- To give pointers to essential tools and techniques when developing, testing and optimizing applications on Arm platforms in the cloud.

The intention has been to show that developing on an Arm Neoverse cloud instance is no different to developing on alternative architectures. In general, the same range of tools, components, applications, operating systems, and libraries is available across the Neoverse range. Many of these components have been specifically optimized for the Neoverse architecture and will be as performant there as they are on other, non-Arm platforms.

Arm and its partner ecosystem have applied considerable effort to remove barriers and smooth the pathways to deploying an application on Arm-based platforms, leaving developers with the opportunity to maximize time spent working on their applications, optimizing them, and deploying them, to the greatest possible commercial success.

When starting work on writing, or porting, an application for a Neoverse platform, it is well worth spending time reviewing the extensive resources available on Arm's website. You will find specifications, how-to manuals, learning paths, pointers to tools and components, advice on development techniques, and more. There are also a wide variety of routes for engaging with experts from Arm and partner companies for specific advice.

Arm's success depends on the success of developers. Arm is here for you!

Proprietary notice

Arm Limited. Company 02557590 registered in England.
110 Fulbourn Road, Cambridge, England CB1 9NJ.
(PRE-1121-V1.0)